

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống học ngoại ngữ thông minh hỗ trợ
bởi AI Agent và kiến trúc GraphRAG

TRẦN DOÃN HUY

Huy.TD225859@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt–Nhật

Giảng viên hướng dẫn: TS. Nguyễn Nhất Hải

Khoa: Khoa học Máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng hệ thống học ngoại ngữ thông minh hỗ trợ
bởi AI Agent và kiến trúc GraphRAG

TRẦN DOÃN HUY

Huy.TD225859@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt–Nhật

Giảng viên hướng dẫn: TS. Nguyễn Nhất Hải

Chữ kí GVHD

Khoa:

Khoa học Máy tính

Trường:

Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới giảng viên hướng dẫn – TS. Nguyễn Nhất Hải, người đã tận tình định hướng, góp ý và hỗ trợ em trong suốt quá trình thực hiện đồ án tốt nghiệp. Những nhận xét và chỉ dẫn quý báu của thầy đã giúp em xác định rõ phạm vi nghiên cứu, lựa chọn kiến trúc phù hợp và từng bước hoàn thiện hệ thống theo hướng có tính ứng dụng thực tiễn.

Em xin cảm ơn các thầy cô Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã truyền đạt cho em nền tảng kiến thức về lập trình, cơ sở dữ liệu, công nghệ phần mềm, phát triển ứng dụng web, bảo mật hệ thống và trí tuệ nhân tạo. Những kiến thức này là cơ sở quan trọng để em có thể triển khai một hệ thống gồm nhiều thành phần như giao diện người dùng, dịch vụ backend, cơ sở dữ liệu quan hệ, cơ sở dữ liệu đồ thị và các pipeline AI.

Em cũng xin gửi lời cảm ơn tới gia đình và bạn bè đã luôn động viên, chia sẻ và tạo điều kiện để em tập trung hoàn thành đồ án. Trong quá trình xây dựng hệ thống, em gặp nhiều khó khăn liên quan đến tích hợp đa công nghệ, xử lý dữ liệu tiếng Nhật, kết nối giữa backend và AI layer, cũng như kiểm thử các chức năng tương tác. Sự hỗ trợ về tinh thần của mọi người là nguồn động lực lớn giúp em kiên trì hoàn thiện sản phẩm.

Do thời gian và kinh nghiệm còn hạn chế, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy cô để có thể tiếp tục hoàn thiện sản phẩm và nâng cao năng lực chuyên môn của bản thân.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh nhu cầu học tiếng Nhật ngày càng tăng, người học thường phải sử dụng nhiều công cụ rời rạc như từ điển, ứng dụng flashcard, tài liệu ngữ pháp, công cụ dịch và nền tảng luyện giao tiếp. Cách học phân tán này khiến người học khó theo dõi tiến độ, khó ôn tập đúng thời điểm và chưa tận dụng hiệu quả dữ liệu lỗi sai cá nhân. Các công cụ hiện có đã hỗ trợ tốt từng nhu cầu riêng lẻ, nhưng còn hạn chế trong việc kết nối dữ liệu học tập, cá nhân hóa nội dung ôn tập và giải thích nghĩa theo ngữ cảnh chuyên ngành.

Để giải quyết vấn đề trên, đồ án lựa chọn hướng tiếp cận xây dựng một hệ thống học tiếng Nhật thông minh, tích hợp các hoạt động tra cứu, ghi nhớ, ôn tập, luyện hội thoại và dịch thuật trong cùng một nền tảng. Hướng tiếp cận này được lựa chọn vì có thể tạo ra vòng học tập liên tục, trong đó dữ liệu từ quá trình tra cứu, flashcard, lỗi sai và phiên luyện tập được sử dụng lại để hỗ trợ người học hiệu quả hơn.

Giải pháp của đồ án gồm các chức năng chính: tra cứu từ vựng, kanji và ngữ pháp; quản lý bộ thẻ flashcard; ôn tập theo thuật toán lặp lại ngắt quãng SM-2; sinh trắc nghiệm và câu chuyện tương tác dựa trên bộ thẻ, cấp độ JLPT và lỗi sai gần đây; luyện hội thoại với AI Tutor qua văn bản hoặc giọng nói; và dịch chuyên ngành tiếng Nhật với sự hỗ trợ của đồ thị tri thức Neo4j kết hợp mô hình ngôn ngữ lớn.

Đóng góp chính của đồ án là thiết kế và triển khai một nguyên mẫu hệ thống học tiếng Nhật có khả năng kết nối frontend, backend, cơ sở dữ liệu và tầng AI trong một sản phẩm thống nhất. Kết quả đạt được là một hệ thống có thể hỗ trợ người học tra cứu, ghi nhớ, ôn tập, luyện giao tiếp và nhận phản hồi thông minh, tạo nền tảng để mở rộng thành trợ lý học ngoại ngữ cá nhân hóa trong tương lai.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	6
2.1 Khảo sát hiện trạng	6
2.1.1 Khảo sát nhu cầu của người dùng mục tiêu.....	6
2.1.2 Khảo sát các hệ thống và ứng dụng tương tự	7
2.2 Tổng quan chức năng	9
2.2.1 Biểu đồ use case tổng quát	9
2.2.2 Các biểu đồ use case phân rã.....	10
2.2.3 Quy trình nghiệp vụ	12
2.3 Đặc tả chức năng	13
2.3.1 Đặc tả use case tra cứu mục từ và thêm vào flashcard.....	13
2.3.2 Đặc tả use case ôn tập flashcard	14
2.3.3 Đặc tả use case luyện hội thoại với AI Tutor	15
2.3.4 Đặc tả use case dịch chuyên sâu	15
2.4 Yêu cầu phi chức năng	16
2.5 Tổng kết.....	16
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	18
3.1 Công nghệ phát triển frontend.....	18
3.2 Công nghệ backend và bảo mật	19
3.3 Lưu trữ dữ liệu quan hệ với PostgreSQL và JPA.....	19

3.4 Python, FastAPI và mô hình ngôn ngữ lớn	20
3.5 Neo4j, GraphRAG và SudachiPy.....	21
3.6 Thuật toán lặp lại ngắt quãng SM-2	21
3.7 Công nghệ tương tác giọng nói	22
3.8 Tổng kết.....	22
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	24
4.1 Thiết kế kiến trúc.....	24
4.1.1 Lựa chọn kiến trúc phần mềm	24
4.1.2 Thiết kế tổng quan.....	26
4.1.3 Thiết kế chi tiết gói	28
4.2 Thiết kế chi tiết.....	31
4.2.1 Thiết kế giao diện	31
4.2.2 Thiết kế lớp	36
4.2.3 Thiết kế cơ sở dữ liệu	40
4.2.4 Thiết kế module AI	45
4.3 Xây dựng ứng dụng.....	49
4.3.1 Thư viện và công cụ sử dụng	49
4.3.2 Kết quả đạt được	51
4.3.3 Minh họa các chức năng chính	53
4.4 Kiểm thử.....	57
4.4.1 Phương pháp và kỹ thuật kiểm thử	58
4.4.2 Kiểm thử chức năng ôn tập flashcard theo SM-2	59
4.4.3 Kiểm thử chức năng AI Tutor đa phương thức.....	59
4.4.4 Tổng kết kết quả kiểm thử	60
4.5 Triển khai	61
4.5.1 Mô hình triển khai thử nghiệm	61

4.5.2 Thiết bị và môi trường triển khai	62
4.5.3 Kết quả triển khai thử nghiệm	62
4.6 Tổng kết.....	63
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	64
5.1 Xây dựng vòng học tập tích hợp cho người học tiếng Nhật.....	64
5.1.1 Giới thiệu bài toán.....	64
5.1.2 Giải pháp	64
5.1.3 Kết quả đạt được	65
5.2 Điều chỉnh thuật toán SM-2 cho luồng học bốn mức đánh giá.....	66
5.2.1 Giới thiệu bài toán.....	66
5.2.2 Giải pháp	66
5.2.3 Kết quả đạt được	67
5.3 Sinh nội dung ôn tập thông minh bằng AI Review Pipeline.....	67
5.3.1 Giới thiệu bài toán.....	67
5.3.2 Giải pháp	67
5.3.3 Kết quả đạt được	68
5.4 Xây dựng AI Tutor đa phương thức.....	69
5.4.1 Giới thiệu bài toán.....	69
5.4.2 Giải pháp	69
5.4.3 Kết quả đạt được	70
5.5 Xây dựng GraphRAG lựa chọn nghĩa cho dịch thuật chuyên ngành.....	70
5.5.1 Giới thiệu bài toán.....	70
5.5.2 Giải pháp	71
5.5.3 Kết quả đạt được	72
5.6 Tổng kết.....	72

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	74
6.1 Kết luận.....	74
6.1.1 So sánh với các sản phẩm tương tự.....	74
6.1.2 Đánh giá kết quả thực hiện.....	74
6.1.3 Đóng góp nổi bật và bài học kinh nghiệm.....	75
6.2 Hướng phát triển.....	75
6.2.1 Hoàn thiện các chức năng hiện tại.....	75
6.2.2 Nâng cấp và mở rộng hướng nghiên cứu mới	76
TÀI LIỆU THAM KHẢO.....	78

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	9
Hình 2.2	Biểu đồ use case phân rã Quản lý tài khoản và Tra cứu tri thức	10
Hình 2.3	Biểu đồ use case phân rã Bình luận từ điển và Quản lý flashcard	11
Hình 2.4	Biểu đồ use case phân rã Ôn tập flashcard và Ôn tập thông minh	11
Hình 2.5	Biểu đồ use case phân rã Luyện hội thoại với AI Tutor và Dịch thuật	12
Hình 2.6	Biểu đồ hoạt động của quy trình học tập tích hợp	13
Hình 4.1	Mô hình kiến trúc ba lớp	25
Hình 4.2	Biểu đồ phụ thuộc gói tổng quan của hệ thống	27
Hình 4.3	Biểu đồ lớp đại diện của gói frontend	29
Hình 4.4	Thiết kế các lớp nghiệp vụ từ điển và flashcard trong gói backend	30
Hình 4.5	Thiết kế các lớp AI Tutor và Review trong gói backend	30
Hình 4.6	Biểu đồ lớp của gói python_pipeline	31
Hình 4.7	Wireframe bố cục chung trên máy tính	33
Hình 4.8	Wireframe bố cục header	33
Hình 4.9	Wireframe bố cục sidebar	34
Hình 4.10	Wireframe màn hình tra cứu tiếng Nhật	35
Hình 4.11	Wireframe màn hình quản lý bộ thẻ	35
Hình 4.12	Thiết kế chi tiết lớp FlashcardServiceImpl	36
Hình 4.13	Thiết kế chi tiết lớp TutorServiceImpl	37
Hình 4.14	Thiết kế chi tiết lớp ReviewServiceImpl	38
Hình 4.15	Biểu đồ trình tự use case ôn tập flashcard	39
Hình 4.16	Biểu đồ trình tự use case gửi tin nhắn AI Tutor	40
Hình 4.17	Sơ đồ cơ sở dữ liệu quan hệ trên PostgreSQL	41
Hình 4.18	Mô hình dữ liệu đồ thị Neo4j	44
Hình 4.19	Kiến trúc tổng quát của module AI	46
Hình 4.20	Giao diện tra cứu từ điển và lưu từ vào flashcard	54
Hình 4.21	Giao diện học flashcard theo thuật toán SM-2	54
Hình 4.22	Giao diện ôn tập bằng Review Quiz	55
Hình 4.23	Giao diện ôn tập bằng Review Story	56
Hình 4.24	Giao diện luyện hội thoại với AI Tutor	56
Hình 4.25	Giao diện dịch thuật và phân tích chuyên sâu	57

Hình 4.26	Mô hình triển khai thử nghiệm trên localhost	61
Hình 5.1	Vòng học tập khép kín dựa trên dữ liệu cá nhân	65
Hình 5.2	Mô hình đồ thị tri thức phục vụ lựa chọn nghĩa	71

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các nhóm công cụ hỗ trợ học tiếng Nhật theo mục tiêu của đề án	8
Bảng 2.2	Đặc tả use case tra cứu mục từ và thêm vào flashcard	14
Bảng 2.3	Đặc tả use case ôn tập flashcard	15
Bảng 2.4	Đặc tả use case luyện hội thoại với AI Tutor	15
Bảng 2.5	Đặc tả use case dịch chuyên sâu	16
Bảng 4.1	Thông số kỹ thuật thiết kế giao diện	32
Bảng 4.2	Thiết kế chi tiết các bảng trong cơ sở dữ liệu PostgreSQL	44
Bảng 4.3	Thiết kế nút và quan hệ trong Neo4j	45
Bảng 4.4	Danh sách API của module AI	47
Bảng 4.5	Danh sách thư viện và công cụ sử dụng	51
Bảng 4.6	Các sản phẩm đóng gói của hệ thống	52
Bảng 4.7	Thống kê thông tin ứng dụng	53
Bảng 4.8	Các trường hợp kiểm thử chức năng ôn tập flashcard theo SM-2	59
Bảng 4.9	Các trường hợp kiểm thử chức năng AI Tutor đa phương thức .	60
Bảng 4.10	Tổng hợp kết quả kiểm thử các chức năng chính	61
Bảng 5.1	Phân chia trách nhiệm trong vòng học tập	65
Bảng 5.2	Ánh xạ phản hồi người học sang chất lượng SM-2	66
Bảng 5.3	Tổng hợp các giải pháp và đóng góp nổi bật	73

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
DATN	Đồ án tốt nghiệp
AI	Artificial Intelligence - Trí tuệ nhân tạo
API	Application Programming Interface - Giao diện lập trình ứng dụng
CLI	Command-Line Interface - Giao diện dòng lệnh
CSS	Cascading Style Sheets - Ngôn ngữ định kiểu giao diện web
CSV	Comma-Separated Values - Định dạng dữ liệu phân tách bằng dấu phẩy
DTO	Data Transfer Object - Đối tượng truyền dữ liệu
EF	Easiness Factor - Hệ số dễ nhớ trong thuật toán SM-2
FK	Foreign Key - Khóa ngoại
FSRS	Free Spaced Repetition Scheduler - Thuật toán lập lịch lặp lại ngắt quãng
GraphRAG	Graph Retrieval-Augmented Generation - Sinh nội dung tăng cường truy xuất bằng đồ thị
HTML	HyperText Markup Language - Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol - Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure - Giao thức HTTP có bảo mật
JAR	Java Archive - Định dạng đóng gói ứng dụng Java
JDBC	Java Database Connectivity - Giao diện kết nối cơ sở dữ liệu của Java
JLPT	Japanese-Language Proficiency Test - Kỳ thi năng lực tiếng Nhật
JPA	Java Persistence API - Đặc tả lưu trữ và ánh xạ dữ liệu trong Java

Thuật ngữ	Ý nghĩa
JSON	JavaScript Object Notation - Định dạng trao đổi dữ liệu
JSONB	JavaScript Object Notation Binary - Kiểu lưu trữ JSON dạng nhị phân
JWT	JSON Web Token - Chuẩn biểu diễn token xác thực
LLM	Large Language Model - Mô hình ngôn ngữ lớn
LTS	Long-Term Support - Phiên bản hỗ trợ dài hạn
MIME	Multipurpose Internet Mail Extensions - Chuẩn mô tả loại nội dung dữ liệu
NLP	Natural Language Processing - Xử lý ngôn ngữ tự nhiên
ORM	Object-Relational Mapping - Ánh xạ đối tượng - quan hệ
PDF	Portable Document Format - Định dạng tài liệu di động
PK	Primary Key - Khóa chính
RAG	Retrieval-Augmented Generation - Sinh nội dung tăng cường truy xuất
RAM	Random-Access Memory - Bộ nhớ truy cập ngẫu nhiên
REST	Representational State Transfer - Kiểu kiến trúc dịch vụ web
RGB	Red, Green, Blue - Hệ màu đỏ, xanh lá và xanh dương
SDK	Software Development Kit - Bộ công cụ phát triển phần mềm
SM-2	SuperMemo 2 - Thuật toán lặp lại ngắt quãng của SuperMemo
SQL	Structured Query Language - Ngôn ngữ truy vấn có cấu trúc

Thuật ngữ	Ý nghĩa
SRS	Spaced Repetition System - Hệ thống lặp lại ngắt quãng
SSD	Solid-State Drive - Ổ lưu trữ thể rắn
STT	Speech-to-Text - Chuyển giọng nói thành văn bản
TTS	Text-to-Speech - Chuyển văn bản thành giọng nói
UI	User Interface - Giao diện người dùng
URL	Uniform Resource Locator - Địa chỉ tài nguyên thống nhất
UUID	Universally Unique Identifier - Mã định danh duy nhất toàn cục
UX	User Experience - Trải nghiệm người dùng
WSD	Word Sense Disambiguation - Phân biệt nghĩa của từ

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này trình bày bối cảnh hình thành đề tài, các vấn đề gặp phải trong quá trình tự học tiếng Nhật, mục tiêu và phạm vi của hệ thống cần xây dựng, định hướng giải pháp được lựa chọn, cũng như bố cục tổng thể của đề án. Nội dung chương là cơ sở để các chương tiếp theo phân tích yêu cầu, lựa chọn công nghệ, thiết kế kiến trúc, triển khai hệ thống và đánh giá kết quả đạt được.

1.1 Đặt vấn đề

Trong những năm gần đây, nhu cầu học tiếng Nhật tại Việt Nam tăng lên rõ rệt do sự phát triển của hợp tác giáo dục, lao động, công nghệ và giao lưu văn hóa giữa Việt Nam và Nhật Bản. Tiếng Nhật được sử dụng trong nhiều môi trường như học thuật, doanh nghiệp, biên dịch tài liệu, giao tiếp công việc và đời sống hằng ngày. Đối với người học, việc đạt được năng lực sử dụng tiếng Nhật không chỉ đòi hỏi ghi nhớ từ vựng và ngữ pháp, mà còn cần khả năng hiểu nghĩa theo ngữ cảnh, vận dụng mẫu câu trong giao tiếp và duy trì thói quen ôn tập trong thời gian dài.

Trong thực tế, quá trình tự học tiếng Nhật thường bị phân tán giữa nhiều công cụ khác nhau. Người học có thể dùng một ứng dụng để tra từ điển, một ứng dụng khác để lưu flashcard, một tài liệu riêng để học ngữ pháp, một công cụ dịch để hiểu văn bản và một nền tảng khác để luyện giao tiếp. Cách học này có thể đáp ứng từng nhu cầu riêng lẻ, nhưng dữ liệu học tập không được kết nối với nhau. Một từ vừa tra cứu không tự động trở thành nội dung ôn tập; lỗi sai trong luyện tập không được dùng để tạo bài học tiếp theo; cấp độ JLPT và lịch sử học của người dùng chưa được khai thác đầy đủ khi gợi ý nội dung.

Một vấn đề quan trọng khác là việc ghi nhớ từ vựng cần có cơ chế ôn tập hợp lý. Nếu ôn tập quá sớm, người học tốn thời gian cho những nội dung đã nhớ; nếu ôn tập quá muộn, kiến thức dễ bị quên. Các thuật toán lặp lại ngắt quãng như SM-2 có thể hỗ trợ lên lịch ôn tập dựa trên mức độ ghi nhớ, nhưng thuật toán chỉ phát huy hiệu quả khi được tích hợp vào một hệ thống có khả năng lưu trạng thái học của từng thẻ, theo dõi tiến độ và cập nhật lịch ôn theo phản hồi thực tế của người học.

Bên cạnh việc ghi nhớ, người học cũng cần luyện khả năng sử dụng tiếng Nhật trong ngữ cảnh. Các công cụ hội thoại và dịch tự động hiện nay đã hỗ trợ người học tiếp cận nội dung nhanh hơn, nhưng dữ liệu từ các phiên luyện tập thường không được liên kết với tiến độ học, bộ thẻ hoặc lỗi sai cá nhân. Đối với các văn bản chuyên ngành, bản dịch hoặc phần giải thích nghĩa cũng có thể thiếu căn cứ rõ ràng từ kho tri thức mà hệ thống quản lý. Điều này làm giảm khả năng kiểm soát, theo

dối và cá nhân hóa quá trình học.

Ngoài ra, kỹ năng giao tiếp là một phần quan trọng trong học ngoại ngữ nhưng thường khó tự luyện một cách hiệu quả. Người học cần một môi trường luyện tập có phản hồi tức thời, có thể gợi ý sửa lỗi và tạo điều kiện sử dụng từ vựng mục tiêu trong ngữ cảnh tự nhiên. Nếu hoạt động hội thoại không gắn với dữ liệu học tập, người học khó biết mình tiến bộ ở đâu, thường mắc lỗi gì và cần ôn lại nội dung nào.

Từ các phân tích trên, đề tài “Hệ thống thông minh hỗ trợ học tiếng Nhật” được lựa chọn nhằm xây dựng một nền tảng học tập tích hợp. Hệ thống hướng tới việc kết nối các hoạt động tra cứu, ghi nhớ, ôn tập, luyện giao tiếp và dịch thuật chuyên ngành trong cùng một sản phẩm, qua đó tận dụng dữ liệu học tập cá nhân và tri thức ngôn ngữ để hỗ trợ người học hiệu quả hơn.

1.2 Mục tiêu và phạm vi đề tài

Các sản phẩm hỗ trợ học tiếng Nhật hiện nay có thể chia thành một số nhóm chính như từ điển trực tuyến, ứng dụng flashcard, nền tảng luyện thi, công cụ dịch tự động và ứng dụng hội thoại với AI. Nhóm từ điển giúp người học tra cứu nhanh từ vựng, kanji và ngữ pháp, nhưng thường chưa kết nối trực tiếp với lịch ôn tập. Nhóm flashcard hỗ trợ ghi nhớ từ vựng, nhưng dữ liệu học thường tách biệt với hoạt động tra cứu và luyện sử dụng. Nhóm công cụ dịch và hội thoại giúp người học tiếp cận văn bản hoặc luyện giao tiếp nhanh hơn, nhưng kết quả học tập và lỗi sai chưa được khai thác đầy đủ để cá nhân hóa nội dung ôn tập.

Từ đánh giá trên, có thể thấy hạn chế chính của các công cụ hiện có không nằm ở việc thiếu từng chức năng riêng lẻ, mà ở việc thiếu sự liên kết giữa các hoạt động học. Người học cần một hệ thống có thể kết nối dữ liệu tra cứu, flashcard, ôn tập, luyện giao tiếp, lỗi sai và dịch thuật trong cùng một quy trình. Trên cơ sở đó, mục tiêu tổng quát của đề án là xây dựng một hệ thống web hỗ trợ học tiếng Nhật, tích hợp các chức năng học tập cốt lõi và một số thành phần AI trong một kiến trúc thống nhất.

Hệ thống hướng tới việc cho phép người dùng tra cứu từ vựng, kanji và ngữ pháp; quản lý bộ thẻ flashcard; ôn tập theo thuật toán lặp lại ngắt quãng; sinh bài ôn tập cá nhân hóa; luyện hội thoại với AI Tutor; và hỗ trợ dịch thuật chuyên ngành dựa trên ngữ cảnh. Về phía người học, hệ thống tạo một quy trình liên tục từ tra cứu, lưu thẻ, ôn tập, luyện sử dụng đến xem lại phản hồi. Về phía kỹ thuật, hệ thống được triển khai theo kiến trúc nhiều tầng gồm giao diện web, backend quản lý nghiệp vụ, cơ sở dữ liệu và tầng AI.

Phạm vi chức năng của đề án tập trung vào năm nhóm chính. Nhóm thứ nhất là tra cứu và quản lý tri thức học tập, bao gồm tìm kiếm từ vựng, kanji, ngữ pháp, xem ví dụ và bình luận tại từng mục từ điển. Nhóm thứ hai là flashcard và ôn tập, bao gồm tạo bộ thẻ, thêm thẻ từ dữ liệu tra cứu, theo dõi trạng thái học và tính toán lịch ôn tập bằng thuật toán SM-2. Nhóm thứ ba là ôn tập thông minh, bao gồm sinh trắc nghiệm và câu chuyện tương tác dựa trên bộ thẻ, cấp độ JLPT và lỗi sai gần đây. Nhóm thứ tư là AI Tutor, cho phép người học luyện hội thoại bằng văn bản và hỗ trợ tương tác giọng nói thông qua khả năng ghi âm/phát âm trên trình duyệt. Nhóm thứ năm là dịch thuật chuyên ngành, sử dụng pipeline xử lý tiếng Nhật, đồ thị tri thức Neo4j và mô hình ngôn ngữ lớn để tăng khả năng kiểm soát ngữ cảnh khi chọn nghĩa.

Trong khuôn khổ đề án tốt nghiệp, hệ thống được triển khai ở mức nguyên mẫu phục vụ thử nghiệm chức năng và đánh giá kiến trúc. Đề án chưa đặt mục tiêu xây dựng một sản phẩm thương mại quy mô lớn, chưa tối ưu toàn diện cho tải người dùng cao, chưa bao phủ đầy đủ toàn bộ tri thức tiếng Nhật ở mọi trình độ và chưa thay thế vai trò của giáo viên trong các tình huống giảng dạy chuyên sâu. Thay vào đó, đề án tập trung chứng minh tính khả thi của một hệ thống học tiếng Nhật tích hợp dữ liệu học tập, thuật toán ôn tập và các thành phần AI trong một nền tảng thống nhất.

1.3 Định hướng giải pháp

Từ các vấn đề và mục tiêu đã nêu, đề án lựa chọn định hướng xây dựng hệ thống theo kiến trúc phân tầng kết hợp với các pipeline AI riêng. Kiến trúc này giúp tách biệt giao diện người dùng, nghiệp vụ backend, lưu trữ dữ liệu và xử lý AI. Frontend giao tiếp với backend thông qua REST API; backend quản lý nghiệp vụ cốt lõi và điều phối các yêu cầu đến AI layer; AI layer xử lý các tác vụ cần sinh nội dung hoặc phân tích ngôn ngữ.

Đối với bài toán dữ liệu học tập bị phân tán, đề án định hướng xây dựng một nền tảng tích hợp, trong đó các chức năng tra cứu, flashcard, ôn tập, hội thoại và dịch thuật cùng sử dụng dữ liệu người học. Cách tiếp cận này giúp kết quả của hoạt động này có thể trở thành đầu vào cho hoạt động khác, ví dụ từ vựng tra cứu có thể được thêm vào flashcard, còn lỗi sai trong hội thoại có thể được dùng cho bài ôn tập sau.

Đối với bài toán ghi nhớ từ vựng, đề án tích hợp thuật toán SM-2 vào chức năng flashcard. Thuật toán này được lựa chọn vì đơn giản, phù hợp với bài toán lặp lại ngắt quãng và có thể cập nhật lịch ôn dựa trên phản hồi của người học. Nhờ đó, hệ thống có thể ưu tiên những nội dung cần ôn thay vì yêu cầu người học tự quản lý

toàn bộ lịch học.

Đối với bài toán ôn tập và luyện sử dụng, đề án xây dựng AI Review Pipeline và AI Tutor. AI Review Pipeline sử dụng bộ thể, cấp độ JLPT và lỗi sai gần đây để sinh trắc nghiệm hoặc câu chuyện tương tác. AI Tutor hỗ trợ người học luyện hội thoại, nhận phản hồi và lưu lại kết quả phiên học. Hai thành phần này được lựa chọn nhằm đưa từ vựng ra khỏi ngữ cảnh thể đơn lẻ và đặt vào tình huống sử dụng cụ thể hơn.

Đối với bài toán dịch thuật chuyên ngành và giải thích nghĩa theo ngữ cảnh, đề án sử dụng hướng tiếp cận GraphRAG với Neo4j. Mô hình ngôn ngữ lớn có thể dịch và giải thích nghĩa trực tiếp nếu được prompt phù hợp, nhưng pipeline GraphRAG giúp hệ thống bổ sung bằng chứng ngữ cảnh từ kho dữ liệu riêng trước khi sinh kết quả. Định hướng này giúp tăng khả năng kiểm soát nguồn tri thức và tạo cơ sở giải thích rõ ràng hơn cho các thuật ngữ đa nghĩa.

Đóng góp chính của đề án là thiết kế và triển khai một nguyên mẫu hệ thống học tiếng Nhật có khả năng kết nối dữ liệu học tập, thuật toán ôn tập và các chức năng AI trong cùng một sản phẩm. Kết quả đạt được là một hệ thống có thể hỗ trợ người học tra cứu, ghi nhớ, ôn tập, luyện giao tiếp và dịch thuật chuyên ngành ở mức nguyên mẫu, tạo cơ sở để mở rộng thành trợ lý học ngoại ngữ cá nhân hóa trong tương lai.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát và phân tích yêu cầu đối với hệ thống học tiếng Nhật thông minh. Chương này làm rõ các nhóm người dùng, nhu cầu sử dụng, các chức năng chính của hệ thống, đặc tả một số ca sử dụng tiêu biểu và các yêu cầu phi chức năng liên quan đến bảo mật, khả dụng, hiệu năng, khả năng mở rộng và tính dễ sử dụng.

Chương 3 giới thiệu các công nghệ được sử dụng trong quá trình xây dựng hệ thống. Nội dung chương tập trung vào vai trò của Vue 3, Vite, Tailwind CSS và Pinia ở tầng frontend; Java, Spring Boot, Spring Security, JPA và JWT ở tầng backend; PostgreSQL và Neo4j ở tầng dữ liệu; cùng Python, FastAPI, SudachiPy, API trình duyệt cho tương tác giọng nói và mô hình ngôn ngữ lớn ở tầng AI.

Chương 4 trình bày quá trình thiết kế, phát triển và triển khai ứng dụng. Chương này mô tả kiến trúc tổng thể, thiết kế các thành phần backend, frontend, cơ sở dữ liệu và AI layer, cách các thành phần giao tiếp với nhau, các chức năng đã xây dựng, quá trình kiểm thử và mô hình triển khai thử nghiệm của hệ thống.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đề án. Nội dung chương tập trung phân tích sâu các điểm chính như tích hợp thuật toán SM-2 cho ôn tập flashcard, sinh bài ôn tập thông minh bằng AI Review Pipeline, xây dựng AI Tutor gắn với dữ liệu học tập, cũng như ứng dụng Neo4j GraphRAG để cung cấp bằng chứng ngữ cảnh cho dịch thuật chuyên ngành tiếng Nhật.

Chương 6 đưa ra kết luận và định hướng phát triển. Chương này tổng kết các kết quả đã đạt được, tự đánh giá mức độ hoàn thành mục tiêu đề tài, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng mở rộng như cải thiện chất lượng dữ liệu, nâng cao khả năng cá nhân hóa, tối ưu hiệu năng, triển khai trên môi trường thực tế và phát triển thêm các chức năng hỗ trợ học ngoại ngữ.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này tập trung khảo sát hiện trạng và phân tích các yêu cầu đối với hệ thống thông minh hỗ trợ học tiếng Nhật. Nội dung khảo sát được xem xét từ ba nguồn chính: nhu cầu của nhóm người dùng mục tiêu, các công cụ đang được sử dụng trong từng hoạt động học và các ứng dụng có chức năng tương tự. Việc khảo sát không nhằm khẳng định một công cụ hiện có tốt hay chưa tốt một cách tuyệt đối, mà nhằm xác định chức năng trọng tâm, phạm vi phục vụ và những điểm còn thiếu khi đặt các công cụ đó trong một quy trình tự học tiếng Nhật liên tục.

Trên cơ sở kết quả khảo sát, chương xác định các nhóm chức năng cần phát triển, tác nhân chính của hệ thống và mối quan hệ giữa các chức năng. Phương pháp phân tích thiết kế hướng đối tượng được sử dụng để mô hình hóa yêu cầu thông qua các biểu đồ use case tổng quát và use case phân rã. Một số use case tiêu biểu được đặc tả chi tiết theo mục tiêu, tiền điều kiện, hậu điều kiện, luồng xử lý chính, luồng thay thế và ngoại lệ. Chương cũng trình bày các yêu cầu phi chức năng liên quan đến bảo mật, tính toàn vẹn dữ liệu, hiệu năng, khả dụng, khả năng mở rộng và tính dễ sử dụng. Đây là cơ sở để lựa chọn công nghệ, thiết kế kiến trúc và đánh giá mức độ đáp ứng của hệ thống trong các chương tiếp theo.

2.1 Khảo sát hiện trạng

Quá trình tự học tiếng Nhật gồm nhiều hoạt động liên quan như tra cứu, ghi nhớ, ôn tập, luyện sử dụng, nhận phản hồi và đọc hiểu tài liệu. Trong thực tế, người học thường thực hiện các hoạt động này trên nhiều công cụ riêng biệt. Vì vậy, khảo sát hiện trạng cần xem xét cả khả năng của từng nhóm sản phẩm và mức độ kết nối dữ liệu giữa các hoạt động học.

Trong phạm vi đồ án, khảo sát được thực hiện theo hướng phân tích định tính dựa trên ba nguồn: nhu cầu của người dùng mục tiêu, các công cụ phục vụ từng hoạt động học và các ứng dụng có chức năng tương tự. Đồ án chưa thực hiện khảo sát định lượng trên một mẫu lớn; kết quả khảo sát được sử dụng để xác định yêu cầu cho nguyên mẫu phần mềm, không được xem là kết luận thống kê đại diện cho toàn bộ người học tiếng Nhật.

2.1.1 Khảo sát nhu cầu của người dùng mục tiêu

Người dùng mục tiêu là người Việt Nam đang tự học tiếng Nhật, bao gồm người học theo cấp độ JLPT, sinh viên, người chuẩn bị làm việc trong môi trường sử dụng tiếng Nhật hoặc cần đọc tài liệu chuyên môn. Từ quy trình học của nhóm này có thể xác định các nhu cầu chính sau:

- **Tra cứu tri thức:** Tra cứu nhanh từ vựng, kanji, ngữ pháp và lưu trực tiếp vào flashcard.
- **Quản lý và ôn tập flashcard:** Ôn tập theo lịch và thuật toán SRS dựa trên mức độ ghi nhớ.
- **Ôn tập thông minh:** Sinh bài tập linh hoạt (trắc nghiệm, câu chuyện tương tác) từ dữ liệu học tập riêng.
- **Luyện hội thoại với AI:** Thực hành giao tiếp có từ vựng mục tiêu và nhận phản hồi sửa lỗi.
- **Dịch thuật chuyên ngành:** Dịch văn bản kèm phân tích thuật ngữ và ngữ cảnh chuyên ngành.
- **Tích hợp dữ liệu:** Quản lý tập trung hồ sơ, bộ thẻ, lịch ôn và lỗi sai để cá nhân hóa quá trình học.

2.1.2 Khảo sát các hệ thống và ứng dụng tương tự

Các sản phẩm được phân thành năm nhóm theo chức năng trọng tâm để tránh so sánh trực tiếp những công cụ có mục tiêu khác nhau:

- **Từ điển trực tuyến (Mazii, Jisho):** Tra cứu nhanh, thông tin ngôn ngữ phong phú nhưng chưa tạo thành một quy trình học thống nhất.
- **Ứng dụng flashcard (Anki):** Thế mạnh về lặp lại ngắt quãng nhưng người dùng phải tự soạn thẻ, không tích hợp hội thoại và dịch chuyên ngành.
- **Nền tảng học theo lộ trình (Duolingo, Bunpo, Renshuu):** Bài học và bài tập có cấu trúc nhưng khó phản ánh và ôn tập các nội dung, lỗi sai riêng.
- **Công cụ dịch và hỏi đáp (Google Translate, DeepL, ChatGPT, Gemini):** Xử lý ngôn ngữ tốt nhưng không quản lý tiến độ học tập và lịch ôn của người học.
- **Chatbot luyện hội thoại:** Tạo tình huống linh hoạt nhưng dữ liệu hội thoại chưa tự động liên kết với flashcard và lịch ôn.

Bảng sau tổng hợp sự khác biệt về trọng tâm giữa các nhóm công cụ. Các nhận xét mang tính tương đối và được xét theo mục tiêu tích hợp dữ liệu học tập của đề án; chúng không phủ nhận khả năng một sản phẩm có thể cung cấp thêm chức năng thông qua phiên bản mới, gói dịch vụ hoặc tiện ích mở rộng.

Nhóm công cụ	Chức năng trọng tâm	Ưu điểm nổi bật	Hạn chế khi đối chiếu với mục tiêu đề án
Mazii, Jisho và từ điển trực tuyến	Tra cứu từ vựng, kanji và ví dụ	Tra cứu nhanh, thông tin ngôn ngữ phong phú	Dữ liệu tra cứu chưa phải lúc nào cũng được nối thành một vòng học gồm ôn tập, hội thoại và phân tích lỗi

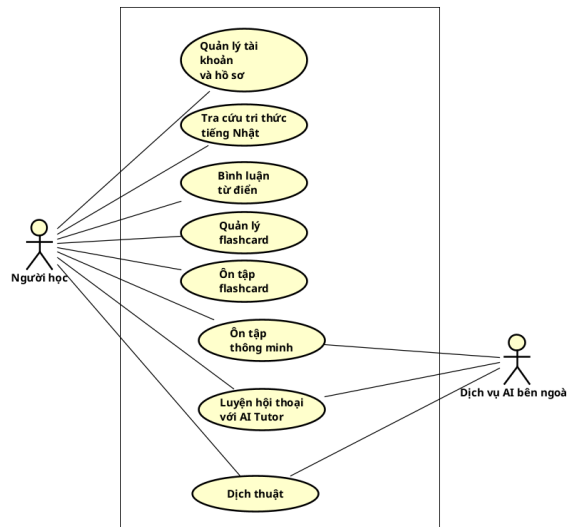
Nhóm công cụ	Chức năng trọng tâm	Ưu điểm nổi bật	Hạn chế khi đối chiếu với mục tiêu đồ án
Anki và ứng dụng flashcard	Tạo thẻ và lặp lại ngắt quãng	Tùy biến thẻ tốt, hỗ trợ ghi nhớ dài hạn	Người học phải chuẩn bị nội dung; tra cứu, hội thoại và dịch không phải chức năng trọng tâm
Duolingo, Bunpo, Ren-shuu và nền tảng theo lộ trình	Bài học và bài tập có cấu trúc	Dễ tiếp cận, tạo động lực và duy trì thói quen	Nội dung chủ yếu theo lộ trình của nền tảng; khó phản ánh đầy đủ bộ thẻ và lỗi sai riêng trong một hệ thống bên ngoài
Google Translate, DeepL	Dịch văn bản	Nhanh, thuận tiện và cho bản dịch tự nhiên trong nhiều tình huống	Không tập trung quản lý tiến độ học, flashcard, lịch ôn và lỗi sai của người học
ChatGPT, Gemini và chatbot AI	Hỏi đáp, giải thích, sinh nội dung và hội thoại	Năng lực ngôn ngữ rộng, phản hồi linh hoạt	Dữ liệu học tập riêng và căn cứ chọn nghĩa chuyên ngành cần được cung cấp, tổ chức và lưu trữ bởi hệ thống tích hợp
Hệ thống đề xuất	Kết nối tra cứu, ghi nhớ, ôn tập, hội thoại và dịch	Dữ liệu của các hoạt động được sử dụng lại trong cùng một tài khoản và quy trình học	Phạm vi dữ liệu, chất lượng AI và quy mô triển khai còn giới hạn ở mức nguyên mẫu

Bảng 2.1: So sánh các nhóm công cụ hỗ trợ học tiếng Nhật theo mục tiêu của đồ án

Kết quả so sánh cho thấy mỗi nhóm công cụ giải quyết tốt một số nhu cầu cụ thể. Khoảng trống mà đồ án lựa chọn không nằm ở việc thay thế các sản phẩm này, mà ở việc xây dựng một nguyên mẫu tích hợp kết nối các chức năng: quản lý tài khoản, tra cứu tri thức, bình luận từ điển, quản lý và ôn tập flashcard (SM-2), ôn tập thông minh (sinh trắc nghiệm/câu chuyện), luyện hội thoại với AI Tutor và dịch thuật chuyên ngành. Các hoạt động này tạo thành một vòng học tập liên tục, trong đó dữ liệu từ một hoạt động tự động trở thành đầu vào phục vụ cá nhân hóa cho hoạt động tiếp theo.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát của hệ thống

Biểu đồ use case tổng quát gồm hai tác nhân là **Người học** và **Dịch vụ AI bên ngoài**. **Người học** là tác nhân chính, trực tiếp sử dụng hệ thống để tra cứu, quản lý nội dung học, ôn tập, luyện hội thoại và dịch thuật. **Dịch vụ AI bên ngoài** là tác nhân phụ, đại diện cho các dịch vụ trí tuệ nhân tạo và xử lý ngôn ngữ được tích hợp thông qua API (như Gemini API hoặc Groq API), hỗ trợ hệ thống thực hiện các chức năng dịch thuật, hội thoại với AI Tutor và tạo nội dung ôn tập thông minh. Các thành phần nội bộ như cơ sở dữ liệu và các mô-đun xử lý bên trong hệ thống không được biểu diễn như tác nhân trong biểu đồ use case vì chúng là một phần của kiến trúc triển khai, không phải các thực thể tương tác trực tiếp với hệ thống.

Biểu đồ gồm tám use case chính:

1. **Quản lý tài khoản và hồ sơ:** cho phép người học đăng ký, đăng nhập và quản lý thông tin học tập cá nhân.
2. **Tra cứu tri thức tiếng Nhật:** hỗ trợ tìm kiếm và xem thông tin từ vựng, kanji, ngữ pháp và ví dụ.
3. **Bình luận từ điển:** cho phép người học trao đổi tại các mục nội dung từ điển.
4. **Quản lý flashcard:** cho phép tạo và quản lý bộ thẻ cùng các thẻ học.
5. **Ôn tập flashcard:** hỗ trợ người học ôn thẻ và cập nhật lịch ôn dựa trên kết quả ghi nhớ.
6. **Ôn tập thông minh:** tạo các hình thức luyện tập từ dữ liệu học tập của người dùng.
7. **Luyện hội thoại với AI Tutor:** tạo môi trường thực hành tiếng Nhật và cung

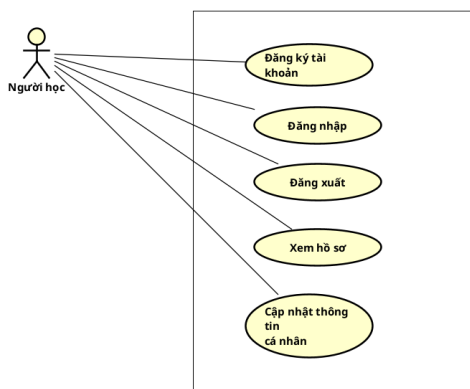
cấp phản hồi cho người học.

8. **Dịch thuật:** hỗ trợ dịch nhanh hoặc phân tích văn bản tiếng Nhật theo ngữ cảnh.

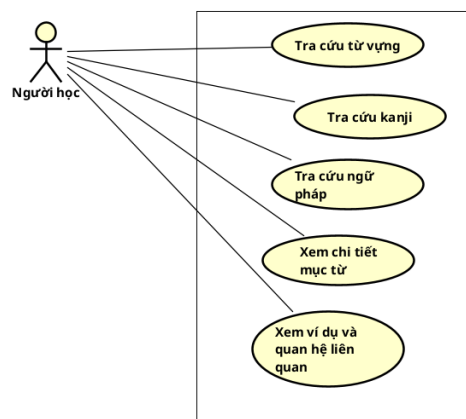
2.2.2 Các biểu đồ use case phân rã

Để làm rõ phạm vi chức năng, các use case chính được phân rã thành các chức năng chi tiết hơn:

- **Quản lý tài khoản và hồ sơ:** Gồm đăng ký, đăng nhập, đăng xuất, xem và cập nhật hồ sơ cá nhân (Hình 2.2a).
- **Tra cứu tri thức tiếng Nhật:** Gồm tìm kiếm từ vựng, kanji, ngữ pháp, xem chi tiết, ví dụ và quan hệ liên quan (Hình 2.2b).



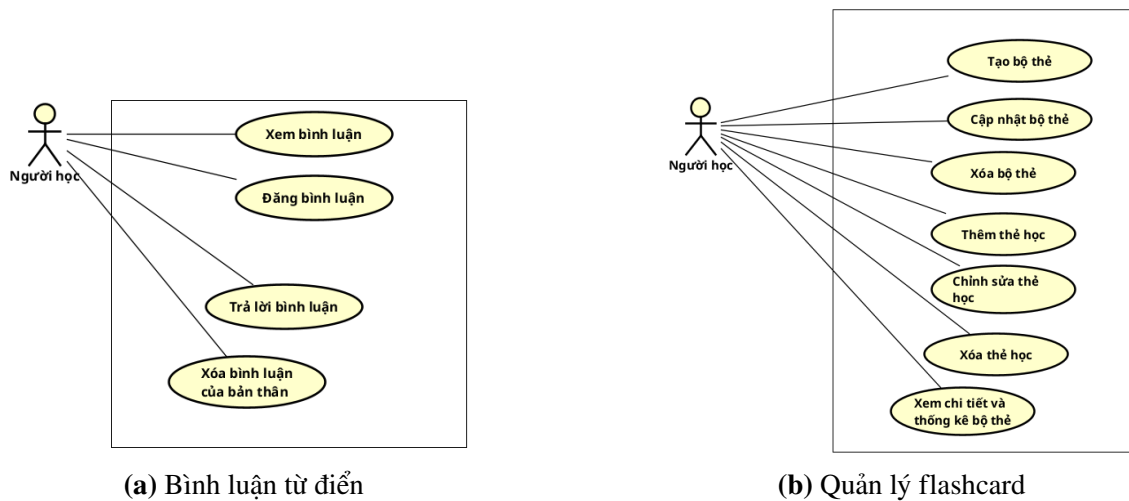
(a) Quản lý tài khoản và hồ sơ



(b) Tra cứu tri thức tiếng Nhật

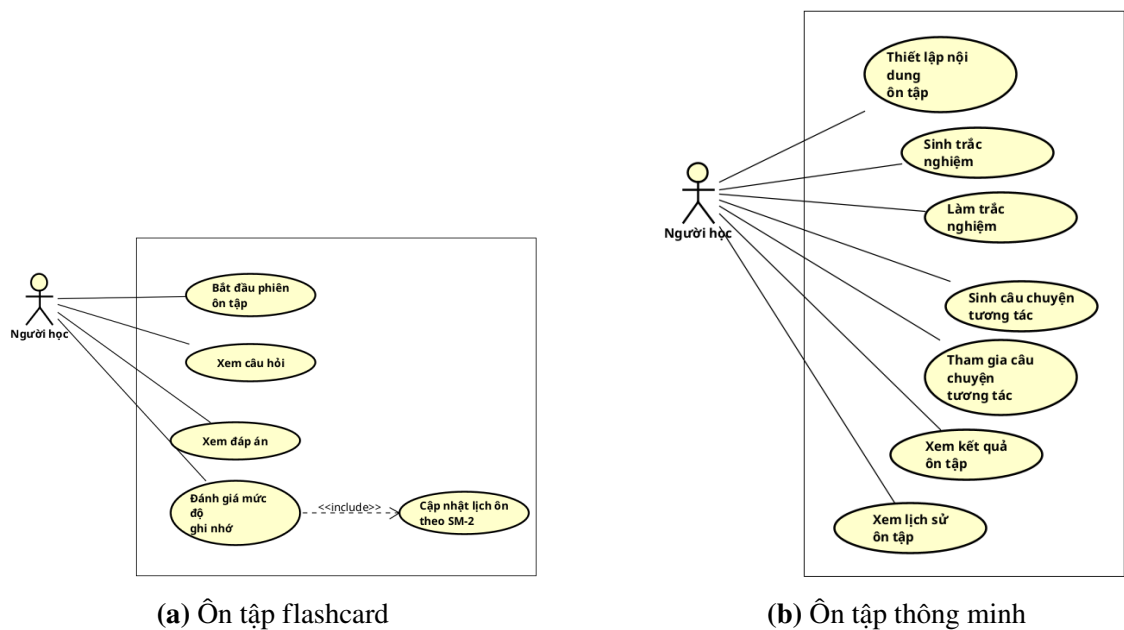
Hình 2.2: Biểu đồ use case phân rã Quản lý tài khoản và Tra cứu tri thức

- **Bình luận từ điển:** Gồm xem, đăng, trả lời và xóa bình luận (Hình 2.3a), tạo không gian trao đổi tại từng mục từ.
- **Quản lý flashcard:** Gồm quản lý bộ thẻ, thẻ học và xem tổng quan bộ thẻ (Hình 2.3b).



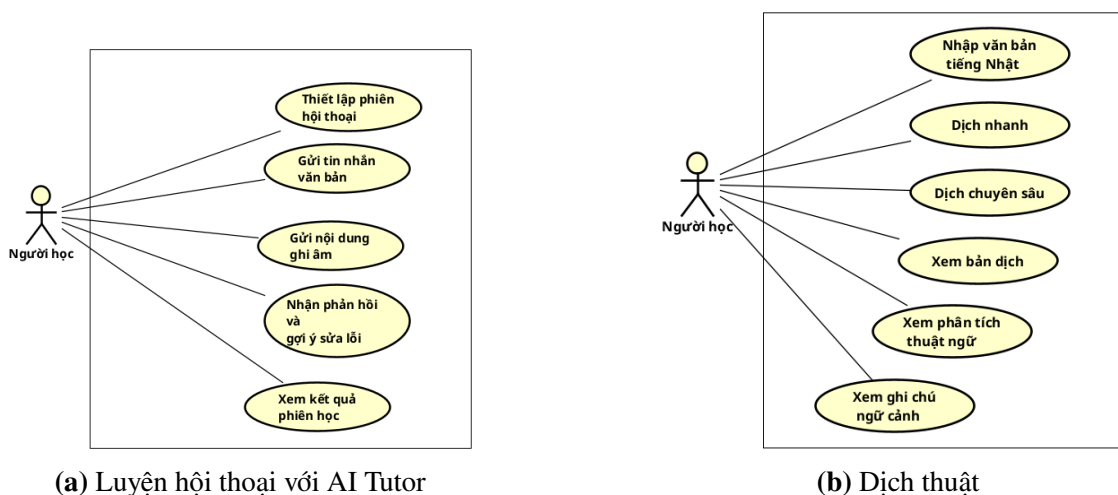
Hình 2.3: Biểu đồ use case phân rã Bình luận từ điển và Quản lý flashcard

- **Ôn tập flashcard:** Gồm bắt đầu phiên ôn, xem câu hỏi/đáp án, đánh giá ghi nhớ và xem kết quả (Hình 2.4a).
- **Ôn tập thông minh:** Thiết lập đầu vào, làm trắc nghiệm/câu chuyện tương tác, xem lịch sử ôn (Hình 2.4b).



Hình 2.4: Biểu đồ use case phân rã Ôn tập flashcard và Ôn tập thông minh

- **Luyện hội thoại với AI Tutor:** Thiết lập phiên, gửi văn bản/ghi âm, nhận phản hồi sửa lỗi và xem kết quả (Hình 2.5a).
- **Dịch thuật:** Gồm dịch nhanh và dịch chuyên sâu phân tích thuật ngữ theo ngữ cảnh (Hình 2.5b).



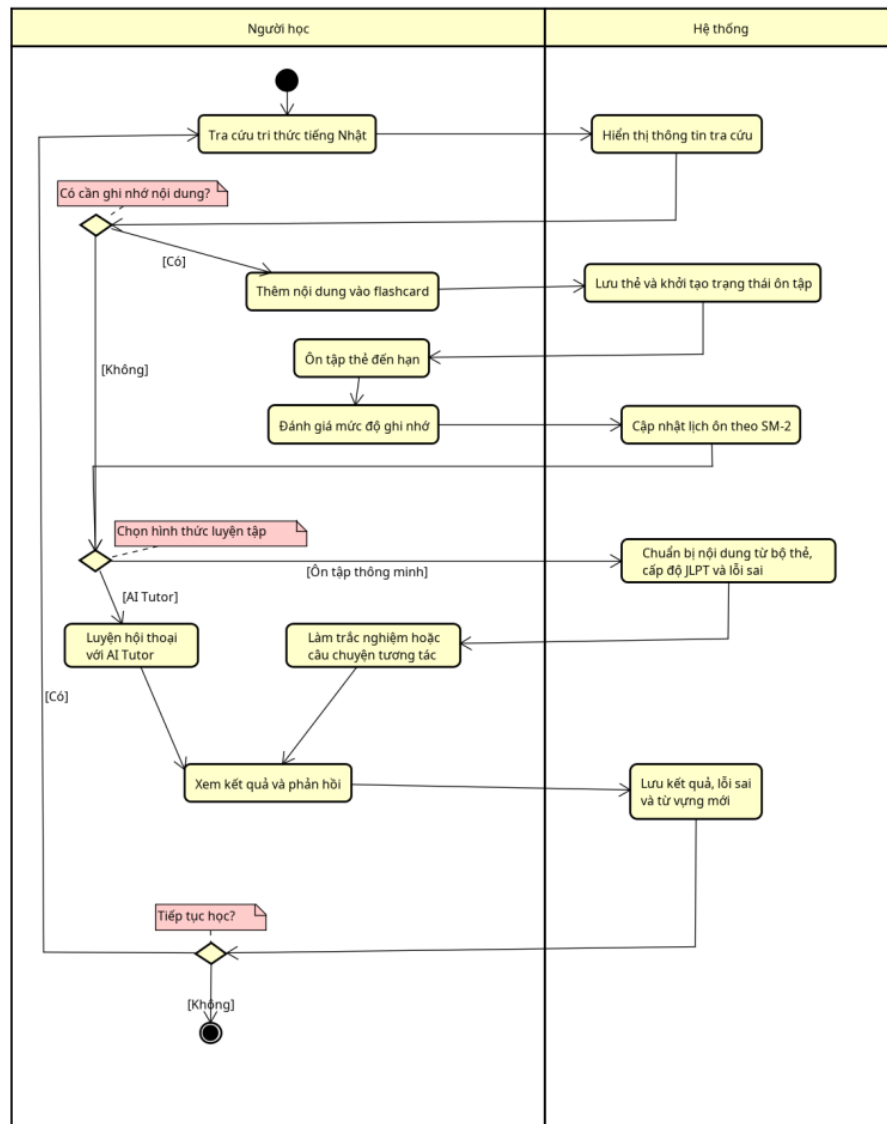
Hình 2.5: Biểu đồ use case phân rã Luyện hội thoại với AI Tutor và Dịch thuật

2.2.3 Quy trình nghiệp vụ

Quy trình nghiệp vụ đáng chú ý của hệ thống là **quy trình học tập tích hợp**. Quy trình này kết hợp các use case tra cứu tri thức tiếng Nhật, quản lý flashcard, ôn tập flashcard, ôn tập thông minh và luyện hội thoại với AI Tutor. Mục tiêu của quy trình là sử dụng lại dữ liệu phát sinh trong từng hoạt động, giúp người học duy trì một vòng học liên tục thay vì thao tác trên các công cụ tách rời.

Quy trình bắt đầu khi người học tra cứu một nội dung tiếng Nhật. Nếu nội dung cần ghi nhớ, người học thêm nội dung đó vào một bộ thẻ flashcard. Hệ thống lưu thẻ và khởi tạo trạng thái ôn tập. Khi thẻ đến hạn, người học thực hiện phiên ôn tập và đánh giá mức độ ghi nhớ; hệ thống dựa trên kết quả này để cập nhật lịch ôn tiếp theo.

Sau bước ghi nhớ, người học có thể lựa chọn luyện tập bằng trắc nghiệm, câu chuyện tương tác hoặc AI Tutor. Hệ thống sử dụng bộ thẻ, cấp độ JLPT và các lỗi sai gần đây để chuẩn bị nội dung phù hợp. Kết quả làm bài, lỗi hội thoại và từ vựng mới được lưu lại. Những dữ liệu này tiếp tục được sử dụng cho các lần ôn tập sau, tạo thành vòng lặp tra cứu - ghi nhớ - luyện sử dụng - nhận phản hồi - ôn lại.



Hình 2.6: Biểu đồ hoạt động của quy trình học tập tích hợp

Điểm quan trọng của quy trình là dữ liệu không chỉ được lưu sau mỗi hoạt động mà còn được chuyển thành đầu vào cho hoạt động tiếp theo. Kết quả tra cứu có thể trở thành flashcard; trạng thái flashcard quyết định nội dung cần ôn; bộ thẻ và lỗi sai được sử dụng để tạo bài tập hoặc phiên hội thoại; phản hồi từ quá trình luyện tập lại bổ sung dữ liệu cho lần ôn sau.

2.3 Đặc tả chức năng

Phần này lựa chọn bốn use case quan trọng nhất để đặc tả chi tiết. Các use case được chọn bao phủ vòng học chính của hệ thống, từ tiếp nhận tri thức, ôn tập, luyện sử dụng đến hỗ trợ đọc hiểu văn bản chuyên ngành. Mỗi đặc tả trình bày tác nhân, tiền điều kiện, hậu điều kiện, luồng sự kiện chính và các luồng phát sinh.

2.3.1 Đặc tả use case tra cứu mục từ và thêm vào flashcard

Mã usecase	UC001	Tên usecase	Tra cứu mục từ và thêm vào flashcard
Tác nhân chính	Người học		
Mô tả	Người học tìm kiếm từ vựng, kanji hoặc ngữ pháp, xem nội dung chi tiết và lưu mục cần ghi nhớ vào một bộ thẻ flashcard		
Tiền điều kiện	Người học đã đăng nhập; dữ liệu từ điển đã được nạp vào hệ thống		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Người học	Mở chức năng tra cứu, chọn loại nội dung và nhập từ khóa.
	2	Hệ thống	Tìm kiếm, hiển thị danh sách kết quả và hiển thị chi tiết khi người học click chọn.
	3	Người học	Chọn chức năng thêm vào flashcard, chọn bộ thẻ đích và xác nhận tạo thẻ.
	4	Hệ thống	Tạo flashcard trong bộ thẻ, khởi tạo trạng thái SRS và thông báo thành công.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	2a	Hệ thống	Không tìm thấy kết quả: hiển thị trạng thái rỗng để người học đổi từ khóa.
	3a	Người học	Chưa có bộ thẻ: tạo bộ thẻ mới trực tiếp rồi tiếp tục bước 3.
Ngoại lệ	Phiên đăng nhập hết hạn và không thể làm mới; mục từ hoặc bộ thẻ không còn tồn tại; người học cố thêm thẻ vào bộ thẻ không thuộc sở hữu; kết nối đến backend bị gián đoạn		
Hậu điều kiện	Một flashcard được tạo trong bộ thẻ do người học lựa chọn; trạng thái SRS ban đầu của thẻ được khởi tạo		

Bảng 2.2: Đặc tả use case tra cứu mục từ và thêm vào flashcard

2.3.2 Đặc tả use case ôn tập flashcard

Mã usecase	UC003	Tên usecase	Ôn tập flashcard
Tác nhân chính	Người học		
Mô tả	Người học ôn các thẻ mới hoặc đến hạn; hệ thống cập nhật lịch ôn tiếp theo dựa trên đánh giá mức độ ghi nhớ		
Tiền điều kiện	Người học đã đăng nhập; bộ thẻ được chọn thuộc sở hữu của người học và có ít nhất một thẻ mới hoặc đến hạn		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Người học	Chọn một bộ thẻ và bắt đầu phiên ôn tập.
	2	Hệ thống	Lấy danh sách thẻ cần ôn và hiển thị mặt trước của thẻ đầu tiên.
	3	Người học	Suy nghĩ câu trả lời, chọn xem đáp án và đánh giá mức độ ghi nhớ (Again/Hard/Good/Easy).
	4	Hệ thống	Tính lại các tham số SRS theo thuật toán SM-2, lưu trạng thái mới và hiển thị thẻ tiếp theo.
	5	Hệ thống	Hiển thị kết quả tổng hợp khi hoàn thành toàn bộ danh sách thẻ ôn tập.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	2a	Hệ thống	Không có thẻ cần ôn: thông báo người học đã hoàn thành nội dung đến hạn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

	3a	Người học	Kết thúc phiên trước khi hết thẻ: hệ thống giữ nguyên trạng thái các thẻ chưa học.
Ngoại lệ	Không tải được danh sách thẻ; cập nhật SRS thất bại; thẻ bị xóa trong khi phiên đang diễn ra; phiên đăng nhập hết hạn		
Hậu điều kiện	Trạng thái SRS của các thẻ đã đánh giá được cập nhật; hệ thống ghi nhận thời điểm ôn tiếp theo		

Bảng 2.3: Đặc tả use case ôn tập flashcard

2.3.3 Đặc tả use case luyện hội thoại với AI Tutor

Mã usecase	UC005	Tên usecase	Luyện hội thoại với AI Tutor
Tác nhân chính	Người học		
Mô tả	Người học luyện giao tiếp tiếng Nhật trong một tình huống xác định bằng văn bản hoặc giọng nói và nhận phản hồi sửa lỗi		
Tiền điều kiện	Người học đã đăng nhập; các thành phần hỗ trợ hội thoại của hệ thống đang hoạt động; trình duyệt hỗ trợ và được cấp quyền microphone nếu sử dụng giọng nói		
	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Người học	Mở chức năng AI Tutor, chọn chủ đề, cấp độ JLPT hoặc bộ thẻ mục tiêu.
	2	Hệ thống	Tạo phiên hội thoại mới và sinh câu chào mở đầu tương ứng với thiết lập.
	3	Người học	Nhập câu tiếng Nhật (bằng văn bản hoặc giọng nói) và gửi đi.
	4	Hệ thống	Phân tích ngữ cảnh để tạo câu trả lời tiếng Nhật, bản dịch, nhận xét lỗi ngữ pháp và gợi ý sửa.
	5	Hệ thống	Đóng phiên, tổng hợp lỗi sai, từ vựng mới và hiển thị báo cáo khi người học chọn kết thúc.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	3a	Người học	Chọn ghi âm: trình duyệt thu âm; hệ thống chuyển giọng nói thành văn bản rồi tiếp tục bước 4.
	4a	Hệ thống	Phát hiện lỗi ngữ pháp: hiển thị câu sửa lỗi kèm giải thích chi tiết trong phản hồi.
Ngoại lệ	Tệp âm thanh vượt giới hạn hoặc sai định dạng; phiên không thuộc người học; phiên đã kết thúc; hệ thống không thể tạo phản hồi; dữ liệu phản hồi không đúng cấu trúc		
Hậu điều kiện	Phiên hội thoại, tin nhắn, lỗi sai, từ vựng mới và kết quả tổng hợp được lưu; lỗi sai có thể được sử dụng cho ôn tập thông minh		

Bảng 2.4: Đặc tả use case luyện hội thoại với AI Tutor

2.3.4 Đặc tả use case dịch chuyên sâu

Mã usecase	UC006	Tên usecase	Dịch chuyên sâu
Tác nhân chính	Người học		
Mô tả	Người học dịch văn bản tiếng Nhật và nhận thêm thông tin phân tích về thuật ngữ, nghĩa được lựa chọn và ngữ cảnh chuyên ngành		
Tiền điều kiện	Người học đã đăng nhập; chức năng dịch thuật và các nguồn dữ liệu cần thiết của hệ thống đang khả dụng		

	STT	Thực hiện bởi	Hành động
Luồng sự kiện chính	1	Người học	Mở chức năng dịch thuật, nhập văn bản tiếng Nhật và chọn dịch chuyên sâu.
	2	Hệ thống	Phân tích văn bản, trích xuất thuật ngữ quan trọng và truy xuất nghĩa chuyên ngành từ Neo4j.
	3	Hệ thống	Lựa chọn nghĩa phù hợp nhất theo ngữ cảnh để tạo bản dịch và ghi chú giải thích thuật ngữ.
	4	Hệ thống	Hiển thị bản dịch chuyên sâu kèm danh sách thuật ngữ, nghĩa tương ứng và miền tri thức.
	STT	Thực hiện bởi	Hành động
Luồng sự kiện thay thế	1a	Người học	Chọn dịch nhanh: hệ thống bỏ qua phân tích Neo4j GraphRAG và hiển thị bản dịch thuần.
	2a	Hệ thống	Không phát hiện thuật ngữ chuyên ngành: dịch theo ngữ cảnh chung và hiển thị bản dịch.
Ngoại lệ	Văn bản đầu vào trống hoặc vượt giới hạn; nguồn dữ liệu phục vụ phân tích không khả dụng; hệ thống không thể tạo hoặc chuẩn hóa kết quả; thời gian xử lý vượt quá giới hạn cho phép		
Hậu điều kiện	Hệ thống hiển thị bản dịch, danh sách thuật ngữ quan trọng, nghĩa được lựa chọn, miền tri thức và ghi chú phân tích nếu có		

Bảng 2.5: Đặc tả use case dịch chuyên sâu

2.4 Yêu cầu phi chức năng

Để đảm bảo khả năng vận hành ổn định trong môi trường thử nghiệm, hệ thống cần đáp ứng các yêu cầu phi chức năng sau:

- **Hiệu năng:** Phản hồi nhanh đối với các tác vụ tra cứu, lưu trữ; hiển thị trạng thái chờ khi gọi API AI.
- **Độ tin cậy:** Bảo đảm toàn vẹn dữ liệu khi đồng bộ flashcard, lịch ôn tập và lưu vết lỗi sai.
- **Bảo mật:** Mã hóa mật khẩu người dùng, kiểm tra quyền sở hữu tài nguyên ở phía máy chủ (backend).
- **Khả dụng:** Giao diện tiếng Việt rõ ràng, tương thích với các kích thước màn hình phổ biến.
- **Kỹ thuật:** Sử dụng các tiêu chuẩn HTTP/JSON, hỗ trợ Unicode tiếng Việt và tiếng Nhật.
- **Bảo trì:** Thiết kế dạng mô-đun tách biệt các lớp giao diện, nghiệp vụ và xử lý AI để dễ mở rộng.

2.5 Tổng kết

Chương này đã thực hiện khảo sát hiện trạng nhu cầu tự học tiếng Nhật và phân tích các nhóm công cụ tương tự để xác định khoảng trống tích hợp dữ liệu học tập. Dựa trên kết quả khảo sát, hệ thống được mô hình hóa thành tám nhóm chức năng với tác nhân chính là người học thông qua các biểu đồ use case tổng quát, biểu đồ

phân rã, biểu đồ hoạt động quy trình học tập tích hợp và bốn đặc tả chi tiết. Chương cũng đã xác định các yêu cầu phi chức năng cơ bản về hiệu năng, độ tin cậy, bảo mật và khả năng bảo trì.

Các kết quả phân tích trong chương này là cơ sở để lựa chọn công nghệ ở Chương 3, thiết kế kiến trúc ở Chương 4 và làm tiêu chí đánh giá mức độ đáp ứng mục tiêu của sản phẩm ở các chương tiếp theo.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định các yêu cầu chính của hệ thống gồm giao diện học tập để sử dụng, backend bảo vệ dữ liệu cá nhân, lưu trữ nhất quán, ôn tập theo lịch, sinh nội dung bằng AI, hội thoại bằng văn bản hoặc giọng nói và dịch thuật có phân tích ngữ cảnh. Chương này trình bày các công nghệ và nền tảng được lựa chọn để đáp ứng những yêu cầu đó. Nội dung tập trung tóm tắt đặc điểm kỹ thuật có liên quan trực tiếp đến đề án; các thiết kế chi tiết và hiện thực được dành cho Chương 4.

Các công nghệ được nhóm theo vấn đề cần giải quyết thay vì liệt kê riêng lẻ. Với mỗi nhóm, đề án giới thiệu khái quát, phân tích ưu nhược điểm, cách ứng dụng thực tế và so sánh với giải pháp thay thế để làm rõ lý do lựa chọn.

3.1 Công nghệ phát triển frontend

Frontend của hệ thống sử dụng Vue 3, Vite, Tailwind CSS, Pinia và Axios, giải quyết yêu cầu về tính dễ sử dụng, khả năng tương thích trình duyệt và tổ chức giao diện thành các thành phần tái sử dụng độc lập (mục 2.4). Vue 3 cung cấp giải pháp xây dựng giao diện theo component linh hoạt với Composition API [1], trong khi Vite đóng vai trò môi trường phát triển và đóng gói tốc độ cao [2]. Trạng thái toàn cục được quản lý tập trung thông qua kho lưu trữ gọn nhẹ Pinia [3], còn các yêu cầu HTTP giao tiếp với backend được Axios thực hiện và xử lý nhất quán nhờ cơ chế interceptor [4]. Bố cục responsive và giao diện người dùng được định hình nhanh chóng bằng các lớp tiện ích của Tailwind CSS [5].

Sự kết hợp này mang lại ưu điểm lớn về hiệu năng biên dịch nhanh của Vite, cú pháp trực quan của Vue và tính nhất quán giao diện của Tailwind CSS. Tuy nhiên, lập trình viên cần duy trì quy tắc tổ chức mã nguồn chặt chẽ để tránh làm phình to component hoặc lạm dụng trạng thái toàn cục trong Pinia. Trong đề án, Vue 3 được ứng dụng để phát triển toàn bộ các màn hình chức năng (xác thực, từ điển, flashcard, ôn tập, AI Tutor và dịch thuật chuyên ngành). Axios đảm nhận việc tự động đính kèm và làm mới JWT token, còn Tailwind CSS đảm bảo giao diện hiển thị tối ưu trên mọi kích thước màn hình.

Khi xem xét phương án thay thế, React và Angular là hai đối thủ lớn nhất. Tuy nhiên, React yêu cầu cấu hình thêm nhiều thư viện bên thứ ba cho định tuyến và quản lý trạng thái, làm tăng độ phức tạp ban đầu; trong khi Angular lại quá đồ sộ và đòi hỏi chi phí học tập lớn đối với một hệ thống nguyên mẫu. Vue 3 được chọn nhờ sự cân bằng giữa tính đơn giản và hiệu năng. Pinia được chọn thay cho Vuex vì cú pháp tối giản và tương thích tốt hơn với Composition API, còn Axios được

ưu tiên hơn `fetch` thuần của trình duyệt để xử lý vòng đời JWT token một cách tự động và đồng bộ thông qua `interceptor`.

3.2 Công nghệ backend và bảo mật

Tầng nghiệp vụ và bảo mật hệ thống sử dụng Java 21, Spring Boot, Spring Security và cơ chế JSON Web Token (JWT). Nhóm công nghệ này đáp ứng yêu cầu quản lý nhiều miền nghiệp vụ phức tạp, bảo vệ dữ liệu cá nhân, kiểm tra quyền sở hữu và cung cấp API RESTful có cấu trúc thống nhất. Spring Boot cung cấp nền tảng vững chắc với cơ chế Dependency Injection, validation dữ liệu và khả năng tích hợp mô-đun mạnh mẽ [6]. Spring Security thiết lập chuỗi bộ lọc bảo mật tập trung [7], phối hợp với định dạng token tự chứa JWT [8] để xác thực không trạng thái (stateless) cho mỗi yêu cầu từ frontend gửi lên.

Kiến trúc Spring Boot giúp mã nguồn backend có tính phân tách rõ ràng (Controller - Service - Repository - DTO) và dễ viết kiểm thử đơn vị. Tuy nhiên, Spring Boot tiêu tốn bộ nhớ lớn hơn và có thời gian khởi chạy chậm hơn các framework tối giản. Spring Security cũng đòi hỏi cấu hình chuỗi bộ lọc chuẩn xác để tránh lỗ hổng bảo mật. Để giải quyết hạn chế của JWT về việc khó thu hồi token tức thời, hệ thống sử dụng cặp access token ngắn hạn và refresh token lưu ở cơ sở dữ liệu. Trong đồ án, Spring Boot đóng vai trò cổng điều phối trung tâm: tiếp nhận yêu cầu, kiểm tra quyền sở hữu đối với flashcard hay phiên chat, xử lý nghiệp vụ lưu trữ PostgreSQL và chỉ gọi module AI Python khi cần thiết.

Các phương án thay thế bao gồm Node.js (Express/NestJS) và Python (Django/FastAPI). Dù Node.js có lợi thế đồng nhất ngôn ngữ Javascript, cấu hình transaction và phân quyền của nó phức tạp hơn; còn Django lại nằm ngoài định hướng phát triển backend bằng Java của hệ thống. Spring Boot và Spring Security được lựa chọn nhờ khả năng cung cấp các tính năng bảo mật cấp doanh nghiệp được tích hợp sẵn, cùng cơ chế JPA giúp thao tác dữ liệu an toàn. Dịch vụ Python vẫn được duy trì nhưng tách biệt hoàn toàn làm module xử lý AI riêng.

3.3 Lưu trữ dữ liệu quan hệ với PostgreSQL và JPA

Hệ thống sử dụng cơ sở dữ liệu quan hệ PostgreSQL phối hợp cùng tầng ánh xạ đối tượng JPA (Java Persistence API) thông qua Spring Data JPA. Giải pháp này nhằm giải quyết yêu cầu lưu trữ thông tin có cấu trúc chặt chẽ và đảm bảo tính nhất quán cao, bao gồm dữ liệu tài khoản người dùng, tiến trình học tập, trạng thái thuật toán SM-2, hệ thống bình luận và lịch sử hội thoại. PostgreSQL nổi tiếng với khả năng hỗ trợ giao dịch ACID, ràng buộc khóa ngoại mạnh mẽ và lập chỉ mục tối ưu [9], trong khi JPA giúp trừu tượng hóa các truy vấn SQL thành các thao tác hướng đối tượng an toàn [6].

Sự kết hợp này giúp lập trình viên thao tác với dữ liệu nhanh chóng, đảm bảo tính toàn vẹn thông tin nhờ cơ chế transaction khi thực hiện các tác vụ phức tạp (ví dụ: tạo flashcard đồng thời khởi tạo bản ghi tiến trình SRS). Nhược điểm của hướng đi này là nguy cơ phát sinh các truy vấn kém tối ưu (như lỗi N+1 truy vấn) nếu không cấu hình nạp dữ liệu (lazy/eager loading) cẩn thận. Trong đồ án, PostgreSQL lưu trữ tất cả thông tin quan hệ cốt lõi, còn Spring Data JPA ánh xạ các thực thể thành đối tượng Java để xử lý nghiệp vụ, sử dụng transaction để đảm bảo tính nhất quán dữ liệu khi người dùng cập nhật kết quả ôn tập hoặc xóa bộ thẻ.

MySQL và MongoDB là hai phương án thay thế được cân nhắc. MySQL đáp ứng tốt nghiệp vụ tương tự nhưng PostgreSQL được chọn nhờ khả năng mở rộng tốt hơn, hỗ trợ kiểu dữ liệu JSONB tối ưu cho các phản hồi AI có cấu trúc động và các tính năng giao dịch nâng cao. MongoDB mang lại sự linh hoạt của NoSQL nhưng không đảm bảo ràng buộc chặt chẽ giữa các thực thể như người dùng, bộ thẻ và flashcard, điều vốn là cốt lõi của nghiệp vụ học tập SRS trong hệ thống.

3.4 Python, FastAPI và mô hình ngôn ngữ lớn

Để thực hiện các tác vụ thông minh như sinh câu hỏi ôn tập, viết câu chuyện tương tác và duy trì hội thoại AI Tutor, đồ án sử dụng ngôn ngữ Python kết hợp framework FastAPI và các mô hình ngôn ngữ lớn (LLM). FastAPI đóng vai trò dịch vụ web nội bộ gọn nhẹ, xử lý bất đồng bộ tốt dựa trên type hint và tự động sinh tài liệu API [10]. Các mô hình ngôn ngữ lớn được tích hợp bao gồm Gemini thông qua Gemini API [11] và Qwen3-32B thông qua Groq API [12], cung cấp năng lượng cho các pipeline xử lý ngôn ngữ tự nhiên.

Hệ sinh thái Python rất mạnh về các thư viện xử lý ngôn ngữ tiếng Nhật và tích hợp LLM. Tuy nhiên, việc tách biệt thành một microservice AI riêng làm tăng độ phức tạp khi triển khai, đồng thời phải đối mặt với tính không chắc chắn (probabilistic) của kết quả sinh ra từ LLM (như lỗi cấu trúc JSON hoặc lỗi dịch vụ ngoại vi). Để hạn chế nhược điểm này, hệ thống triển khai `LLMClient` làm cổng trừu tượng hóa chung, cho phép cấu hình linh hoạt giữa các nhà cung cấp và áp dụng cơ chế kiểm tra schema JSON nghiêm ngặt. Phản hồi AI chỉ đóng vai trò sinh nội dung gợi ý, không can thiệp vào các quy tắc nghiệp vụ cốt lõi tại backend.

Có thể gọi trực tiếp API của mô hình từ Spring Boot hoặc viết toàn bộ backend bằng Python Django. Tuy nhiên, gọi trực tiếp từ Java làm tăng độ phức tạp mã nguồn và khó tích hợp các công cụ tiền xử lý tiếng Nhật; trong khi Django lại quá nặng nề cho một dịch vụ API phi trạng thái. FastAPI được lựa chọn nhờ sự gọn nhẹ và hiệu năng vượt trội. Gemini là mô hình chủ đạo nhờ khả năng sinh JSON có cấu trúc ổn định, trong khi Qwen3-32B được chọn làm phương án dự phòng hiệu quả

nhờ tốc độ suy luận nhanh từ Groq và khả năng hỗ trợ tiếng Nhật tốt, giúp hệ thống hoạt động ổn định ngay cả khi một nhà cung cấp gặp sự cố.

3.5 Neo4j, GraphRAG và SudachiPy

Đối với chức năng dịch thuật chuyên ngành chuyên sâu, hệ thống áp dụng cơ sở dữ liệu đồ thị Neo4j kết hợp kỹ thuật GraphRAG (Retrieval-Augmented Generation dựa trên đồ thị) [13] và công cụ phân tách từ vựng SudachiPy [14]. Nhóm công nghệ này giải quyết yêu cầu phân tích từ đa nghĩa và tối ưu hóa bản dịch dựa trên ngữ cảnh chuyên ngành bằng cách kết nối các thực thể từ vựng, nghĩa dịch tiếng Việt và các từ đồng xuất hiện hỗ trợ làm tín hiệu dẫn đường.

Cơ sở dữ liệu đồ thị Neo4j cho phép mô hình hóa tự nhiên quan hệ đa chiều phức tạp giữa từ vựng và chuyên ngành, giúp truy vấn nhanh chóng mà không cần các phép join bảng nặng nề. Kỹ thuật GraphRAG giúp bổ sung tri thức có cấu trúc vào prompt để hướng dẫn mô hình ngôn ngữ lớn dịch chính xác mà không cần fine-tune. Nhược điểm của giải pháp là yêu cầu tài nguyên để vận hành Neo4j và độ trễ tăng thêm do các bước phân tách từ, truy vấn đồ thị và tính toán xếp hạng nghĩa. Trong đồ án, SudachiPy thực hiện tách từ tiếng Nhật thành các token, Neo4j cung cấp các nghĩa ứng viên cùng ngữ cảnh đồng xuất hiện, làm cơ sở để thuật toán xếp hạng lựa chọn bằng chứng phù hợp trước khi gửi sang LLM dịch thuật [15].

Các lựa chọn thay thế là lưu từ điển dạng bảng trong PostgreSQL hoặc sử dụng Vector RAG thông thường. PostgreSQL gặp khó khăn và kém trực quan khi biểu diễn đồ thị ngữ nghĩa nhiều bậc; còn Vector RAG tuy tốt cho việc tìm văn bản tương tự nhưng không thể trích xuất chính xác các quan hệ ngữ nghĩa có cấu trúc cần thiết cho việc phân tích từ vựng chuyên ngành. Neo4j GraphRAG được lựa chọn vì nó giải quyết triệt để bài toán truy vết nguồn gốc nghĩa từ vựng và tối ưu ngữ cảnh chuyên ngành một cách tường minh.

3.6 Thuật toán lặp lại ngắt quãng SM-2

Thuật toán lặp lại ngắt quãng SM-2 được sử dụng làm cốt lõi cho tính năng quản lý lịch ôn tập flashcard cá nhân hóa của hệ thống [16]. Thuật toán này giải quyết yêu cầu tối ưu hóa thời gian học tập bằng cách tính toán ngày ôn tập tiếp theo dựa trên lịch sử lặp lại và đánh giá chất lượng phản hồi từ người học. Các tham số chính được duy trì gồm số lần lặp liên tiếp (n), khoảng cách ôn (I), và hệ số dễ nhớ (EF).

Ưu điểm nổi bật của SM-2 là cấu hình cực kỳ đơn giản, chi phí tính toán thấp và có thể chạy độc lập cho từng thẻ mà không cần dữ liệu lịch sử lớn hay máy chủ mạnh. Tuy nhiên, nhược điểm là thuật toán phụ thuộc nhiều vào đánh giá chủ quan của người học và không tự động phản ánh các dữ liệu hành vi khác như thời gian trả lời hay kết quả làm bài quiz. Trong đồ án, thuật toán được cài đặt trực tiếp tại

backend Spring Boot. Khi người học hoàn thành ôn tập một thẻ và chọn mức độ nhớ, hệ thống sẽ tính toán ngay lập tức khoảng thời gian ôn tập tiếp theo để cập nhật vào cơ sở dữ liệu.

Các phương án thay thế gồm phương pháp hộp Leitner truyền thống hoặc thuật toán FSRS (Free Spaced Repetition Scheduler) hiện đại. Leitner quá đơn giản và chỉ điều chỉnh lịch ôn ở mức thô; còn FSRS dự đoán trí nhớ chính xác hơn nhưng lại đòi hỏi cấu hình phức tạp và lượng dữ liệu lịch sử ôn tập rất lớn để tối ưu hóa tham số. SM-2 được lựa chọn vì tính cân bằng hoàn hảo giữa hiệu quả cá nhân hóa và độ phức tạp triển khai trong giai đoạn thử nghiệm của đồ án.

3.7 Công nghệ tương tác giọng nói

Để hiện thực hóa trải nghiệm đàm thoại đa phương thức trong tính năng AI Tutor, hệ thống tích hợp công nghệ Nhận dạng giọng nói (STT) và Tổng hợp giọng nói (TTS). Đồ án sử dụng Web Speech API [17] được tích hợp sẵn trên các trình duyệt hiện đại để thực hiện cả hai nhiệm vụ này: giao diện `SpeechRecognition` đảm nhận lắng nghe phát âm tiếng Nhật của người học để chuyển thành văn bản, và đối tượng `speechSynthesis` chịu trách nhiệm đọc các câu phản hồi tiếng Nhật sinh ra từ AI Tutor.

Việc sử dụng Web Speech API giúp hệ thống hoạt động hoàn toàn ở phía client, giảm tải tài nguyên xử lý âm thanh cho máy chủ backend và không phát sinh thêm chi phí vận hành. Tuy nhiên, nhược điểm lớn nhất là mức độ hỗ trợ không đồng đều giữa các trình duyệt (hoạt động tốt nhất trên Google Chrome) và chất lượng nhận dạng phụ thuộc nhiều vào thiết bị thu âm cũng như giọng đọc của người học. Trong đồ án, Web Speech API đóng vai trò là giải pháp giao tiếp giọng nói chính trực tiếp tại trình duyệt của người dùng, hoạt động song song với kênh nhập liệu bằng văn bản truyền thống để đảm bảo tính sẵn sàng của ứng dụng.

Các lựa chọn thay thế bao gồm các dịch vụ đám mây trả phí (như Google Cloud Speech-to-Text, Amazon Polly) hoặc tự triển khai mô hình cục bộ như Faster-Whisper [18]. Các dịch vụ đám mây có độ chính xác cao nhưng làm tăng chi phí vận hành và độ trễ mạng; còn Faster-Whisper đòi hỏi tài nguyên máy chủ có cấu hình phần cứng mạnh (GPU) để xử lý. Web Speech API được lựa chọn cho nguyên mẫu hệ thống để tối ưu hóa chi phí và tài nguyên máy chủ, trong khi phương án dự phòng nhập liệu văn bản luôn được duy trì để đảm bảo ứng dụng hoạt động ổn định trên mọi thiết bị.

3.8 Tổng kết

Chương 3 đã phân tích và lựa chọn các công nghệ phù hợp để hiện thực hóa các yêu cầu từ Chương 2. Vue 3 cùng Tailwind CSS và Pinia tạo nên giao diện tương

tác linh hoạt; Spring Boot phối hợp Spring Security bảo mật hệ thống bằng JWT; PostgreSQL đảm bảo tính nhất quán của dữ liệu quan hệ. FastAPI kết hợp Gemini và Qwen3-32B đóng vai trò động cơ AI; Neo4j cùng GraphRAG hỗ trợ dịch thuật chuyên ngành chuyên sâu; thuật toán SM-2 quản lý lịch ôn tập flashcard; và Web Speech API mang lại khả năng tương tác giọng nói trực quan. Mỗi liên kết chặt chẽ giữa các công nghệ này tạo nên một hệ thống đồng nhất, sẵn sàng được hiện thực hóa và đánh giá chi tiết trong Chương 4.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày các công nghệ được lựa chọn để xây dựng hệ thống. Chương này mô tả quá trình thiết kế và triển khai ứng dụng dựa trên các công nghệ đó. Nội dung chương bao gồm thiết kế kiến trúc tổng thể, thiết kế chi tiết các thành phần chính, mô hình cơ sở dữ liệu, các chức năng đã xây dựng, hoạt động kiểm thử và mô hình triển khai thử nghiệm.

4.1 Thiết kế kiến trúc

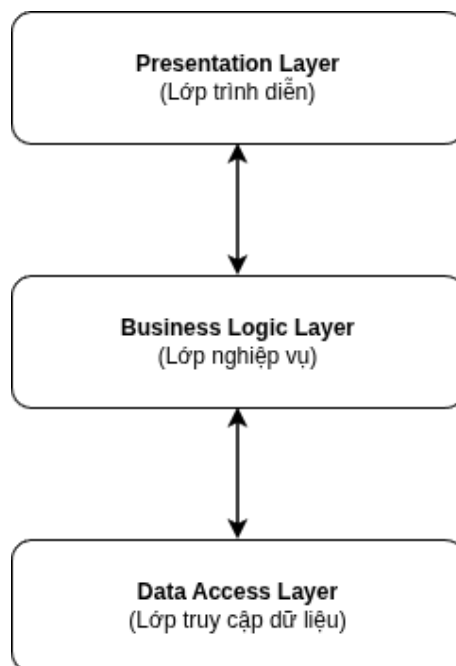
4.1.1 Lựa chọn kiến trúc phần mềm

a, Lựa chọn kiến trúc: kiến trúc ba lớp

Trong đề án này, kiến trúc ba lớp được lựa chọn để xây dựng hệ thống thông minh hỗ trợ học tiếng Nhật. Đây là kiến trúc phần mềm chia ứng dụng thành ba phần có trách nhiệm riêng biệt gồm lớp trình diễn, lớp nghiệp vụ và lớp truy cập dữ liệu.

Kiến trúc ba lớp phù hợp với hệ thống vì ứng dụng cần đồng thời xử lý giao diện web, các quy tắc học tập, xác thực người dùng, lưu trữ dữ liệu và tích hợp chức năng trí tuệ nhân tạo. Việc phân tách trách nhiệm giúp mã nguồn dễ tổ chức, kiểm thử và bảo trì hơn. Khi giao diện hoặc phương thức lưu trữ thay đổi, các quy tắc nghiệp vụ cốt lõi ít bị ảnh hưởng.

Hệ thống được triển khai theo mô hình client-server. Phía client cung cấp giao diện cho người học và gửi yêu cầu đến server qua REST API. Phía server xử lý xác thực, nghiệp vụ và truy cập dữ liệu trước khi trả kết quả. Các chức năng AI được xem là dịch vụ hỗ trợ cho lớp nghiệp vụ, không được gọi trực tiếp từ phía client.



Hình 4.1: Mô hình kiến trúc ba lớp

b, Lớp trình diễn

Lớp trình diễn chịu trách nhiệm hiển thị thông tin và tiếp nhận thao tác của người học thông qua giao diện Vue 3, giao tiếp với backend qua các REST API. Lớp này đảm nhiệm các nhiệm vụ chính:

- Hiển thị giao diện các màn hình chức năng (từ điển, flashcard, ôn tập, dịch thuật, AI Tutor) và tiếp nhận tương tác người dùng.
- Quản lý trạng thái giao diện, điều hướng màn hình và gửi/nhận dữ liệu JSON/-FormData.
- Hiển thị kết quả học tập và thông báo phản hồi từ backend.

c, Lớp nghiệp vụ

Lớp nghiệp vụ (Spring Boot) đóng vai trò trung tâm điều phối hệ thống, xử lý logic học tập và tích hợp dịch vụ. Lớp này đảm nhiệm:

- Xác thực, phân quyền người dùng và quản lý phiên đăng nhập bằng JWT.
- Xử lý nghiệp vụ cốt lõi: tra cứu từ điển, quản lý bộ thẻ và tính lịch ôn tập lặp lại ngắt quãng (SM-2).
- Điều phối phiên hội thoại AI Tutor và chuẩn bị dữ liệu cá nhân hóa cho bài ôn tập (quiz, story).
- Kết nối tầng xử lý AI (Python) và chuẩn hóa kết quả trước khi phản hồi client.

Việc sử dụng các giao diện trung gian cho dịch vụ AI và âm thanh giúp lớp

ng nghiệp vụ độc lập với công nghệ cụ thể, dễ dàng thay thế hoặc nâng cấp mô hình.

d, Lớp truy cập dữ liệu

Lớp truy cập dữ liệu chịu trách nhiệm giao tiếp với các nguồn lưu trữ vật lý để thực hiện các thao tác truy vấn và chuyển dữ liệu theo yêu cầu từ lớp nghiệp vụ. Các nhiệm vụ chính gồm:

- Lưu trữ thông tin tài khoản, hồ sơ người học, dữ liệu từ điển và bình luận.
- Lưu trữ thông số học tập SRS, lịch ôn tập, lịch sử hội thoại và kết quả học tập.
- Cung cấp dữ liệu tri thức chuyên ngành phục vụ dịch thuật.

Hệ thống kết hợp PostgreSQL (dữ liệu nghiệp vụ quan hệ có cấu trúc) và Neo4j (đồ thị tri thức ngữ nghĩa tiếng Nhật) để tối ưu hóa hiệu quả lưu trữ và truy vấn.

e, Thành phần xử lý AI mở rộng

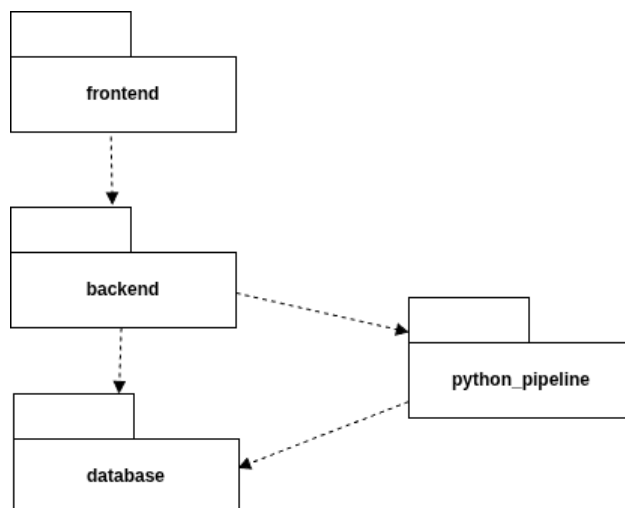
Thành phần xử lý AI bằng Python đóng vai trò mở rộng, hỗ trợ các tác vụ xử lý ngôn ngữ tự nhiên phức tạp như sinh nội dung ôn tập, phản hồi hội thoại, sửa lỗi và dịch chuyên ngành.

Thành phần này hoạt động không trạng thái (stateless) dưới sự điều phối của backend Spring Boot. Cách tổ chức này đảm bảo giao diện không phụ thuộc trực tiếp vào mô hình AI hay khóa dịch vụ, đồng thời dễ dàng nâng cấp mô hình độc lập.

Tóm lại, kiến trúc hệ thống duy trì cấu trúc ba lớp cốt lõi với thành phần AI là phần mở rộng linh hoạt. Các gói chi tiết và mối quan hệ giữa chúng sẽ được trình bày ở các mục tiếp theo.

4.1.2 Thiết kế tổng quan

Ở mức tổng quan, hệ thống được chia thành bốn gói chính gồm `frontend`, `backend`, `database` và `python_pipeline`. Các gói được sắp xếp theo hướng từ tầng trình diễn xuống tầng nghiệp vụ, tầng xử lý AI và tầng dữ liệu. Mỗi gói đảm nhiệm một nhóm trách nhiệm riêng, giao tiếp qua giao diện xác định và không phụ thuộc vòng lẫn nhau.



Hình 4.2: Biểu đồ phụ thuộc gói tổng quan của hệ thống

a, Gói **frontend**

Gói `frontend` (Vue 3) cung cấp giao diện web và tương tác trực tiếp với người dùng. Các nhiệm vụ chính:

- Hiển thị các màn hình chức năng: từ điển, flashcard, ôn tập, dịch thuật, hồ sơ, AI Tutor.
- Quản lý điều hướng, trạng thái client và trao đổi dữ liệu với backend qua REST API.

Gói chỉ kết nối với `backend`, độc lập hoàn toàn với `database` và `python_pipeline`.

b, Gói **backend**

Gói `backend` (Spring Boot) điều phối toàn bộ nghiệp vụ hệ thống. Các nhiệm vụ chính:

- Cung cấp REST API bảo mật qua JWT và quản lý người dùng.
- Xử lý logic từ điển, flashcard (thuật toán SM-2) và kết quả học tập.
- Gọi `python_pipeline` cho tác vụ AI và thao tác cơ sở dữ liệu qua JPA Repository.

c, Gói **database**

Gói `database` gồm PostgreSQL (dữ liệu quan hệ) và Neo4j (đồ thị tri thức). Các nhiệm vụ chính:

- Lưu trữ thông tin người dùng, từ điển, flashcard, lịch học SRS và lịch sử hội thoại.
- Lưu trữ các quan hệ ngữ nghĩa từ vựng tiếng Nhật phục vụ dịch chuyên ngành.

Backend truy cập PostgreSQL qua JPA, còn `python_pipeline` kết nối Neo4j bằng Neo4j Driver.

d, Gói `python_pipeline`

Gói `python_pipeline` (Python) thực hiện các tác vụ AI. Các nhiệm vụ chính:

- Sinh nội dung ôn tập cá nhân hóa (quiz, story) từ dữ liệu đầu vào của backend.
- Xử lý hội thoại AI Tutor (phản hồi, sửa lỗi, gợi ý từ vựng).
- Thực hiện dịch thuật chuyên ngành bằng GraphRAG kết hợp LLM và Neo4j.

e, Quan hệ phụ thuộc giữa các gói

Mỗi quan hệ phụ thuộc được thiết lập một chiều từ trên xuống dưới, tránh phụ thuộc vòng:

- **frontend** → **backend**: Giao tiếp qua REST API; không truy cập database hay AI pipeline.
- **backend** → **database**: Lưu trữ dữ liệu nghiệp vụ PostgreSQL thông qua JPA.
- **backend** → **python_pipeline**: Gọi dịch vụ AI qua REST API nội bộ (FastAPI) hoặc CLI.
- **python_pipeline** → **database**: Truy cập Neo4j Driver để lấy dữ liệu tri thức hỗ trợ dịch thuật.

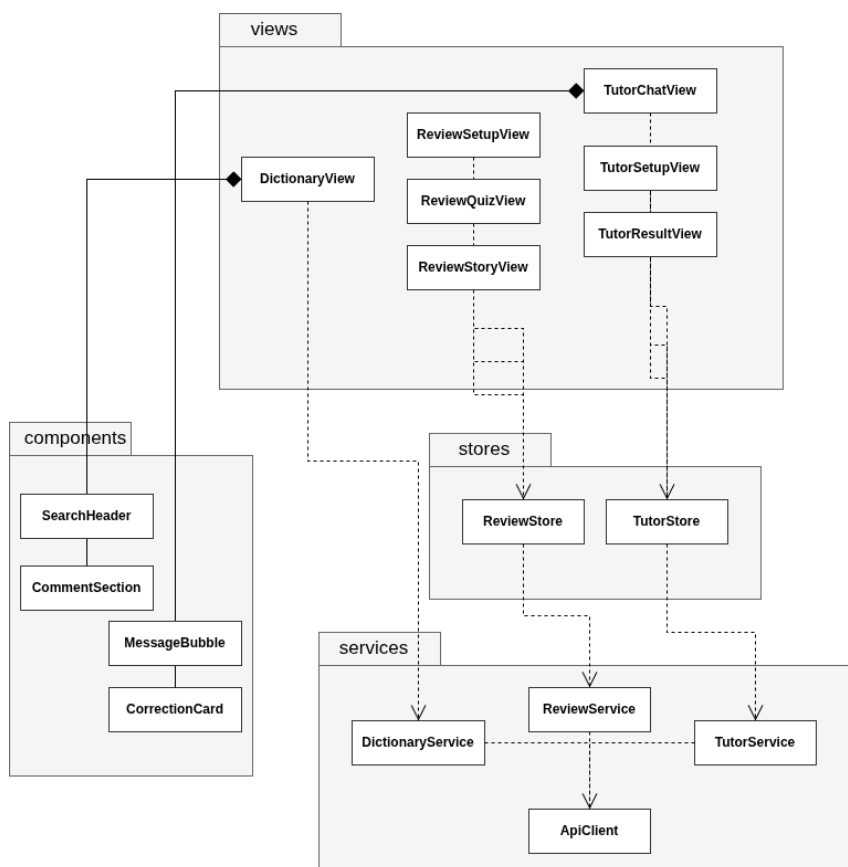
Kiến trúc này đảm bảo tính đóng gói cao, giúp dễ dàng bảo trì và phát triển độc lập các phân hệ.

4.1.3 Thiết kế chi tiết gói

Phần này trình bày thiết kế lớp bên trong từng gói chính. Để biểu đồ dễ đọc, các lớp được nhóm theo chức năng liên quan và chỉ hiển thị tên lớp, không liệt kê thuộc tính hoặc phương thức. Đường nét đứt biểu diễn quan hệ phụ thuộc; tam giác rỗng biểu diễn kế thừa hoặc thực thi interface; hình thoi rỗng biểu diễn kết tập; hình thoi đặc biểu diễn hợp thành; đường liền biểu diễn liên kết.

a, Thiết kế gói **frontend**

Gói `frontend` được tổ chức thành các nhóm `views`, `components`, `stores` và `services`. View sử dụng component để tạo giao diện, phụ thuộc store để quản lý trạng thái và gọi service để trao đổi với backend. Các service chức năng sử dụng chung lớp cấu hình API.

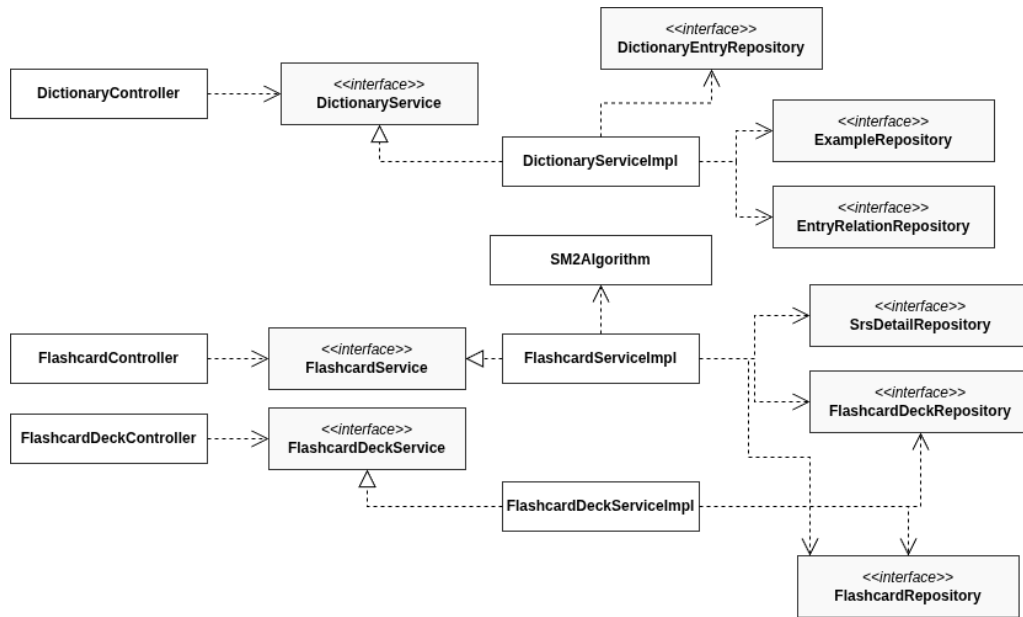


Hình 4.3: Biểu đồ lớp đại diện của gói frontend

DictionaryView hợp thành từ các component tìm kiếm và bình luận, đồng thời phụ thuộc DictionaryService để lấy dữ liệu. Nhóm màn hình Review dùng chung ReviewStore; nhóm màn hình Tutor dùng chung TutorStore. Store lưu trạng thái của luồng chức năng và phụ thuộc service tương ứng. Các service cuối cùng sử dụng ApiClient, nơi cấu hình Axios, JWT và cơ chế làm mới token.

b, Thiết kế gói backend

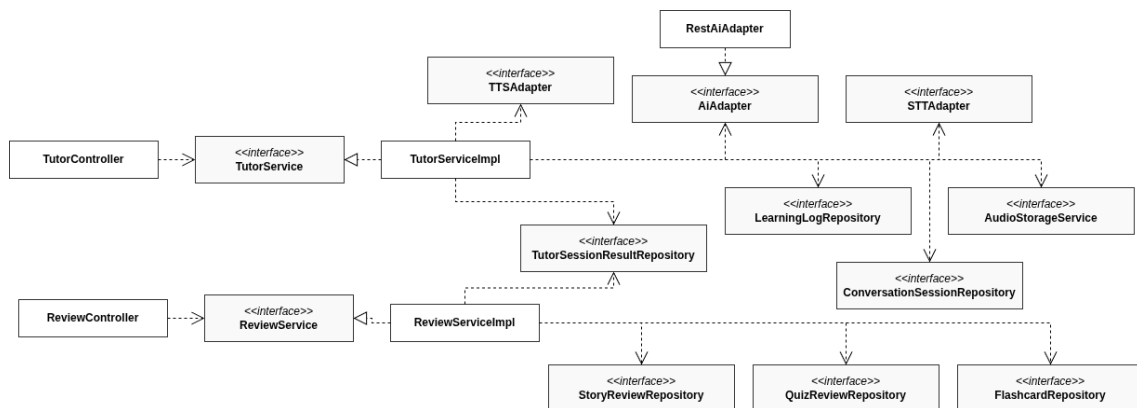
Gói backend tuân theo luồng Controller–Service–Repository. Controller chỉ tiếp nhận yêu cầu và phụ thuộc vào service interface. Lớp ServiceImpl thực thi interface, chứa nghiệp vụ và phụ thuộc repository hoặc lớp tiện ích.



Hình 4.4: Thiết kế các lớp nghiệp vụ từ điển và flashcard trong gói backend

Nhóm từ điển tách DictionaryController khỏi lớp cài đặt bằng DictionaryService. DictionaryServiceImpl sử dụng các repository riêng cho mục từ, ví dụ và quan hệ mục từ. Nhóm flashcard cũng tách nghiệp vụ bộ thẻ và từng thẻ thành hai service. FlashcardServiceImpl sử dụng SM2Algorithm để tính trạng thái ôn tập, sau đó lưu kết quả qua các repository.

Các chức năng AI Tutor và ôn tập thông minh sử dụng cùng cấu trúc nhưng bổ sung adapter để tách nghiệp vụ khỏi phương thức gọi Python.

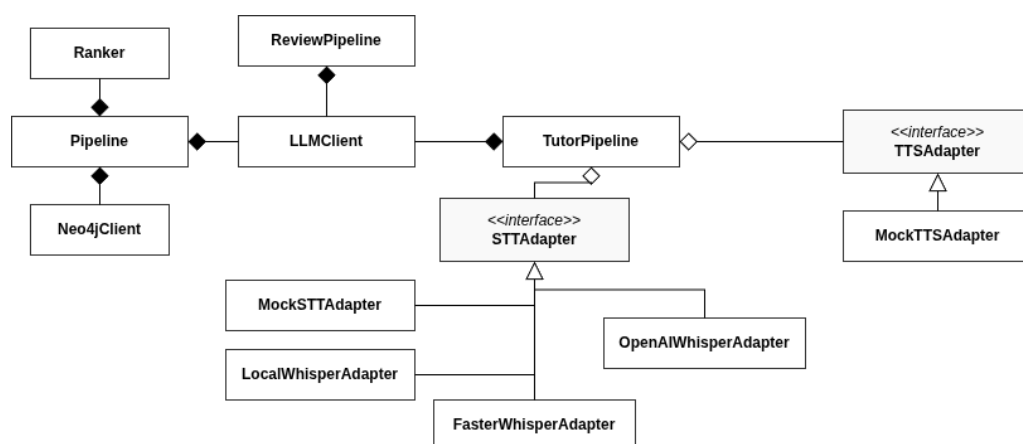


Hình 4.5: Thiết kế các lớp AI Tutor và Review trong gói backend

TutorServiceImpl điều phối phiên học, lưu lịch sử và gọi các interface AI, STT, TTS hoặc lưu trữ âm thanh. RestAiAdapter thực thi AiAdapter và phụ trách giao tiếp với FastAPI. ReviewServiceImpl lấy flashcard và lỗi gần đây, gọi pipeline tạo nội dung, sau đó lưu kết quả quiz hoặc story. Thiết kế qua interface cho phép thay thế adapter thật bằng lớp mock khi kiểm thử.

c, Thiết kế gói `python_pipeline`

Gói `python_pipeline` gồm ba pipeline chính: hội thoại, ôn tập và dịch chuyên ngành. Các pipeline sử dụng chung `LLMClient`; pipeline dịch bổ sung `Neo4jClient` và `Ranker`. Các adapter `STT/TTS` sử dụng quan hệ kế thừa để thay đổi phương thức xử lý âm thanh.



Hình 4.6: Biểu đồ lớp của gói `python_pipeline`

`TutorPipeline` xây dựng ngữ cảnh hội thoại, sử dụng `LLMClient` để sinh phản hồi và lựa chọn một cài đặt `STT/TTS` phù hợp. `ReviewPipeline` sử dụng cùng client để sinh quiz hoặc story. Lớp `Pipeline` điều phối luồng dịch: truy vấn tri thức qua `Neo4jClient`, xếp hạng nghĩa bằng `Ranker`, tạo prompt và gọi `LLMClient`. Các lớp kế thừa `STTAdapter` cung cấp nhiều phương án nhận dạng giọng nói nhưng giữ cùng một giao diện sử dụng.

Thiết kế trên duy trì hướng phụ thuộc từ lớp điều phối đến interface hoặc lớp hạ tầng. Các lớp lưu trữ và pipeline không phụ thuộc ngược vào controller hay giao diện. Nhờ vậy, từng gói có thể được kiểm thử hoặc thay đổi cài đặt với phạm vi ảnh hưởng được giới hạn.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện của hệ thống được thiết kế cho ứng dụng web hỗ trợ học tiếng Nhật. Mục tiêu là cung cấp bố cục thống nhất giữa các chức năng, giúp người học dễ nhận biết vị trí điều hướng và tập trung vào nội dung học. Các hình trong mục này là wireframe dùng để xác định cấu trúc màn hình và vị trí thành phần, không phải giao diện của sản phẩm sau cùng.

a, Thông số kỹ thuật

Wireframe được xây dựng trên độ phân giải cơ sở 1920×1080 pixel, tương ứng với tỷ lệ 16:9. Đây là kích thước tham chiếu để xác định tỷ lệ giữa header, sidebar và vùng nội dung. Khi hiện thực, giao diện sử dụng thiết kế responsive để thích nghi với các độ phân giải nhỏ hơn.

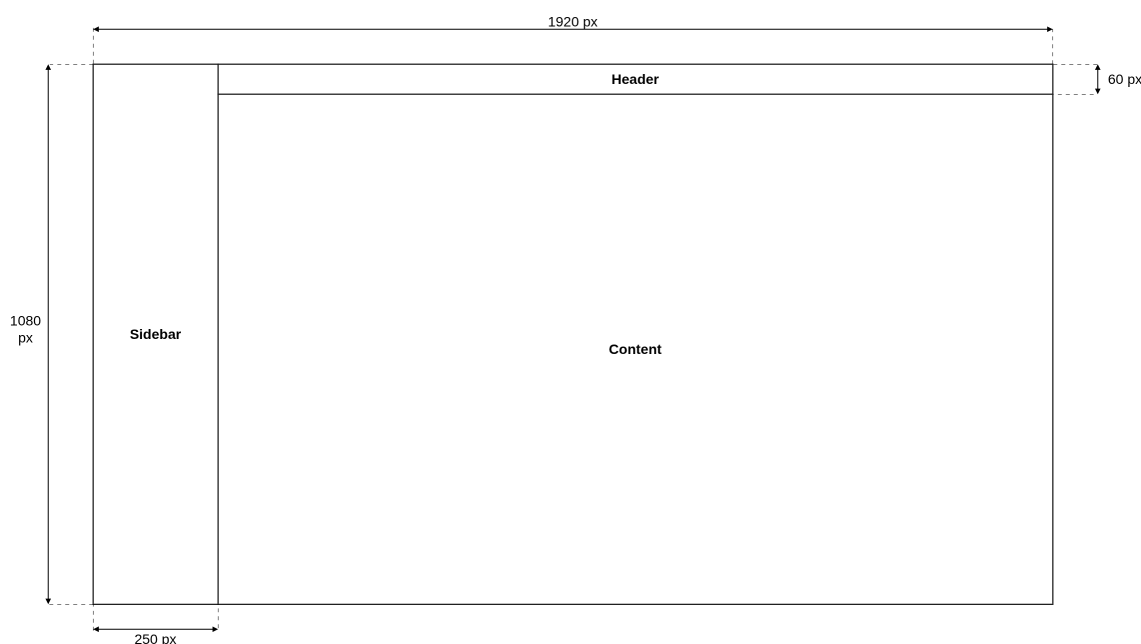
STT	Thông tin	Chi tiết
1	Độ phân giải thiết kế cơ sở	1920×1080 pixel
2	Độ phân giải máy tính tối thiểu	1366×768 pixel
3	Tỷ lệ khung hình ưu tiên	16:9
4	Kích thước màn hình mục tiêu	Máy tính cá nhân từ 13 inch trở lên
5	Số lượng màu hỗ trợ	Không gian màu RGB, tối thiểu 24 bit
6	Trình duyệt hỗ trợ	Chrome, Edge, Firefox và Safari phiên bản hiện đại
7	Chiều cao header	60 pixel trên bản thiết kế cơ sở
8	Chiều rộng sidebar	250 pixel trên bản thiết kế cơ sở
9	Kiểu chữ	Plus Jakarta Sans cho tiêu đề; Inter cho nội dung
10	Cỡ chữ thông thường	14–16 pixel; tiêu đề từ 24 pixel
11	Nút bấm	Chiều cao tối thiểu 40 pixel; vùng bấm rõ ràng
12	Màu sắc chủ đạo	Xanh #45617D, nền #F9F9F9, chữ #2D3435
13	Vị trí thông báo	Gần thao tác phát sinh; thông báo chung ở góc trên bên phải

Bảng 4.1: Thông số kỹ thuật thiết kế giao diện

Các màn hình sử dụng chung một số quy chuẩn. Nút chính có nền xanh, chữ sáng và được bo tròn; nút phụ dùng nền trung tính. Ô nhập liệu có nhãn hoặc nội dung gợi ý rõ ràng và hiển thị trạng thái khi được chọn. Các khối nội dung dùng góc bo mềm, khoảng trắng và màu nền khác nhau để phân tách. Thông báo lỗi được đặt gần trường nhập hoặc thao tác gây lỗi; phản hồi thành công ngắn được hiển thị ở vị trí dễ quan sát nhưng không che nội dung chính.

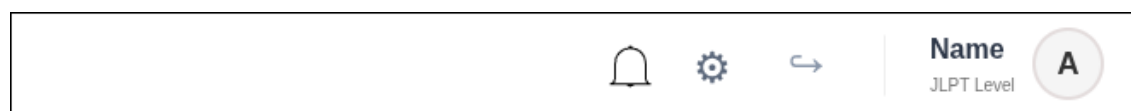
b, Bố cục giao diện

Bố cục tổng thể. Giao diện chính gồm ba vùng: sidebar, header và content. Sidebar nằm cố định bên trái với chiều rộng 250 pixel. Header nằm phía trên vùng content và có chiều cao 60 pixel. Phần diện tích còn lại được dành cho nội dung của từng chức năng. Cấu trúc này giữ vị trí điều hướng ổn định khi người dùng chuyển giữa từ điển, dịch thuật, flashcard, ôn tập và AI Tutor.



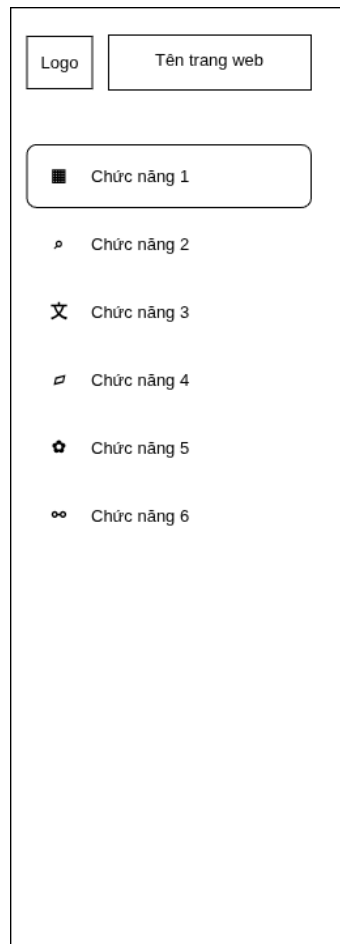
Hình 4.7: Wireframe bố cục chung trên máy tính

Bố cục header. Header tập trung các thao tác dùng chung của hệ thống. Phía bên phải lần lượt bố trí biểu tượng thông báo, thiết lập, đăng xuất và thông tin người dùng. Thông tin người dùng gồm tên, cấp độ JLPT và ảnh đại diện. Các thành phần được căn theo chiều ngang để không chiếm nhiều không gian của vùng content.



Hình 4.8: Wireframe bố cục header

Bố cục sidebar. Sidebar là khu vực điều hướng chính. Phần trên cùng chứa logo và tên hệ thống; bên dưới là danh sách các chức năng kèm biểu tượng. Chức năng đang được sử dụng được làm nổi bật để người dùng nhận biết vị trí hiện tại. Thứ tự các chức năng được giữ thống nhất trên mọi màn hình.

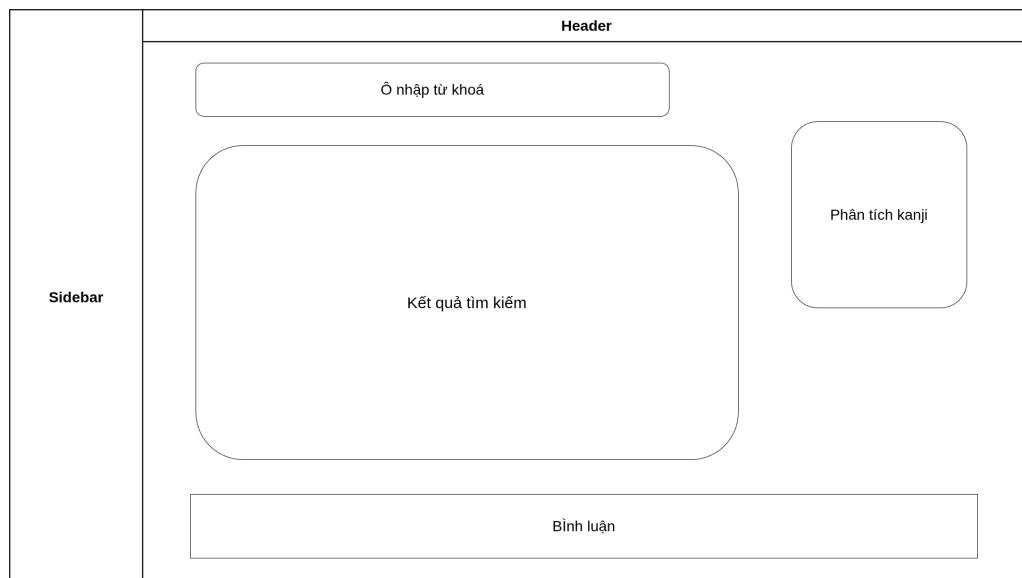


Hình 4.9: Wireframe bố cục sidebar

c, Màn hình minh họa

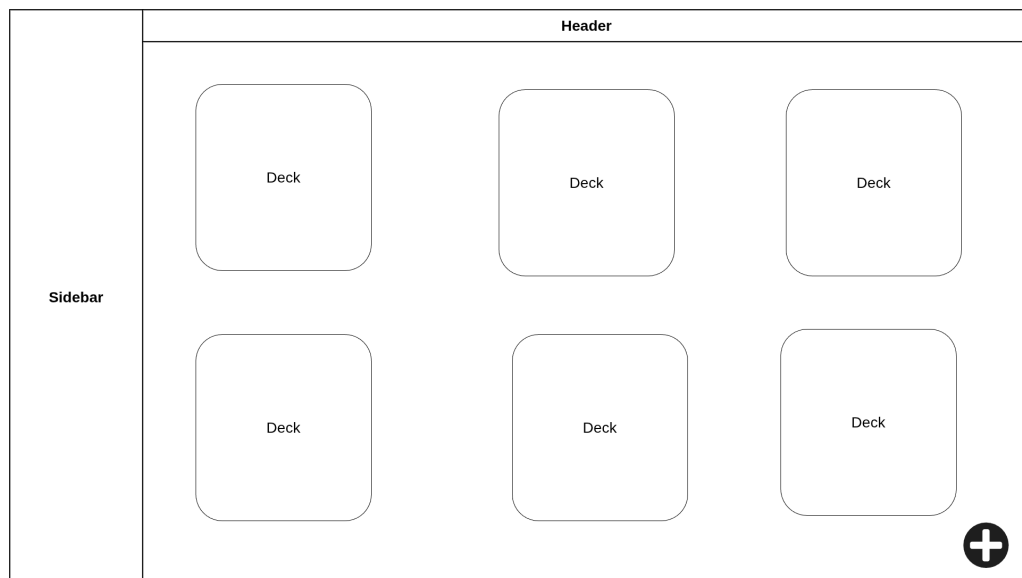
Do các màn hình sử dụng chung header và sidebar, phần này trình bày hai wireframe đại diện cho hai chức năng quan trọng là tra cứu tiếng Nhật và quản lý bộ thể. Các hình chỉ mô tả bố cục dự kiến, chưa thể hiện màu sắc, hiệu ứng hoặc dữ liệu của sản phẩm hoàn thiện.

Màn hình tra cứu tiếng Nhật. Màn hình tra cứu đặt ô nhập từ khóa ở đầu vùng content để người học có thể thao tác ngay khi mở trang. Kết quả tìm kiếm chiếm phần lớn diện tích trung tâm. Khối phân tích kanji nằm bên phải nhằm cung cấp thông tin bổ sung mà không làm gián đoạn quá trình đọc kết quả. Phần bình luận được đặt phía dưới, tách khỏi nội dung tra cứu chính nhưng vẫn thuộc cùng màn hình.



Hình 4.10: Wireframe màn hình tra cứu tiếng Nhật

Màn hình quản lý bộ thẻ. Màn hình quản lý bộ thẻ sử dụng bố cục lưới ba cột trên độ phân giải thiết kế cơ sở. Mỗi khối đại diện cho một bộ thẻ và là điểm truy cập đến danh sách thẻ hoặc phiên học tương ứng. Cách bố trí dạng lưới giúp người học quan sát nhiều bộ thẻ cùng lúc. Nút thêm mới được đặt nổi ở góc dưới bên phải để dễ nhận biết nhưng không chiếm diện tích của danh sách.



Hình 4.11: Wireframe màn hình quản lý bộ thẻ

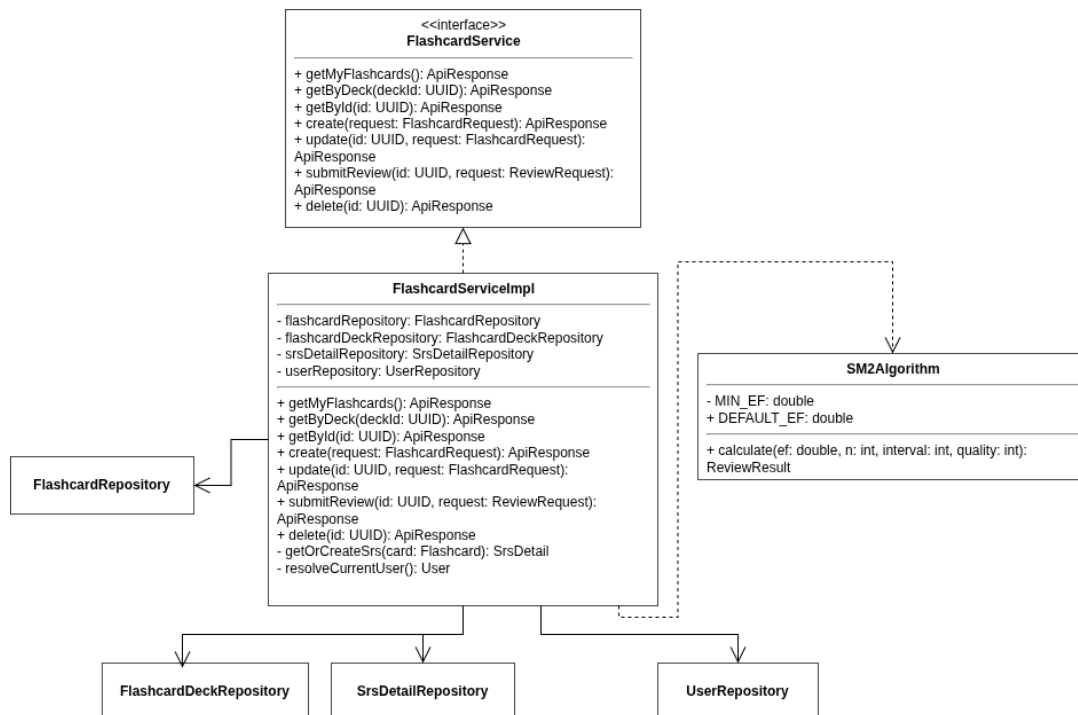
Các wireframe trên xác định cấu trúc chung, vị trí điều hướng và mức độ ưu tiên của thông tin. Trong quá trình hiện thực, màu sắc, khoảng cách, kích thước thẻ và số cột có thể được điều chỉnh theo độ rộng thiết bị, nhưng quan hệ giữa header, sidebar và content vẫn được duy trì thống nhất.

4.2.2 Thiết kế lớp

Phần này trình bày thiết kế chi tiết cho ba lớp nghiệp vụ cốt lõi: `FlashcardServiceImpl`, `TutorServiceImpl` và `ReviewServiceImpl`. Các lớp này chịu trách nhiệm hiện thực hóa các chức năng chính của hệ thống bao gồm học lặp lại ngắt quãng (SM-2), luyện hội thoại với AI và ôn tập cá nhân hóa. Trong các biểu đồ lớp dưới đây, thuộc tính và phương thức private ký hiệu bằng -, public bằng +; các phương thức getter, setter và constructor sinh tự động bằng Lombok được ẩn để tăng tính tường minh.

a, Lớp `FlashcardServiceImpl`

Lớp `FlashcardServiceImpl` chịu trách nhiệm quản lý vòng đời của flashcard, kiểm tra quyền sở hữu bộ thẻ và cập nhật lịch ôn tập SRS của người dùng.

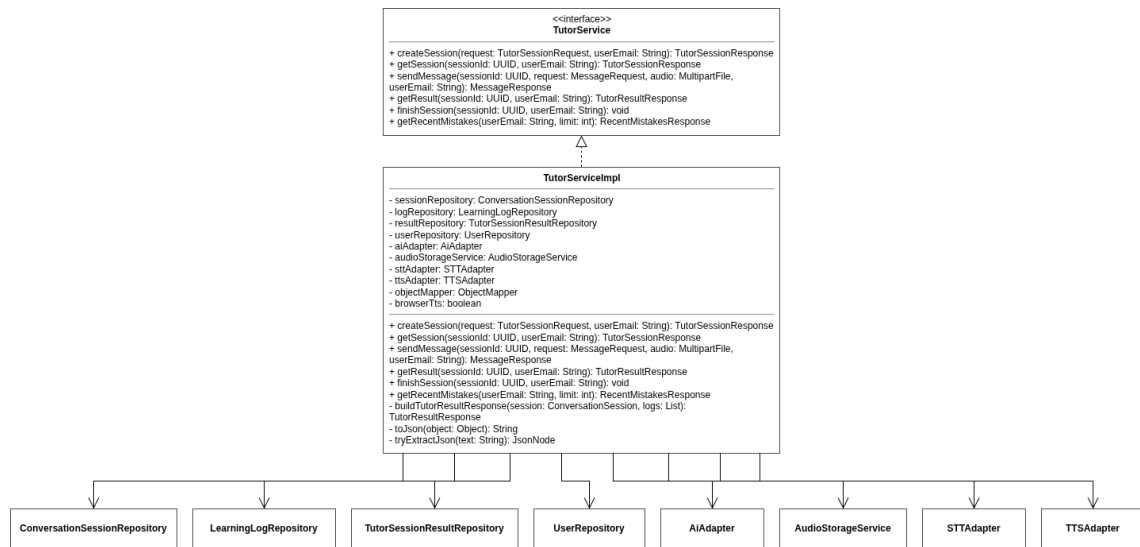


Hình 4.12: Thiết kế chi tiết lớp `FlashcardServiceImpl`

Các repository (`flashcard`, `flashcardDeck`, `srsDetail`) được tiêm qua constructor để truy xuất dữ liệu. Lớp cung cấp các phương thức chính: `getMyFlashcards()` và `getByDeck()` (truy xuất danh sách thẻ); `getById()` (lấy chi tiết thẻ và trạng thái SRS); `create()` (tạo thẻ mới kèm SRS mặc định); `update()` (cập nhật nội dung); `submitReview()` (gọi thuật toán SM-2 cập nhật lịch ôn tập); và `delete()` (xóa thẻ sau khi xác minh quyền sở hữu).

b, Lớp TutorServiceImpl

Lớp TutorServiceImpl điều phối các phiên luyện hội thoại AI Tutor, quản lý lịch sử trò chuyện và lưu kết quả đánh giá.

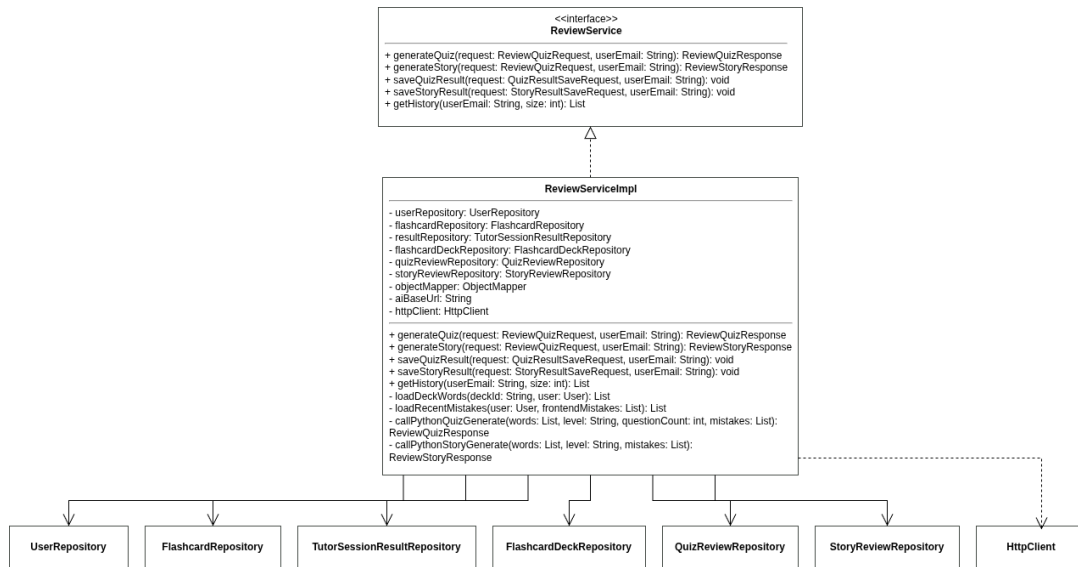


Hình 4.13: Thiết kế chi tiết lớp TutorServiceImpl

Lớp giao tiếp với mô hình AI thông qua aiAdapter và quản lý âm thanh qua các adapter STT/TTS cùng dịch vụ lưu trữ. Các phương thức nghiệp vụ chính bao gồm: createSession() (khởi tạo phiên hội thoại và tạo lời chào); sendMessage() (tiếp nhận tin nhắn, điều phối xử lý âm thanh, gọi AI và lưu phản hồi dạng JSON); finishSession() (kết thúc phiên, tổng hợp lỗi ngữ pháp, từ vựng mới và lưu kết quả); và getRecentMistakes() (truy xuất các lỗi gần đây phục vụ ôn tập).

c, Lớp ReviewServiceImpl

Lớp ReviewServiceImpl đóng vai trò kết nối dữ liệu học tập cá nhân (flashcard đến hạn, lỗi sai gần đây) với Python pipeline để sinh nội dung ôn tập cá nhân hóa.



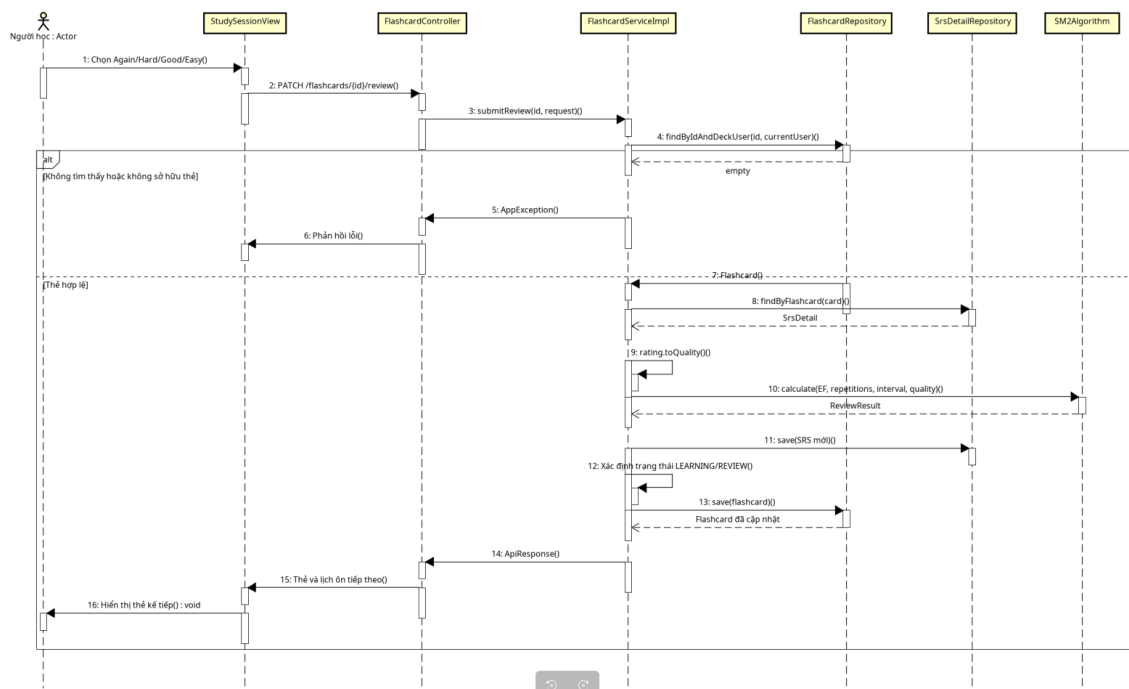
Hình 4.14: Thiết kế chi tiết lớp ReviewServiceImpl

Lớp sử dụng `httpClient` để gọi các API sinh câu hỏi trắc nghiệm hoặc câu chuyện tương tác từ Python pipeline. Các phương thức cốt lõi gồm: `generateQuiz()` và `generateStory()` (chuẩn bị dữ liệu từ vựng/lỗi sai và gọi dịch vụ AI); `saveQuizResult()` và `saveStoryResult()` (lưu kết quả làm bài ôn tập); và `getHistory()` (truy xuất lịch sử ôn tập tổng hợp).

d, Luồng truyền thông điệp giữa các đối tượng

Để minh họa sự phối hợp hoạt động, phần này trình bày hai biểu đồ trình tự cho các use case cốt lõi.

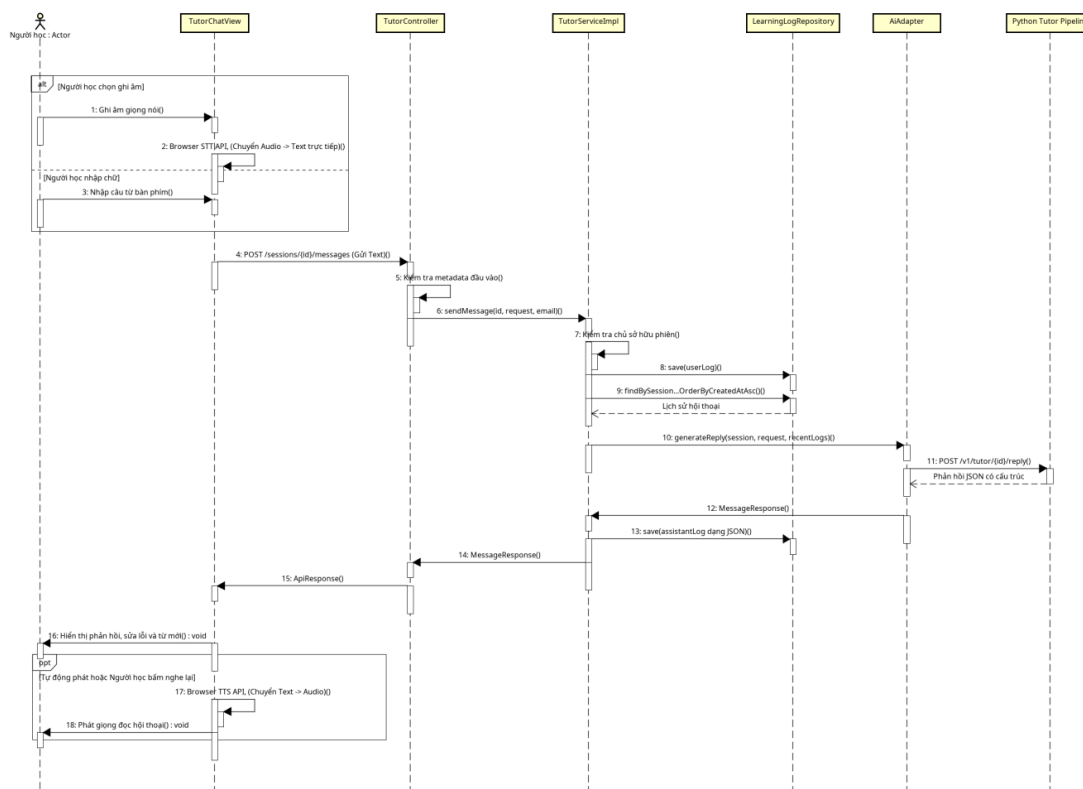
Use case ôn tập flashcard.



Hình 4.15: Biểu đồ trình tự use case ôn tập flashcard

Mỗi đánh giá của người dùng được chuyển thành điểm chất lượng để đưa vào SM2Algorithm. Lớp thuật toán này không giữ trạng thái, chỉ nhận tham số và trả về kết quả tính toán SRS mới. FlashcardServiceImpl áp kết quả này vào SrsDetail và lưu vào cơ sở dữ liệu trong cùng một giao dịch.

Use case gửi tin nhắn trong phiên AI Tutor.



Hình 4.16: Biểu đồ trình tự use case gửi tin nhắn AI Tutor

Khi người học gửi tin nhắn, backend xác thực phiên hội thoại và lấy lịch sử tin nhắn gần nhất làm ngữ cảnh. Do việc xử lý giọng nói (STT, TTS) được thực hiện hoàn toàn tại Browser API ở phía client, backend chỉ truyền nhận văn bản và gọi Python AI pipeline. Phản hồi có cấu trúc từ AI (nội dung hội thoại, dịch nghĩa, sửa lỗi ngữ pháp) được lưu dưới dạng JSON trong LearningLog để lưu trữ và hiển thị.

Tóm lại, các lớp service trên thực hiện toàn bộ logic nghiệp vụ cốt lõi, tương tác với cơ sở dữ liệu thông qua repository và kết nối với các phân hệ AI. Việc lập trình hướng giao tiếp qua interface giúp tăng tính đóng gói và linh hoạt cho hệ thống.

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng kết hợp PostgreSQL và Neo4j. PostgreSQL là cơ sở dữ liệu chính, lưu các dữ liệu nghiệp vụ có cấu trúc và yêu cầu ràng buộc toàn vẹn như tài khoản, từ điển, flashcard, trạng thái SRS, phiên AI Tutor và lịch sử ôn tập. Neo4j lưu đồ thị tri thức phục vụ chức năng dịch chuyên sâu, trong đó các nghĩa của từ được liên kết với miền chuyên ngành và các từ gợi ý ngữ cảnh. Việc phân chia này giúp dữ liệu giao dịch được quản lý bằng mô hình quan hệ, còn các truy vấn nhiều bước giữa từ, nghĩa và ngữ cảnh được thực hiện phù hợp hơn trên mô hình đồ thị.

- **Tự tham chiếu và Nhiều-Nhiều:** Bảng `comments` tự tham chiếu qua `parent_id` để hỗ trợ trả lời bình luận nhiều cấp. Bảng trung gian `entry_relations` liên kết các mục từ trong `dictionary_entries` để biểu diễn từ đồng nghĩa, trái nghĩa, từ ghép.

b, Thiết kế chi tiết cơ sở dữ liệu PostgreSQL

Thiết kế chi tiết các bảng được tóm tắt theo từng nhóm nghiệp vụ trong bảng dưới đây. Các thuộc tính có ràng buộc `NOT NULL` là bắt buộc; các trường `JSONB` được sử dụng cho dữ liệu động từ phản hồi AI để tránh chia nhỏ bảng phức tạp.

Tên bảng	Thuộc tính chính	Mô tả và ràng buộc
Nhóm Người dùng và Xác thực		
<code>users</code>	<code>id</code> , <code>username</code> , <code>email</code> , <code>password_hash</code> , <code>role</code> , <code>target_level</code> , <code>created_at</code>	Lưu tài khoản người học; <code>username</code> và <code>email</code> duy nhất; mật khẩu lưu dưới dạng băm; <code>role</code> mặc định <code>USER</code> .
<code>user_profiles</code>	<code>user_id</code> , <code>streak_count</code> , <code>last_active</code>	Thông tin học tập mở rộng; <code>user_id</code> là PK kiêm FK tới <code>users</code> .
<code>app_refresh_tokens</code>	<code>id</code> , <code>user_id</code> , <code>token</code> , <code>expiry_date</code>	Refresh token phục vụ xoay vòng JWT; <code>token</code> có ràng buộc duy nhất.
Nhóm Từ điển và Bình luận		
<code>dictionary_entries</code>	<code>id</code> , <code>entry_type</code> , <code>text</code> , <code>reading</code> , <code>meaning_vn</code> , ...	Lưu mục từ (từ vựng, <code>kanji</code> , ngữ pháp); <code>entry_type</code> nhận <code>word</code> , <code>kanji</code> , <code>grammar</code> .
<code>entry_relations</code>	<code>id</code> , <code>source_id</code> , <code>target_id</code> , <code>relation_type</code> , <code>order_index</code>	Biểu diễn quan hệ tự liên kết (đồng nghĩa, trái nghĩa, từ ghép) giữa hai mục từ.
<code>examples</code>	<code>id</code> , <code>entry_id</code> , <code>japanese_sentence</code> , <code>vietnamese_sentence</code>	Lưu câu ví dụ tiếng Nhật kèm bản dịch tiếng Việt cho mục từ.

Tên bảng	Thuộc tính chính	Mô tả và ràng buộc
comments	id, entry_id, user_id, parent_id, content, ...	Lưu bình luận về mục từ; hỗ trợ phản hồi nhiều cấp.
Nhóm Flashcard và SRS		
flashcard_decks	id, user_id, name, description, is_public, ...	Lưu bộ thẻ flashcard; is_public mặc định false.
flashcards	id, deck_id, front_text, front_reading, back_text, ...	Nội dung flashcard; status nhận new, learning, review.
srs_details	flashcard_id, ease_factor, interval_days, repetitions, ...	Trạng thái SRS theo thuật toán SM-2; chia sẻ khóa với flashcards.
Nhóm AI Tutor và Ôn tập thông minh		
conversation_sessions	id, user_id, scenario_name, target_words, ...	Lưu phiên AI Tutor; từ vựng mục tiêu lưu dạng mảng JSONB.
learning_logs	id, session_id, role, content, created_at	Lịch sử hội thoại từng lượt; nội dung hội thoại chi tiết lưu JSONB.
tutor_session_results	id, session_id, overall_score, mistakes, new_vocabulary, ...	Kết quả phiên học (điểm, số lượt nói, danh sách lỗi sai và từ mới lưu JSONB).
quiz_reviews	id, user_id, deck_id, score, total_questions, questions_data	Lịch sử ôn tập quiz; danh sách câu hỏi lưu dạng JSONB.

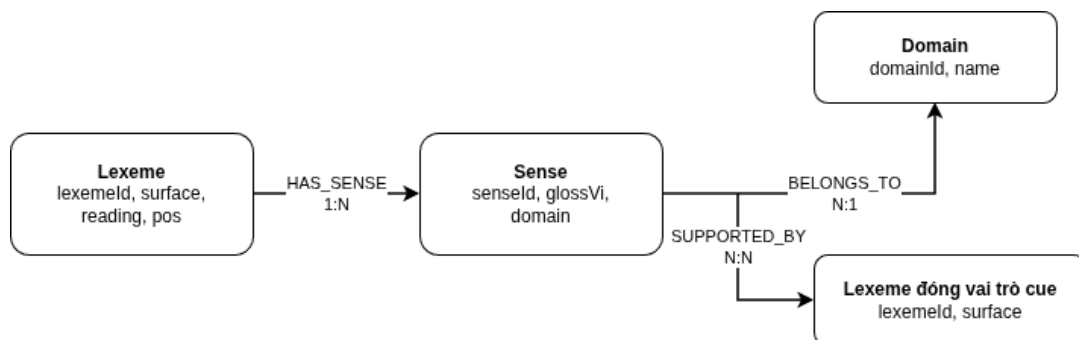
Tên bảng	Thuộc tính chính	Mô tả và ràng buộc
story_reviews	id, user_id, deck_id, title, score, story_data, answers_data	Lịch sử ôn tập story; chi tiết story và câu trả lời lưu dạng JSONB.

Bảng 4.2: Thiết kế chi tiết các bảng trong cơ sở dữ liệu PostgreSQL

Hệ thống lập chỉ mục (index) trên các trường truy vấn thường xuyên như `users.email`, `dictionary_entries.text`, `flashcards.deck_id`, và chỉ mục ghép trên (`deck_id`, `status`) để tối ưu hóa hiệu năng lấy thẻ đến hạn. Các thao tác cập nhật lịch ôn và kết quả hội thoại được thực hiện trong cùng transaction để đảm bảo toàn vẹn dữ liệu.

c, Thiết kế cơ sở dữ liệu đồ thị Neo4j

Neo4j lưu trữ đồ thị tri thức dùng chung cho pipeline dịch chuyên sâu. Mô hình gồm ba loại nút Lexeme, Sense, Domain và ba loại quan hệ HAS_SENSE, BELONGS_TO, SUPPORTED_BY.



Hình 4.18: Mô hình dữ liệu đồ thị Neo4j

Một Lexeme (từ tiếng Nhật) có thể có nhiều Sense (nghĩa dịch tiếng Việt) để biểu diễn từ đa nghĩa. Mỗi Sense liên kết với một Domain (chuyên ngành) để phân biệt ngữ cảnh. Quan hệ SUPPORTED_BY kết nối nghĩa dịch với các từ thường đồng xuất hiện để làm tín hiệu gợi ý.

Thành phần	Thuộc tính/quan hệ	Vai trò
Lexeme	lexemeId, surface, reading, pos	Biểu diễn từ tiếng Nhật; làm tín hiệu ngữ cảnh cho nghĩa khác.
Sense	senseId, glossVi, domain	Biểu diễn nghĩa dịch tiếng Việt; hỗ trợ lọc theo chuyên ngành.

Thành phần	Thuộc tính/quan hệ	Vai trò
Domain	domainId, name	Biểu diễn miền chuyên ngành (y tế, CNTT, kinh tế...).
HAS_SENSE	Lexeme $\rightarrow$ Sense	Xác định các nghĩa tương ứng của một từ vựng.
BELONGS_TO	Sense $\rightarrow$ Domain	Xác định miền chuyên ngành của nghĩa.
SUPPORTED_BY	Sense $\rightarrow$ Lexeme	Biểu diễn các từ đồng xuất hiện hỗ trợ gợi ý ngữ cảnh.

Bảng 4.3: Thiết kế nút và quan hệ trong Neo4j

Chỉ mục được tạo trên `Lexeme.surface` và `Domain.name` để đẩy nhanh tốc độ duyệt đồ thị. Đồ thị tri thức hoạt động hoàn toàn độc lập với cơ sở dữ liệu quan hệ PostgreSQL; sự tách biệt này đảm bảo các nâng cấp về tri thức từ vựng không gây ảnh hưởng đến dữ liệu cá nhân của người học.

4.2.4 Thiết kế module AI

Module AI có vai trò sinh phản hồi hội thoại, tạo nội dung ôn tập và hỗ trợ dịch văn bản tiếng Nhật theo ngữ cảnh chuyên ngành. Module được tách khỏi backend nghiệp vụ và triển khai bằng Python để tận dụng các thư viện xử lý ngôn ngữ, kết nối mô hình ngôn ngữ lớn và Neo4j. Backend vẫn chịu trách nhiệm xác thực, kiểm tra quyền sở hữu, lựa chọn dữ liệu cá nhân hóa và lưu kết quả; module AI chỉ nhận dữ liệu cần thiết, thực hiện xử lý và trả kết quả có cấu trúc.

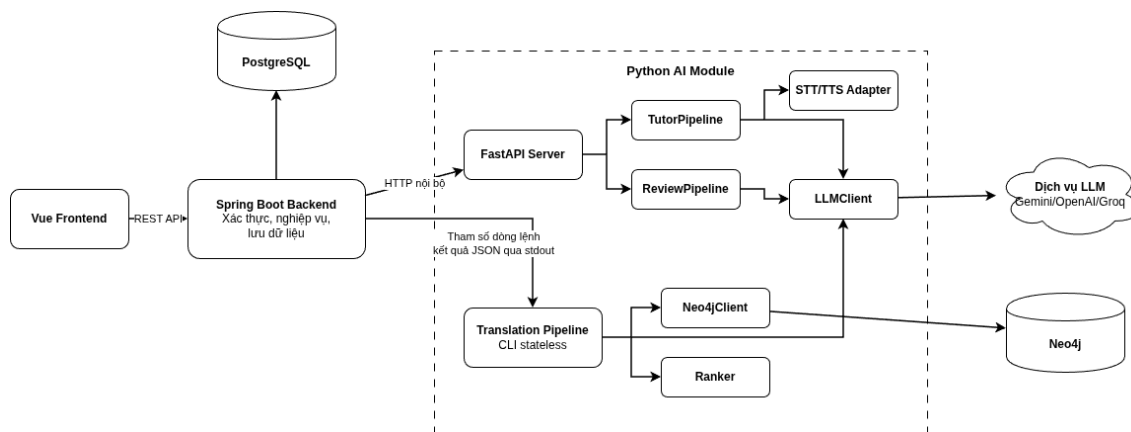
Khác với module học máy truyền thống sử dụng một mô hình đã huấn luyện để dự đoán một giá trị, module của đồ án sử dụng các pipeline điều phối mô hình ngôn ngữ lớn. Chất lượng đầu ra không chỉ phụ thuộc vào mô hình mà còn phụ thuộc vào cách xây dựng prompt, dữ liệu ngữ cảnh, bước truy xuất tri thức, cơ chế kiểm tra JSON và phương án xử lý khi dịch vụ ngoài không khả dụng.

a, Thiết kế tổng quát

Module được tổ chức theo mô hình client-server. Spring Boot backend đóng vai trò client nội bộ; Python AI layer đóng vai trò server xử lý AI. Hai thành phần không dùng chung mô hình entity hay truy cập trực tiếp dữ liệu cá nhân của nhau. Dữ liệu được trao đổi bằng JSON hoặc biểu mẫu HTTP theo hợp đồng đã xác định.

Riêng pipeline dịch chuyên sâu được thiết kế stateless và được backend khởi chạy dưới dạng tiến trình Python. Cách tích hợp này phù hợp với tác vụ độc lập, không cần giữ phiên và có đầu vào duy nhất là văn bản. AI Tutor và AI Review cần

phục vụ nhiều lời gọi liên tiếp nên được cung cấp qua FastAPI chạy tại cổng nội bộ.



Hình 4.19: Kiến trúc tổng quát của module AI

Các thành phần chính gồm:

- `server.py`: khởi tạo FastAPI, kiểm tra và chuẩn hóa request, gọi pipeline tương ứng và trả phản hồi JSON.
- `TutorPipeline`: xây dựng ngữ cảnh hội thoại, tạo prompt theo chủ đề và cấp độ, sinh phản hồi, sửa lỗi và từ vựng mới.
- `ReviewPipeline`: tạo quiz hoặc story từ flashcard, cấp độ JLPT và lỗi gần đây.
- `Pipeline` trong `runner.py`: điều phối luồng dịch chuyên sâu từ tách từ, truy vấn đồ thị, xếp hạng nghĩa đến sinh bản dịch.
- `LLMClient`: cung cấp giao diện gọi mô hình thống nhất; cài đặt hiện tại ưu tiên Gemini và có thể chuyển sang OpenAI hoặc Groq bằng cấu hình.
- `STTAdapter` và `TTSAdapter`: trừu tượng hóa nhận dạng và tổng hợp giọng nói, cho phép dùng cài đặt thật hoặc mock khi kiểm thử.
- `Neo4jClient`: thực hiện truy vấn đọc theo danh sách token, không ghi dữ liệu vào đồ thị trong lúc dịch.
- `Ranker`: tính điểm và lựa chọn nghĩa phù hợp nhất dựa trên miền chuyên ngành và cue ngữ cảnh.

Thiết kế trên giữ module AI không trạng thái ở mức dịch vụ. Trạng thái phiên Tutor, nhật ký hội thoại và kết quả học tập được lưu tại PostgreSQL bởi backend. Khi gọi AI, backend chỉ truyền lịch sử gần nhất và dữ liệu mục tiêu cần thiết. Nhờ đó, việc khởi động lại AI service không làm mất dữ liệu học của người dùng.

b, Thiết kế module API

FastAPI là cổng giao tiếp cho hai nhóm chức năng AI Tutor và AI Review. Các endpoint chỉ được backend gọi trong mạng nội bộ; frontend không gọi trực tiếp module AI. Dữ liệu người dùng đã được backend xác thực trước khi chuyển sang Python.

Phương thức và end-point	Dữ liệu đầu vào	Dữ liệu đầu ra	Chức năng
POST /v1/tutor/{session_id}/initial	Chủ đề, cấp độ, từ vựng mục tiêu	Tin nhắn mở đầu, bản dịch, gợi ý trả lời	Khởi tạo hội thoại theo thiết lập phiên.
POST /v1/tutor/{session_id}/reply	Câu người học, từ mục tiêu, lịch sử gần nhất	Phản hồi tiếng Nhật, tiếng Việt, lỗi sửa, gợi ý, từ mới	Sinh lượt trả lời tiếp theo của AI Tutor.
POST /v1/review/quiz/generate	Danh sách từ, cấp độ, số câu, lỗi gần đây	Mảng câu hỏi trắc nghiệm và cảnh báo	Sinh quiz cá nhân hóa.
POST /v1/review/story/generate	Danh sách từ, cấp độ, lỗi gần đây	Tiêu đề, bối cảnh và các phân đoạn story	Sinh câu chuyện tương tác.
python -m python_pipeline.runner-text ... -json	Văn bản tiếng Nhật	Bản dịch, miễn phát hiện, từ quan trọng, ghi chú	Thực hiện dịch chuyên sâu dưới dạng tiến trình độc lập.

Bảng 4.4: Danh sách API của module AI

Luồng xử lý chung của API gồm bốn bước: nhận request và chuẩn hóa cấu trúc dữ liệu; chuyển đổi dữ liệu thành các metadata học tập tương ứng; xây dựng prompt và gọi `LLMClient`; phân tích, định dạng lại kết quả theo schema và trả về backend. Nếu xảy ra lỗi hoặc phản hồi không thể phân tích, API trả về các mã lỗi có kiểm soát để backend xử lý và hiển thị thông báo an toàn.

c, Thiết kế AI Tutor Pipeline

AI Tutor Pipeline nhận thông tin phiên, danh sách từ mục tiêu, lịch sử hội thoại gần nhất và câu người học vừa gửi để thực hiện hai thao tác: tạo lời mở đầu và tạo lượt phản hồi tiếp theo. Với tương tác giọng nói, backend thực hiện nhận dạng (STT) thành văn bản trước khi truyền vào pipeline.

Đầu ra của pipeline được chuẩn hóa thành một cấu trúc JSON thống nhất đóng vai trò hợp đồng dữ liệu giữa Python service và backend. Cấu trúc này bao gồm các trường chính:

- `contentJa` (chuỗi): Phản hồi bằng tiếng Nhật phù hợp với cấp độ JLPT của phiên học.
- `contentVn` (chuỗi): Bản dịch nghĩa hoặc giải thích ngắn gọn bằng tiếng Việt.
- `corrections` (mảng đối tượng): Danh sách các câu gốc, câu sửa lỗi và giải thích chi tiết lỗi trợ từ, ngữ pháp của người học.
- `suggestions` (mảng chuỗi): Các gợi ý câu trả lời nhanh giúp người học dễ dàng tiếp tục hội thoại.
- `newVocabulary` (mảng đối tượng): Các từ vựng mới hữu ích xuất hiện trong câu phản hồi.
- `audioUrl` (chuỗi hoặc rỗng): Đường dẫn tệp âm thanh phản hồi được tổng hợp bởi dịch vụ TTS.

Toàn bộ trạng thái phiên, nhật ký hội thoại được backend lưu trữ tại PostgreSQL; pipeline hoạt động stateless và không trực tiếp quản lý dữ liệu người dùng. Các thuật toán chi tiết về prompt sửa lỗi và tích hợp âm thanh được trình bày tại Mục 5.4.

d, Thiết kế AI Review Pipeline

AI Review Pipeline cung cấp hai dịch vụ chính là sinh câu hỏi trắc nghiệm (quiz) và sinh câu chuyện tương tác (story). Pipeline hoạt động không trạng thái (stateless): backend chịu trách nhiệm truy vấn PostgreSQL, lựa chọn bộ flashcard, thu thập lỗi gần đây và cấp độ JLPT của người dùng trước khi gửi yêu cầu.

Dữ liệu quiz được cấu trúc thành danh sách câu hỏi trắc nghiệm (chứa câu hỏi điền từ, các lựa chọn, đáp án đúng và giải thích). Dữ liệu story bao gồm tiêu đề, bối cảnh dẫn dắt và danh sách các phân đoạn tương tác. Cả hai phản hồi đều hỗ trợ trường `warning` giúp thông báo cho hệ thống khi dữ liệu sinh ra chỉ đáp ứng được một phần. Các cơ chế xây dựng prompt, phân tích cú pháp và tự phục hồi dữ liệu được trình bày chi tiết tại Mục 5.3.

e, Thiết kế Specialized Translation Pipeline

Specialized Translation Pipeline nhận văn bản tiếng Nhật và thực hiện dịch thuật chuyên ngành dưới dạng tiến trình độc lập (stateless). Backend khởi chạy pipeline bằng lệnh hệ thống, truyền văn bản qua luồng đầu vào chuẩn (STDIN) và nhận kết

quả JSON từ luồng đầu ra chuẩn (STDOUT). Các thành phần cấu thành chính bao gồm tokenizer (SudachiPy), Neo4jClient để lấy nghĩa từ ứng viên, Ranker để xếp hạng bằng chứng dựa trên miền chuyên ngành, và LLMClient để dịch thuật.

Cấu trúc đầu ra JSON của pipeline được định nghĩa chặt chẽ để trả về đầy đủ các thông tin:

- `translation` (chuỗi): Bản dịch tiếng Việt hoàn chỉnh của toàn bộ văn bản gốc.
- `detectedDomains` (mảng chuỗi): Danh sách các miền chuyên ngành được phát hiện.
- `keyVocabulary` (mảng đối tượng): Các thuật ngữ đa nghĩa quan trọng kèm cách đọc, nghĩa dịch tương ứng, miền chuyên ngành được xác định và điểm số xếp hạng của nghĩa đó.
- `notes` (mảng đối tượng): Các ghi chú phân tích từ vựng chuyên sâu (sắc thái nghĩa, ngữ cảnh sử dụng).

Nếu tiến trình trả mã lỗi hoặc JSON đầu ra không hợp lệ, backend sẽ phát sinh ngoại lệ nghiệp vụ `DEEP_TRANSLATION_FAILED` để ngăn log kỹ thuật hoặc phản hồi không hoàn chỉnh được trả trực tiếp cho frontend. Mô hình đồ thị Neo4j được trình bày tại Mục 4.2.3; thuật toán xếp hạng và truy xuất được chi tiết hóa tại Mục 5.5.

f, Cơ chế tích hợp mô hình và xử lý lỗi

LLMClient đóng vai trò adapter chung giúp các pipeline độc lập với SDK của từng nhà cung cấp. Tên mô hình và nhà cung cấp được cấu hình linh hoạt qua biến môi trường (ưu tiên Gemini, dự phòng qua Groq hoặc OpenAI). Các pipeline tuân thủ các nguyên tắc thiết kế an toàn: giới hạn chặt chẽ dữ liệu đầu vào, yêu cầu phản hồi có cấu trúc JSON, ánh xạ các lỗi kết nối thành trạng thái có kiểm soát, và luôn hỗ trợ chế độ tương tác văn bản không dùng AI khi các chức năng âm thanh (STT/TTS) bị gián đoạn. Thiết kế phân rã này cho phép cập nhật prompt và mô hình ở module AI mà không ảnh hưởng tới nghiệp vụ cốt lõi tại backend. Các cơ chế tự phục hồi chi tiết được phân tích tại các Mục 5.3, 5.4 và 5.5.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Trong quá trình xây dựng hệ thống, đồ án sử dụng các công cụ, ngôn ngữ lập trình, API, thư viện, IDE, công cụ kiểm thử với các phiên bản cụ thể được lấy từ cấu hình phụ thuộc và môi trường phát triển thực tế của dự án. Danh sách các thư viện và công cụ chính được trình bày chi tiết trong bảng dưới đây:

Mục đích sử dụng	Công cụ/thư viện	Phiên bản sử dụng	Địa chỉ URL
Ngôn ngữ backend	Java	21	https://www.oracle.com/java/
Framework back-end	Spring Boot	4.0.5	https://spring.io/projects/spring-boot
Bảo mật	Spring Security	7.0.4	https://spring.io/projects/spring-security
Ảnh xạ dữ liệu ORM	Hibernate ORM	7.2.7.Final	https://hibernate.org/orm/
Ngôn ngữ frontend	JavaScript	ES2015+	https://developer.mozilla.org/docs/Web/JavaScript
Framework frontend	Vue.js	3.5.33	https://vuejs.org/
Build	Vite	8.0.10	https://vite.dev/
Môi trường chạy	Node.js & npm	24.15, 11.12	https://nodejs.org/
Styling	Tailwind CSS	4.2.4	https://tailwindcss.com/
Ngôn ngữ AI/Pipeline	Python	3.12.3	https://www.python.org/
Framework API cho AI	FastAPI	>= 0.95.0	https://fastapi.tiangolo.com/
Gọi mô hình Gemini	Google Gen AI SDK	2.3.0	https://ai.google.dev/gemini-api/docs
Nhận dạng giọng nói	Faster-Whisper	>= 1.0.0	https://github.com/SYSTRAN/faster-whisper
Phân tích tiếng Nhật	SudachiPy & Dict	0.6.11	https://github.com/WorksApplications/SudachiPy
Cơ sở dữ liệu chính	PostgreSQL	16.14	https://www.postgresql.org/

Mục đích sử dụng	Công cụ/thư viện	Phiên bản sử dụng	Địa chỉ URL
Cơ sở dữ liệu đồ thị	Neo4j Community	2026.04.0	https://neo4j.com/
IDE phát triển	VS Code	1.119.0	https://code.visualstudio.com/
Quản lý mã nguồn	Git	2.43.0	https://git-scm.com/
Kiểm thử backend	JUnit	6.0.3	https://junit.org/
Kiểm thử frontend	Cypress	15.15.0	https://www.cypress.io/

Bảng 4.5: Danh sách thư viện và công cụ sử dụng

Các thư viện phụ thuộc gián tiếp do framework tự quản lý không được liệt kê riêng nhằm tránh bảng quá dài.

4.3.2 Kết quả đạt được

Sau quá trình phân tích, thiết kế và xây dựng, đồ án đã hoàn thành một hệ thống web hỗ trợ học tiếng Nhật ở mức nguyên mẫu hoàn chỉnh. Kết quả không chỉ là một giao diện minh họa mà gồm các thành phần có thể build, khởi chạy và giao tiếp với nhau qua API.

a, Các sản phẩm được xây dựng

Sản phẩm phần mềm của hệ thống được đóng gói thành các thành phần độc lập để dễ dàng triển khai, vận hành và nâng cấp. Chi tiết về các sản phẩm đóng gói được trình bày trong bảng dưới đây:

Sản phẩm	Thành phần đóng gói	Kích thước	Ý nghĩa và vai trò
Frontend production	Thư mục frontend/dist (HTML, CSS, JS)	545,5 KB; 155,8 KB nén tar.gz	Giao diện người học, giao tiếp với backend qua REST API.
Backend Spring Boot	Tệp backend-0.0.1-SNAPSHOT.jar thực thi độc lập	63,1 MB	Cung cấp API, bảo mật JWT, nghiệp vụ SRS và điều phối hệ thống.

Sản phẩm	Thành phần đóng gói	Kích thước	Ý nghĩa và vai trò
Python AI module	Thư mục <code>python_pipeline</code> và <code>requirements.txt</code>	86,6 KB	FastAPI server chạy cổng nội bộ; cung cấp các AI pipeline.
Dữ liệu Neo4j	Các tệp CSV (<code>domain</code> , <code>lexeme</code> , <code>sense</code>)	1,98 MB	Dữ liệu khởi tạo đồ thị tri thức phục vụ dịch chuyên sâu.
Thiết kế PostgreSQL	15 thực thể JPA	Nằm trong backend JAR	Ánh xạ tự động và lưu trữ toàn bộ dữ liệu nghiệp vụ quan hệ.

Bảng 4.6: Các sản phẩm đóng gói của hệ thống

Cách đóng gói này giúp hệ thống thuận tiện cho phát triển, kiểm thử và mở rộng về sau.

b, Kết quả chức năng

Hệ thống đã hoàn thiện đầy đủ các chức năng theo yêu cầu thiết kế bao gồm: đăng ký/dăng nhập và quản lý hồ sơ; tra cứu từ điển và bình luận; quản lý bộ thẻ flashcard và ôn tập lặp lại ngắt quãng SM-2; ôn tập cá nhân hóa thông qua hình thức sinh quiz và story từ lỗi sai gần đây; luyện hội thoại đa phương thức với AI Tutor; dịch chuyên sâu tích hợp đồ thị tri thức Neo4j và mô hình ngôn ngữ lớn.

c, Thống kê mã nguồn

Số liệu được thống kê trực tiếp từ mã nguồn tại thời điểm hoàn thiện luận văn. Dòng mã là số dòng vật lý trong các tệp `.java`, `.vue`, `.js`, `.css` và `.py`, bao gồm mã chính và mã kiểm thử. Phép đo loại trừ `node_modules`, môi trường Python `.venv`, thư mục `build`, dữ liệu `seed`, tệp âm thanh tải lên, `cache` và tài liệu luận văn.

Thông tin thống kê	Giá trị	Mô tả chi tiết
Tổng quy mô mã nguồn	17.678 dòng code (176 tệp)	Tổng dòng vật lý của Java, Vue/JS và Python.
Phần hệ Backend (Java/Spring Boot)	6.530 dòng code (121 tệp)	Gồm 16 gói, 15 JPA entity, 36 DTO, 42 REST API endpoint. Dung lượng nguồn: 266 KB.

Thông tin thống kê	Giá trị	Mô tả chi tiết
Phân hệ Frontend (Vue 3/Vite)	9.135 dòng code (27 tệp .vue)	Gồm 20 route, 11 module JS, 40 dòng test. Dung lượng nguồn: 595 KB.
Phân hệ AI (Python/FastAPI)	2.013 dòng code (15 tệp .py)	Gồm 20 class, 3 tệp kiểm thử, các pipeline. Dung lượng nguồn: 86,6 KB.
Cơ sở dữ liệu quan hệ PostgreSQL	15 bảng	Ánh xạ qua JPA entity và lưu trữ dữ liệu nghiệp vụ người dùng.
Đồ thị tri thức Neo4j	12.958 nút, 24.225 cạnh	Dữ liệu từ các tệp CSV seed (1,98 MB) cho pipeline dịch.
Tổng dung lượng mã nguồn sạch	Khoảng 0,95 MB	Loại bỏ hoàn toàn thư viện <code>node_modules</code> , <code>.venv</code> và build artifacts.

Bảng 4.7: Thống kê thông tin ứng dụng

Kết quả build cho thấy frontend được Vite chuyển đổi thành 43 tệp production, còn backend biên dịch thành 163 tệp `.class` và được Spring Boot đóng gói cùng các thư viện runtime.

4.3.3 Minh họa các chức năng chính

Trong quá trình xây dựng hệ thống, giao diện được thiết kế theo hướng trực quan, thống nhất và hỗ trợ người học thực hiện liên tục các hoạt động tra cứu, ghi nhớ, ôn tập và luyện sử dụng tiếng Nhật. Phần này lựa chọn sáu màn hình đại diện cho các chức năng chính, quan trọng và mang tính đặc trưng nhất của hệ thống, gồm: tra cứu từ điển, học flashcard theo SM-2, ôn tập bằng Review Quiz, ôn tập bằng Review Story, luyện hội thoại với AI Tutor và dịch thuật chuyên sâu bằng GraphRAG.

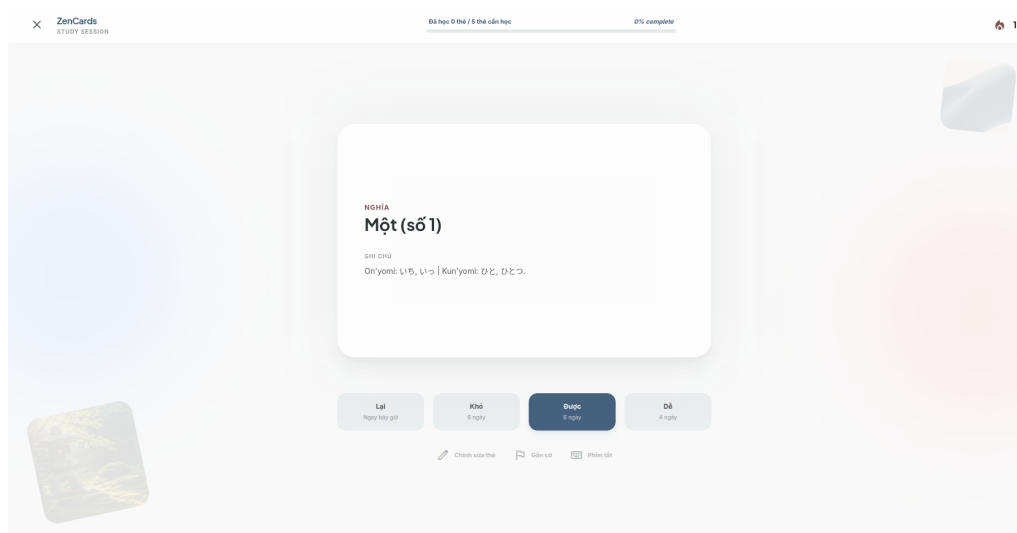
a, Giao diện tra cứu từ điển và lưu từ vào flashcard



Hình 4.20: Giao diện tra cứu từ điển và lưu từ vào flashcard

Giao diện cho phép người học nhập Kanji, Kana hoặc Romaji để tra cứu một mục từ tiếng Nhật. Kết quả hiển thị chữ viết, cách đọc, nghĩa tiếng Việt, cấp độ JLPT, ví dụ sử dụng, các thành phần Kanji và những từ có liên quan. Nút thêm flashcard ở góc thẻ từ mở hộp thoại chọn bộ thẻ; người học cũng có thể tạo bộ thẻ mới ngay trong hộp thoại này. Nhờ đó, nội dung vừa tra cứu có thể được chuyển trực tiếp thành dữ liệu học tập mà không cần nhập lại thủ công.

b, Giao diện học flashcard theo thuật toán SM-2



Hình 4.21: Giao diện học flashcard theo thuật toán SM-2

Màn hình học sử dụng thẻ lật: mặt trước hiển thị từ tiếng Nhật và cách đọc, còn mặt sau hiển thị nghĩa cùng ghi chú. Sau khi xem đáp án, người học tự đánh giá

mức độ ghi nhớ bằng bốn lựa chọn **Lại**, **Khó**, **Được** và **Đế**. Bên dưới mỗi lựa chọn, hệ thống hiển thị trước khoảng thời gian dự kiến đến lần ôn tiếp theo. Kết quả đánh giá được gửi về backend để cập nhật các tham số SM-2 và lập lịch ôn riêng cho từng thẻ; thanh tiến độ phía trên giúp người học theo dõi số thẻ đã hoàn thành trong phiên.

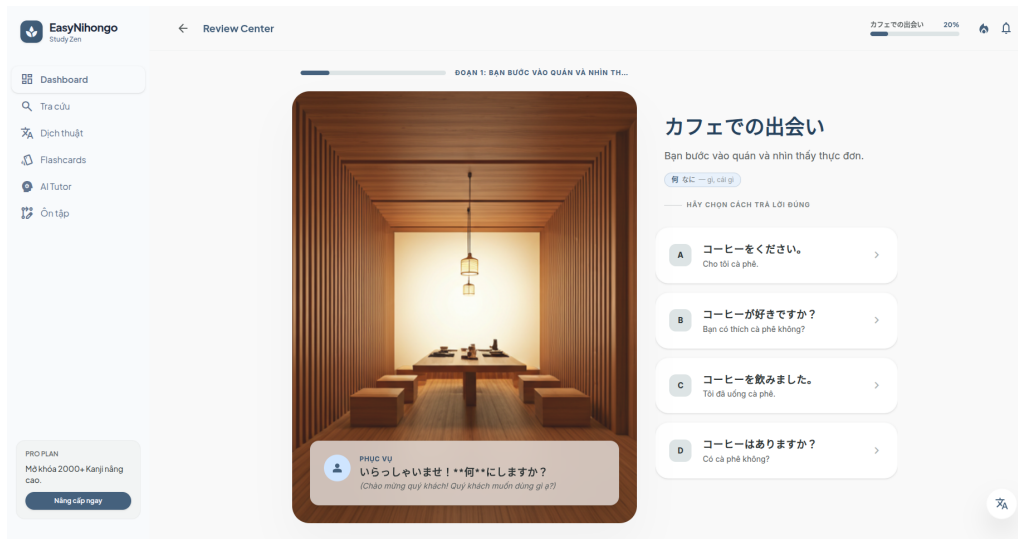
c, Giao diện ôn tập bằng Review Quiz



Hình 4.22: Giao diện ôn tập bằng Review Quiz

Review Quiz trình bày các câu hỏi trắc nghiệm được AI sinh từ bộ thẻ, cấp độ JLPT và lỗi sai gần đây của người học. Mỗi câu gồm nội dung tiếng Nhật, bản dịch hỗ trợ và các phương án trả lời. Ngay sau khi người học chọn đáp án, giao diện đánh dấu phương án đúng hoặc sai, đồng thời hiển thị lời giải thích ngắn bằng tiếng Việt và gợi ý liên quan. Khu vực phía trên cho biết tiến độ làm bài và số câu trả lời đúng, giúp người học nhận phản hồi ngay trong quá trình ôn tập.

d, Giao diện ôn tập bằng Review Story



Hình 4.23: Giao diện ôn tập bằng Review Story

Review Story đưa từ vựng cần ôn vào một câu chuyện tương tác gồm nhiều phân đoạn. Mỗi phân đoạn hiển thị bối cảnh, lời thoại tiếng Nhật, bản dịch tiếng Việt và các từ vựng nổi bật. Tại những điểm cần tương tác, người học lựa chọn câu trả lời hoặc hành động để tiếp tục câu chuyện; hệ thống chấm lựa chọn và giải thích đáp án trước khi chuyển sang phân đoạn tiếp theo. Cách trình bày này giúp người học gặp lại từ vựng trong ngữ cảnh thay vì chỉ ghi nhớ từng thẻ rời rạc.

e, Giao diện luyện hội thoại với AI Tutor

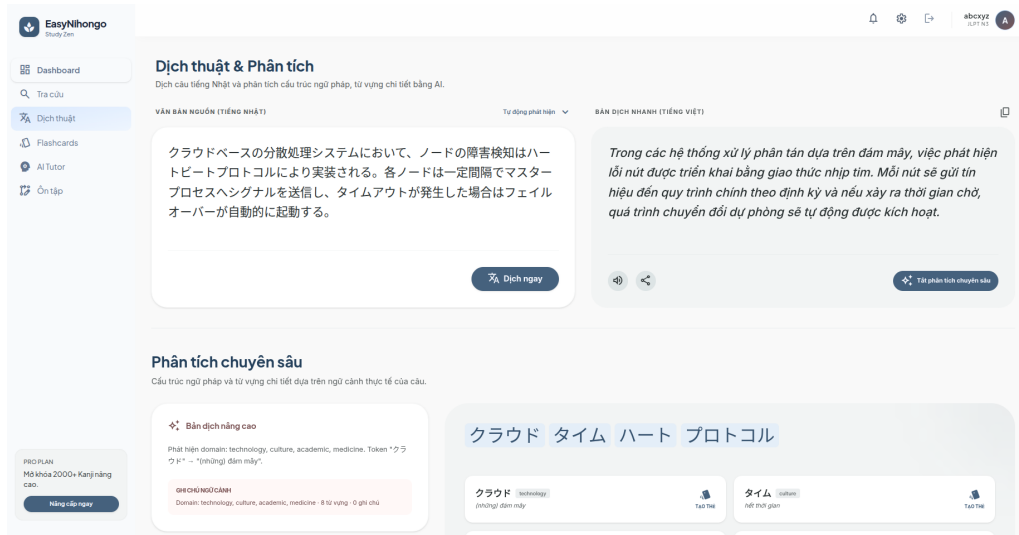


Hình 4.24: Giao diện luyện hội thoại với AI Tutor

Giao diện AI Tutor được tổ chức theo dạng phòng chat, cho phép người học gửi câu tiếng Nhật bằng văn bản hoặc ghi âm trực tiếp từ trình duyệt. Phản hồi của AI

gồm nội dung hội thoại, bản dịch, gợi ý trả lời và thẻ sửa lỗi được đặt gần câu sai để người học dễ đối chiếu. Bảng thông tin bên phải hiển thị tiến độ phiên học, số lượt nói và các từ vựng mới xuất hiện trong hội thoại. Người học có thể bật chế độ tập trung để ẩn bảng phụ hoặc kết thúc phiên để chuyển sang màn hình tổng hợp kết quả.

f, Giao diện dịch thuật và phân tích chuyên sâu



Hình 4.25: Giao diện dịch thuật và phân tích chuyên sâu

Màn hình dịch thuật gồm vùng nhập câu tiếng Nhật và vùng hiển thị bản dịch tiếng Việt. Ngoài chế độ dịch nhanh, người học có thể bật **Phân tích chuyên sâu** để hệ thống tách các từ quan trọng, xác định miền ngữ cảnh và truy xuất bằng chứng từ đồ thị tri thức Neo4j. Kết quả phân tích trình bày cách đọc, nghĩa theo ngữ cảnh, cấp độ JLPT, nhãn miền và các ghi chú dịch thuật cho từng từ nổi bật. Đây là giao diện thể hiện trực tiếp việc kết hợp xử lý ngôn ngữ tự nhiên, GraphRAG và mô hình ngôn ngữ lớn trong đồ án.

Các giao diện trên cho phép người học chuyển tiếp giữa tra cứu, flashcard, Review và AI Tutor. Mục này chỉ minh họa kết quả giao diện; cơ chế liên kết dữ liệu giữa các hoạt động được trình bày tại Mục 5.1.

4.4 Kiểm thử

Kiểm thử được thực hiện song song với quá trình phát triển nhằm phát hiện lỗi nghiệp vụ và lỗi tích hợp giữa các dịch vụ. Hai chức năng cốt lõi được lựa chọn để trình bày chi tiết là: ôn tập flashcard theo thuật toán SM-2 và luyện hội thoại đa phương thức với AI Tutor. Đây là các chức năng có logic chuyển trạng thái phức tạp, tích hợp nhiều dịch vụ (frontend, backend, database, AI) và có tính cá nhân hóa cao. Các kịch bản kiểm thử cho các nhóm chức năng khác (như xác thực, từ

điển, hồ sơ, dịch chuyên sâu và AI Review) được chuyển vào phần Phụ lục A để tối ưu hóa không gian trình bày của đề án.

4.4.1 Phương pháp và kỹ thuật kiểm thử

Kiểm thử hệ thống được thực hiện kết hợp giữa **kiểm thử hộp đen** (Black-box testing) dưới góc nhìn trải nghiệm của người học và **kiểm thử tích hợp** (Integration testing) để xác minh tính ổn định khi liên kết giữa các phân hệ Spring Boot, PostgreSQL, Web Speech API của trình duyệt và Python AI service.

Các kỹ thuật kiểm thử cụ thể được áp dụng bao gồm:

- **Phân vùng tương đương & Phân tích giá trị biên:** Áp dụng để xác minh tính đúng đắn khi xử lý tham số đầu vào của thuật toán ôn tập SM-2. Dải giá trị đánh giá chất lượng ôn tập (tham số `quality` từ 0 – 5) được chia thành phân vùng hợp lệ ($quality \in [0, 5]$) và phân vùng không hợp lệ ($quality \geq 6$ hoặc $quality < 0$).
- **Kiểm thử chuyển trạng thái:** Áp dụng để kiểm tra quy trình thay đổi trạng thái của flashcard qua các giai đoạn học tập (NEW, LEARNING, REVIEW) dựa trên lựa chọn phản hồi của người học, cũng như việc cập nhật trạng thái luồng hội thoại của AI Tutor qua các bước gửi và nhận tin nhắn.
- **Kiểm thử ngoại lệ và Khả năng phục hồi (Robustness & Exception Testing):** Áp dụng cho các tình huống lỗi môi trường hoặc sự cố ngoài ý muốn của người dùng, cụ thể là trường hợp trình duyệt cũ không hỗ trợ hoặc bị từ chối quyền truy cập Web Speech API (thiết lập cơ chế fallback nhập liệu bằng bàn phím) và trường hợp mất kết nối mạng đột ngột trong khi đang thực hiện nhận dạng giọng nói (hiển thị thông báo lỗi kết nối và cho phép thực hiện lại thao tác).

Để thực hiện các kịch bản kiểm thử này, đề án kết hợp cả phương pháp tự động hóa và thủ công:

- **Kiểm thử tự động ở Backend:** Sử dụng thư viện JUnit kết hợp MockMvc để kiểm tra các API endpoint của hệ thống, giả lập (mock) các lời gọi dịch vụ AI bên ngoài nhằm kiểm tra tính độc lập của logic nghiệp vụ SRS và xử lý ngoại lệ đầu vào.
- **Kiểm thử giao diện E2E:** Sử dụng Cypress để giả lập các thao tác của người dùng trên giao diện web đối với các luồng nghiệp vụ cơ bản.
- **Kiểm thử thủ công đa phương thức:** Được áp dụng để trực tiếp thử nghiệm tính năng nhận dạng và tổng hợp giọng nói của Web Speech API, kiểm tra hành vi phản hồi của giao diện khi tương tác với microphone và mô phỏng các

sự cố mạng vật lý.

4.4.2 Kiểm thử chức năng ôn tập flashcard theo SM-2

Nhóm này kiểm tra quan hệ giữa đánh giá của người học, khoảng cách ôn và trạng thái thẻ. Thẻ mới khởi tạo có hệ số dễ nhớ 2,5, số lần lặp lại bằng 0 và khoảng cách bằng 0 ngày.

Mã	Trường hợp kiểm thử	Dữ liệu đầu vào	Kết quả mong đợi	Đánh giá
SM2-01	Không nhớ thẻ	Thẻ mới; chọn Lại (quality = 1)	Số lần lặp lại về 0; khoảng cách 0 ngày; thẻ ở LEARNING và được đưa lại cuối hàng đợi	Đạt
SM2-02	Nhớ nhưng còn khó	Thẻ mới; chọn Khó (quality = 3)	Số lần lặp lại là 1; ôn sau 1 ngày; trạng thái LEARNING	Đạt
SM2-03	Nhớ ở mức bình thường	Thẻ mới; chọn Được (quality = 4)	Số lần lặp lại là 1; ôn sau 2 ngày; trạng thái LEARNING	Đạt
SM2-04	Nhớ dễ dàng	Thẻ mới; chọn Dễ (quality = 5)	Khoảng cách 4 ngày; thẻ chuyển ngay sang REVIEW	Đạt
SM2-05	Chất lượng ngoài miền	quality = 6	Phát sinh IllegalArgumentException; dữ liệu SRS không được cập nhật	Đạt

Bảng 4.8: Các trường hợp kiểm thử chức năng ôn tập flashcard theo SM-2

Các lựa chọn tạo ra khoảng cách ôn đúng thiết kế. **Lại** đưa thẻ trở lại hàng đợi, còn **Dễ** chuyển thẻ sớm sang giai đoạn ôn tập. Giá trị ngoài miền bị từ chối nên không làm sai dữ liệu SRS.

4.4.3 Kiểm thử chức năng AI Tutor đa phương thức

Nhóm kiểm thử AI Tutor tập trung xác minh luồng hội thoại kết hợp nhận dạng giọng nói (STT) và tổng hợp giọng nói (TTS) được thực hiện trực tiếp tại client thông qua các API của trình duyệt (Browser Web Speech API).

Mã	Trường hợp kiểm thử	Dữ liệu đầu vào	Kết quả mong đợi	Đánh giá
TUT-01	Gửi tin nhắn văn bản	Phiên hợp lệ; gửi chuỗi văn bản raamen wo hitotsu kudasai	Hệ thống trả phản hồi dạng JSON có cấu trúc (gồm phản hồi tiếng Nhật, dịch tiếng Việt, sửa lỗi ngữ pháp, gợi ý hội thoại và từ mới)	Đạt
TUT-02	Nhận dạng giọng nói (Browser STT)	Người học nói câu raamen wo hitotsu kudasai vào microphone	Web Speech API (STT) của trình duyệt nhận dạng chính xác giọng nói và chuyển đổi thành văn bản tiếng Nhật thành công	Đạt
TUT-03	Phát âm thanh phản hồi (Browser TTS)	Phản hồi tiếng Nhật dạng văn bản được nhận từ AI	Web Speech API (TTS) của trình duyệt tổng hợp âm thanh và phát giọng đọc tiếng Nhật chuẩn xác, tự nhiên	Đạt
TUT-04	Trình duyệt không hỗ trợ Web Speech API	Khởi chạy ứng dụng trên trình duyệt cũ hoặc không cấp quyền truy cập microphone	Giao diện hiển thị cảnh báo không sử dụng được microphone, ấn nút ghi âm và cho phép người học nhập liệu bình thường bằng bàn phím	Đạt
TUT-05	Xử lý lỗi kết nối mạng khi đang STT	Thiết bị mất kết nối Internet trong lúc Web Speech API đang nhận dạng giọng nói	Giao diện hiển thị thông báo lỗi kết nối mạng trên thanh trạng thái và cho phép người học bấm nút thử lại	Đạt

Bảng 4.9: Các trường hợp kiểm thử chức năng AI Tutor đa phương thức

4.4.4 Tổng kết kết quả kiểm thử

Hệ thống đã trải qua tổng cộng 10 ca kiểm thử tiêu biểu cho các phân hệ, bao gồm các ca kiểm thử chính được phân tích chi tiết ở trên. Kết quả cho thấy cả 10 trường hợp đều hoạt động đúng thiết kế, đạt tỷ lệ thành công 100%.

Nhóm chức năng	Số ca test	Đạt	Không đạt	Tỷ lệ đạt
Flashcard và thuật toán SM-2	5	5	0	100%
AI Tutor đa phương thức	5	5	0	100%
Tổng cộng	10	10	0	100%

Bảng 4.10: Tổng hợp kết quả kiểm thử các chức năng chính

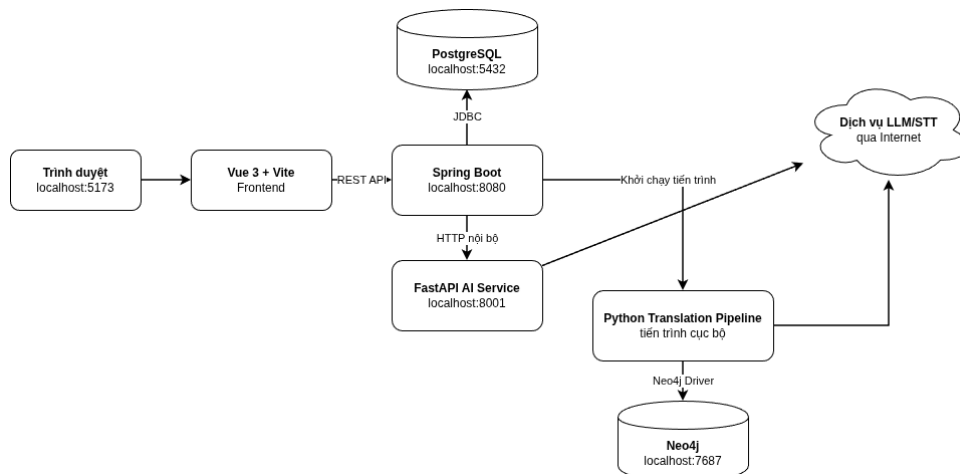
Nhìn chung, các chức năng cốt lõi đáp ứng yêu cầu ở mức nguyên mẫu hoàn chỉnh. Sự kết hợp giữa kiểm thử tự động (JUnit, pytest, Cypress) và kiểm thử thủ công đã giúp phát hiện sớm các vấn đề giao tiếp liên dịch vụ và cấu hình bảo mật.

4.5 Triển khai

Trong phạm vi đồ án, hệ thống được triển khai thử nghiệm trên máy tính cá nhân theo mô hình localhost. Mục tiêu của lần triển khai này là đánh giá khả năng tương thích và vận hành đồng thời của các phân hệ, kết nối giữa các dịch vụ, và tính đúng đắn của các luồng nghiệp vụ cốt lõi.

4.5.1 Mô hình triển khai thử nghiệm

Mô hình triển khai thử nghiệm bao gồm năm thành phần chính chạy trên cùng một thiết bị: frontend Vue, backend Spring Boot, Python AI service, PostgreSQL và Neo4j.



Hình 4.26: Mô hình triển khai thử nghiệm trên localhost

Trình duyệt của người dùng truy cập vào frontend tại địa chỉ `http://localhost:5173`. Cấu hình reverse proxy của Vite sẽ chuyển tiếp các yêu cầu API có tiền tố `/api` và `/auth` đến backend Spring Boot chạy tại cổng 8080. Backend đảm nhận việc xác thực, kiểm tra quyền và truy xuất cơ sở dữ liệu PostgreSQL (cổng 5432). Đối với các tính năng thông minh như AI Tutor và AI Review, backend giao tiếp với Python AI service qua giao thức HTTP REST tại cổng 8001. Khi

thực hiện dịch chuyên sâu, Python AI service sẽ truy vấn cơ sở dữ liệu đồ thị Neo4j qua cổng 7687 (giao thức Bolt).

Quy trình khởi chạy hệ thống được thực hiện bằng cách khởi động các cơ sở dữ liệu PostgreSQL và Neo4j, tiếp theo chạy AI service (FastAPI) bằng lệnh `python server.py`, backend Spring Boot bằng lệnh `./mvnw spring-boot:run`, và cuối cùng là frontend Vue bằng lệnh `npm run dev` trên các cửa sổ terminal độc lập.

4.5.2 Thiết bị và môi trường triển khai

Hệ thống được triển khai thử nghiệm trực tiếp từ mã nguồn trên một laptop Lenovo 82K2 chạy hệ điều hành Ubuntu 24.04 LTS (kiến trúc x86-64). Cấu hình chi tiết của thiết bị bao gồm bộ vi xử lý AMD Ryzen 5 5600H (6 nhân, 12 luồng), bộ nhớ RAM 16 GB, và ổ cứng SSD 512 GB. Các công cụ runtime được cài đặt gồm OpenJDK 25.0.3 (cho Java 21), Python 3.12.3, Node.js 24.15.0 và Google Chrome làm trình duyệt kiểm thử. Việc chạy trực tiếp trên môi trường máy chủ cục bộ giúp lập trình viên dễ dàng theo dõi log hệ thống, gỡ lỗi nhanh và tối ưu hóa hiệu năng của các pipeline AI trước khi triển khai thực tế.

4.5.3 Kết quả triển khai thử nghiệm

Kết quả thử nghiệm thực tế cho thấy các dịch vụ khởi động bình thường và kết nối thông suốt với nhau. Người phát triển có thể thao tác hoàn chỉnh tất cả các luồng nghiệp vụ của ứng dụng: từ đăng ký tài khoản, tra cứu từ điển, quản lý flashcard, ôn tập SM-2 đến hội thoại tiếng Nhật và dịch GraphRAG.

Vì đồ án hiện đang dừng lại ở mức xây dựng nguyên mẫu thử nghiệm (prototype), hệ thống mới chỉ được chạy bởi một lập trình viên duy nhất trên máy cục bộ phục vụ công tác kiểm thử tích hợp. Do đó, đồ án chưa triển khai cho nhóm người dùng bên ngoài, chưa có số liệu thống kê về lượt truy cập thực tế, phản hồi người dùng hay thực hiện kiểm thử tải (load testing) để xác định khả năng chịu tải tối đa của hệ thống. Đồng thời, thời gian phản hồi của các tính năng AI phụ thuộc nhiều vào chất lượng kết nối Internet đến nhà cung cấp mô hình ngôn ngữ lớn (Gemini API) nên chưa có phép đo thống kê chính thức.

Để đưa ứng dụng vào vận hành thực tế (production), đồ án định hướng đóng gói các thành phần bằng Docker container, cấu hình reverse proxy bảo mật bằng Nginx, tách biệt các thông tin cấu hình nhạy cảm, sử dụng giao thức HTTPS, và thực hiện kiểm thử tải bằng các công cụ chuyên dụng (như JMeter hay Locust) trước khi triển khai lên các dịch vụ đám mây.

4.6 Tổng kết

Chương 4 đã trình bày quá trình thiết kế chi tiết, kiểm thử và triển khai thử nghiệm hệ thống học tiếng Nhật thông minh. Dựa trên các yêu cầu phân tích từ Chương 3, hệ thống đã được xây dựng theo kiến trúc nhiều tầng sử dụng các công nghệ hiện đại bao gồm Spring Boot, Vue 3, FastAPI, PostgreSQL và Neo4j. Các biểu đồ lớp và biểu đồ trình tự được thiết kế chi tiết cho các ca sử dụng quan trọng đã cung cấp tài liệu kỹ thuật rõ ràng cho việc hiện thực hóa mã nguồn.

Đồ án đã hiện thực hóa thành công các chức năng cốt lõi hỗ trợ một quy trình học tiếng Nhật toàn diện. Kết quả kiểm thử tiêu biểu đạt tỷ lệ thành công 100% (10/10 ca test chính đạt yêu cầu), chứng minh tính đúng đắn và độ tin cậy của hệ thống. Quá trình triển khai thử nghiệm thành công trên môi trường localhost đã khẳng định tính khả thi trong việc kết hợp xử lý ngôn ngữ tự nhiên, đồ thị tri thức và mô hình ngôn ngữ lớn để hỗ trợ cá nhân hóa học tập. Những kết quả thực nghiệm và triển khai này là cơ sở thực tế quan trọng để Chương 5 trình bày và đánh giá chi tiết các đóng góp giải pháp cốt lõi của đồ án.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày kiến trúc, thiết kế cơ sở dữ liệu, các module phần mềm và kết quả triển khai của hệ thống. Vì vậy, chương này không lặp lại cấu trúc lớp hoặc mô tả giao diện, mà tập trung phân tích những bài toán khó, các quyết định kỹ thuật và phần đóng góp mà sinh viên trực tiếp đề xuất, hiện thực và kiểm chứng trong quá trình thực hiện đồ án.

Năm đóng góp chính được lựa chọn gồm: thiết kế vòng học tập khép kín dựa trên dữ liệu cá nhân; điều chỉnh thuật toán SM-2 cho luồng học bốn mức đánh giá; xây dựng AI Review Pipeline có khả năng sinh nội dung và phục hồi khi đầu ra mô hình không hoàn chỉnh; xây dựng AI Tutor đa phương thức gắn hội thoại với dữ liệu ôn tập; và xây dựng pipeline GraphRAG để lựa chọn nghĩa của từ đa nghĩa trong văn bản chuyên ngành. Các nội dung này không hoàn toàn độc lập. Chúng được liên kết bởi một nguyên tắc chung: dữ liệu phát sinh ở một hoạt động học phải có khả năng trở thành đầu vào hữu ích cho hoạt động tiếp theo.

5.1 Xây dựng vòng học tập tích hợp cho người học tiếng Nhật

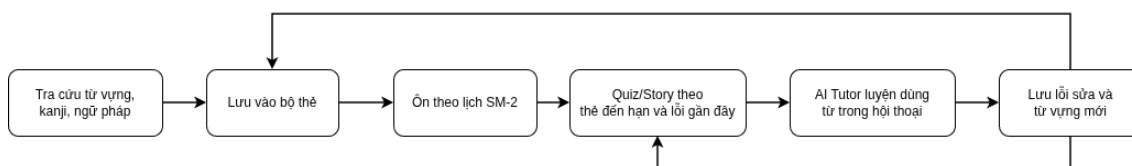
5.1.1 Giới thiệu bài toán

Khi tự học tiếng Nhật, người học thường phải sử dụng nhiều ứng dụng phân mảnh cho các tác vụ tra từ, học flashcard, làm bài tập và luyện nói. Việc thiếu liên kết dữ liệu giữa các công cụ này khiến thông tin học tập bị phân mảnh: từ vựng vừa tra cứu không tự động chuyển thành thẻ ôn tập; lỗi ngữ pháp khi hội thoại không được đưa vào bài tập củng cố; và kết quả của mỗi phiên học không tác động đến kế hoạch học tiếp theo. Đối với tiếng Nhật, một từ vựng có nhiều thuộc tính phức tạp (kanji, cách đọc, nghĩa, ví dụ, cấp độ JLPT) và yêu cầu rèn luyện cả khả năng ghi nhớ lẫn vận dụng ngữ cảnh. Do đó, bài toán đặt ra là xây dựng một vòng học tập tích hợp giúp tự động chuyển tiếp dữ liệu kiến thức, lịch ôn, lỗi sai và kết quả giữa các phân hệ một cách có chủ đích, đồng thời đảm bảo tính độc lập và phân định rõ ràng trách nhiệm của từng thành phần.

5.1.2 Giải pháp

Đồ án đề xuất giải pháp tổ chức hệ thống theo một vòng dữ liệu khép kín gồm năm giai đoạn (Hình 5.1): **tiếp nhận tri thức** (chuẩn hóa kết quả tra từ điển sang flashcard nhưng giữ quyền chủ động lưu thẻ cho người dùng); **lập lịch ghi nhớ** (tách biệt nội dung thẻ và trạng thái SRS để thay đổi thuật toán lập lịch mà không mất lịch sử ôn); **kiểm tra trong ngữ cảnh** (Review Pipeline nhận dữ liệu thẻ đến hạn/mới, cấp độ JLPT và lỗi gần đây từ backend); **vận dụng qua hội thoại** (AI

Tutor tương tác dựa trên từ mục tiêu và phản hồi cấu trúc chi tiết); và **phản hồi lại kế hoạch ôn tập** (các lỗi sai từ AI Tutor được lưu và gửi lại cho Review Service để sinh bài tập củng cố).



Hình 5.1: Vòng học tập khép kín dựa trên dữ liệu cá nhân

Để tránh liên kết chặt, hệ thống sử dụng backend làm tầng điều phối dữ liệu trung tâm. PostgreSQL lưu trữ thông tin tài khoản, flashcard, SRS và lịch sử; Python AI service xử lý nghiệp vụ AI phi trạng thái; còn Neo4j chỉ lưu tri thức từ vựng phục vụ dịch chuyên sâu. Sự phân vai trách nhiệm được tổng hợp trong Bảng 5.1.

Thành phần	Dữ liệu chịu trách nhiệm	Nguyên tắc
Frontend	Trạng thái tương tác tạm thời	Không tự quyết định lịch ôn hoặc quyền truy cập.
Backend và PostgreSQL	Người dùng, flashcard, SRS, phiên học, kết quả	Là nguồn dữ liệu chính và kiểm tra quyền sở hữu.
Python AI và Neo4j	Sinh nội dung, phân tích ngôn ngữ, tri thức nghĩa từ	Không trực tiếp thay đổi dữ liệu học cá nhân.

Bảng 5.1: Phân chia trách nhiệm trong vòng học tập

5.1.3 Kết quả đạt được

Hệ thống đã hiện thực hóa các đường chuyển dữ liệu cốt lõi và tích hợp thành công trên môi trường thử nghiệm. Đóng góp nổi bật là chuyển hóa dữ liệu học từ dạng tĩnh sang dữ liệu có vòng đời (một mục từ có thể lần lượt làm thẻ học, câu hỏi ôn tập, từ mục tiêu hội thoại, và lỗi sai quay lại làm tài liệu ôn tập tiếp theo). Kiến trúc này giúp dễ dàng nâng cấp hoặc thay thế từng thành phần (như đổi mô hình LLM, đổi thuật toán SRS) nhờ các hợp đồng dữ liệu rõ ràng ở backend.

Hạn chế hiện tại là chưa tự động điều chỉnh thông số SRS dựa trên điểm số Quiz/Story, và từ mới phát sinh trong Tutor cần xác nhận của người học trước khi lưu. Đây là những lựa chọn thiết kế an toàn để bảo đảm quyền kiểm soát tối đa cho người dùng.

5.2 Điều chỉnh thuật toán SM-2 cho luồng học bốn mức đánh giá

5.2.1 Giới thiệu bài toán

Để tối ưu hóa thời gian ghi nhớ từ vựng, hệ thống cần cá nhân hóa khoảng cách thời gian ôn tập cho từng thẻ thay vì hiển thị chúng đồng đều. Thuật toán SM-2 [16] được chọn nhờ ưu điểm không cần dữ liệu huấn luyện và có trạng thái nhỏ gọn. Tuy nhiên, việc áp dụng nguyên bản SM-2 vào ứng dụng thực tế gặp hai bất cập: (1) SM-2 sử dụng thang điểm từ 0 đến 5, không tương thích trực tiếp với trải nghiệm người dùng hiện đại vốn quen thuộc với 4 nút đánh giá nhanh là **Lại, Khó, Được, Dễ**; (2) Khoảng cách khởi đầu của SM-2 cổ điển chưa phân biệt rõ ý nghĩa của các lựa chọn này. Ngoài ra, hệ thống cần giải quyết đồng thời việc lập lịch dài hạn lẫn quản lý hàng đợi hiển thị tạm thời trong phiên (ví dụ: thẻ chọn "Lại" phải xuất hiện lại ngay trong phiên hiện tại).

5.2.2 Giải pháp

Đề án đề xuất ánh xạ 4 lựa chọn giao diện vào thang chất lượng SM-2 (Bảng 5.2), đồng thời tích hợp xử lý hàng đợi phiên học.

Lựa chọn	Chất lượng q	Ý nghĩa xử lý
Lại	1	Không nhớ; đặt số lần lặp về 0 và đưa lại trong phiên.
Khó	3	Nhớ có khó khăn; tăng khoảng cách chậm.
Được	4	Nhớ bình thường; tăng khoảng cách theo hệ số dễ nhớ.
Dễ	5	Nhớ chắc; tăng nhanh và chuyển sớm sang ôn dài hạn.

Bảng 5.2: Ánh xạ phản hồi người học sang chất lượng SM-2

Trạng thái SRS của mỗi thẻ bao gồm: hệ số dễ nhớ EF (mặc định 2.5), số lần ôn thành công liên tiếp n , và khoảng cách hiện tại I . Hệ số EF được cập nhật theo công thức kinh điển [16]:

$$EF' = \max(1,3; EF + 0,1 - (5 - q) [0,08 + (5 - q) \times 0,02])$$

Nếu chọn **Lại** ($q = 1$), hệ thống đặt $n' = 0$, $I' = 0$, đưa thẻ về hàng đợi tạm trong phiên. Nếu chọn các nút đạt ($q \geq 3$), tại lần đầu tiên ($n = 0$), khoảng cách khởi tạo tương ứng là 1, 2 và 4 ngày cho **Khó, Được, Dễ**. Tại lần thứ hai ($n = 1$), khoảng

cách là 6 ngày với **Khó/Được** và 4 ngày với **Đễ**. Từ lần thứ ba ($n \geq 2$):

$$I' = \text{round}(I \times m_q)$$

Với hệ số điều chỉnh $m_3 = 1,2$, $m_4 = EF'$, và $m_5 = 1,3 \times EF'$.

Trạng thái nghiệp vụ được tách biệt khỏi khoảng cách: **Lại** đưa thẻ về LEARN-ING; **Khó** và **Được** chỉ chuyển sang REVIEW sau lần lượt 3 và 2 lần thành công liên tiếp; còn **Đễ** chuyển thẳng sang REVIEW. Logic tính toán được đóng gói dạng hàm thuần `SM2Algorithm.calculate` độc lập, đảm bảo an toàn giao dịch ở backend và dễ dàng kiểm thử.

5.2.3 Kết quả đạt được

Kết quả là mỗi thẻ được tự động lập lịch ôn tập riêng biệt dựa trên phản hồi của người học. Việc phân biệt rõ các nhóm trạng thái giúp tối ưu luồng ôn tập và hiển thị trực quan khoảng cách dự kiến ngay trên các nút bấm. Kiểm thử tích hợp tại Mục 4.4.2 xác nhận thuật toán tính toán chính xác trong mọi trường hợp biên. Đóng góp cốt lõi của giải pháp là chuyển đổi hiệu quả thuật toán SM-2 thành luồng học 4 mức trực quan, kết hợp quản lý hàng đợi phiên học và tách biệt logic tính toán khỏi phần lưu trữ. Hạn chế hiện tại là chưa tích hợp thời gian phản hồi hoặc điểm số các bài tập bổ trợ để tự động tinh chỉnh các tham số thuật toán.

5.3 Sinh nội dung ôn tập thông minh bằng AI Review Pipeline

5.3.1 Giới thiệu bài toán

Học từ vựng qua flashcard giúp ghi nhớ nghĩa nhanh nhưng chưa đủ để đánh giá khả năng sử dụng từ trong ngữ cảnh thực tế. Việc biên soạn thủ công các bài tập trắc nghiệm (Quiz) hay câu chuyện (Story) cho mọi tổ hợp từ vựng, cấp độ JLPT và lỗi sai cá nhân là bất khả thi về mặt chi phí. Mô hình ngôn ngữ lớn (LLM) có thể tự động sinh nội dung ôn tập cá nhân hóa, nhưng thường gặp rủi ro về độ ổn định: trả về sai cấu trúc JSON, thiếu số lượng câu hỏi, hoặc bị cắt cụt phản hồi giữa chừng. Do giao diện ứng dụng yêu cầu dữ liệu đầu ra có cấu trúc nghiêm ngặt (câu hỏi, đáp án, giải thích) để chấm điểm, bài toán đặt ra là thiết kế một pipeline sinh nội dung ôn tập thông minh từ dữ liệu bộ thẻ của người dùng, có khả năng tự động xử lý và phục hồi dữ liệu khi LLM trả về kết quả không hoàn hảo.

5.3.2 Giải pháp

Giải pháp gồm hai tầng kiểm soát chặt chẽ:

- **Tầng backend:** Điều phối dữ liệu đầu vào bằng cách chọn thẻ ưu tiên (thẻ đến hạn, thẻ mới), giới hạn số lượng từ và tích hợp tối đa 10 lỗi sai gần đây.

- **Tầng Python AI:** Nhận yêu cầu đã chuẩn hóa (như trong Listing 5.1) để thiết lập prompt cho mô hình với nhiệt độ thấp (0.3). Đối với Quiz, mô hình sinh câu hỏi điền vào chỗ trống tích hợp sửa lỗi. Đối với Story, mô hình sinh chuỗi tình huống liên kết từ vựng.

```
{
  "words": [
    {"surface": "yoyaku", "reading": "yoyaku", "meaning": "
    dat truoc"}
  ],
  "level": "N3",
  "question_count": 10,
  "recent_mistakes": [
    {"original": "eki wo ikimasu", "corrected": "eki ni
    ikimasu"}
  ]
}
```

Listing 5.1: Ví dụ đầu vào của AI Review Pipeline

Đóng góp kỹ thuật cốt lõi là chiến lược phục hồi dữ liệu theo nguyên tắc **suy giảm có kiểm soát**: (1) Loại bỏ bao Markdown json; (2) Phân tích JSON toàn bộ; (3) Nếu lỗi, dùng Regex trích xuất mảng `questions` hoặc `segments`; (4) Lọc và chỉ giữ các câu hỏi/phân đoạn có đủ trường tối thiểu; (5) Thử lại tối đa 2 lần nếu thiếu số lượng; (6) Trả về phần dữ liệu hợp lệ kèm cảnh báo thay vì hủy bỏ toàn bộ phiên ôn tập. Kết quả sau khi người học hoàn thành được lưu độc lập tại backend để theo dõi lịch sử học tập lâu dài.

5.3.3 Kết quả đạt được

Review Pipeline đã hỗ trợ sinh thành công hai dạng nội dung ôn tập là Quiz và Story trực quan. Các ca kiểm thử tại Mục 4.4.3 chứng minh cơ chế suy giảm có kiểm soát hoạt động ổn định: khi mô hình chỉ trả về một phần câu hỏi hợp lệ (ví dụ 6/10 câu), hệ thống vẫn hiển thị phần nội dung tốt kèm cảnh báo thay vì báo lỗi toàn trang. Đóng góp chính của giải pháp là quy trình phối hợp và phục hồi đầu ra có cấu trúc để cá nhân hóa nội dung ôn tập theo thẻ học và lỗi sai thực tế. Hạn chế hiện tại là tính chính xác ngôn ngữ vẫn phụ thuộc hoàn toàn vào LLM, chưa có bộ lọc kiểm chứng chất lượng tự động, và cơ chế gọi lại (retry) có thể làm tăng chi phí và độ trễ.

5.4 Xây dựng AI Tutor đa phương thức

5.4.1 Giới thiệu bài toán

Việc chuyển tiếp từ nhận biết từ vựng sang vận dụng trôi chảy trong hội thoại là thử thách lớn đối với người học ngoại ngữ do thiếu môi trường thực hành phản hồi thời gian thực. Các chatbot AI đa năng có thể trò chuyện trôi chảy nhưng thiếu định hướng sư phạm: không kiểm soát được cấp độ từ vựng, dễ bỏ qua lỗi sai của người học hoặc không tích hợp được các từ cần ôn tập. Đồng thời, việc truyền tải dữ liệu âm thanh dung lượng lớn phục vụ STT/TTS trực tiếp lên máy chủ thường tạo ra độ trễ cao và dễ gián đoạn do lỗi kết nối hoặc thiếu thiết bị đầu vào (microphone). Vì vậy, bài toán đặt ra là xây dựng một AI Tutor đa phương thức vừa ràng buộc chặt chẽ cuộc hội thoại theo mục tiêu học tập (cấp độ JLPT, từ vựng đích), vừa xử lý tương tác giọng nói tối ưu và thích ứng linh hoạt với các lỗi phần cứng hoặc mạng tại client.

5.4.2 Giải pháp

AI Tutor hoạt động theo từng phiên độc lập liên kết chặt chẽ với backend. Khi tạo phiên, người học cung cấp cấp độ JLPT, chủ đề hoặc chế độ tự chọn chủ đề, thời lượng và danh sách từ cần luyện. Backend tạo `ConversationSession`, kiểm tra quyền sở hữu ở mọi thao tác và gọi AI để sinh lời mở đầu.

Nếu không có chủ đề cụ thể, pipeline tự chọn một trong sáu nhóm: nhà hàng, mua sắm, du lịch, phỏng vấn, trường học và sinh hoạt hằng ngày. Điểm chủ đề được tính từ hai nguồn. Từ tiếng Nhật khớp với `surface` hoặc `reading` được nhân trọng số 2; từ khóa tiếng Việt khớp với `meaning` được tính trọng số 1. Cách làm này ưu tiên trực tiếp dữ liệu tiếng Nhật nhưng vẫn tận dụng nghĩa tiếng Việt khi từ không trùng hoàn toàn. Nếu không có tín hiệu, pipeline chọn một chủ đề mặc định ngẫu nhiên để vẫn khởi tạo được phiên.

Trong mỗi lượt, prompt chỉ chứa tối đa năm log gần nhất. Giới hạn này là sự đánh đổi giữa tính liên tục và kích thước ngữ cảnh. Với một phiên luyện ngắn, năm lượt đủ để duy trì câu hỏi hiện tại; phần lịch sử cũ vẫn được lưu ở backend để tổng hợp cuối phiên nhưng không phải gửi lặp lại cho mô hình.

Mỗi prompt yêu cầu mô hình thực hiện song song hai tác vụ: (1) kiểm tra và phát hiện lỗi ngữ pháp (trợ từ, chia động từ, mức độ lịch sự), và (2) phản hồi hội thoại kèm 2–3 mẫu câu gợi ý và từ vựng mới theo đúng schema JSON (Listing 5.2).

```
{
  "content_ja": "Phan hoi va cau hoi tiep theo",
  "content_vn": "Ban dich hoac giai thich ngan",
  "corrections": [
```

```
{
  "original": "...", "corrected": "...", "note": "..."}
],
"suggestions": ["...", "..."],
"newVocabulary": [
  {"surface": "...", "reading": "...", "meaning": "..."}
]
}
```

Listing 5.2: Schema phản hồi của AI Tutor

Để tối ưu hóa hiệu năng truyền tải, hệ thống chuyển giao hoàn toàn tác vụ STT và TTS về phía client thông qua Web Speech API trên trình duyệt. Khi người học tương tác bằng giọng nói, đối tượng `SpeechRecognition` sẽ nhận dạng và chuyển đổi âm thanh cục bộ thành văn bản trước khi gửi đến server. Ngược lại, trình duyệt sử dụng đối tượng `SpeechSynthesis` để chuyển văn bản phản hồi của AI thành giọng đọc tiếng Nhật tự nhiên. Thiết kế này giúp toàn bộ giao tiếp client-server ở dạng văn bản gọn nhẹ. Nếu client phát sinh lỗi thiết bị (không có microphone, trình duyệt không hỗ trợ API) hoặc mất kết nối, giao diện tự động chuyển đổi linh hoạt sang chế độ nhập liệu bằng bàn phím kèm theo thông báo hướng dẫn.

5.4.3 Kết quả đạt được

Chức năng AI Tutor hoạt động ổn định trên cả giao diện văn bản và giọng nói. Kết quả kiểm thử tích hợp (Mục 4.4.3) chứng minh hệ thống vượt qua 100% các ca kiểm thử đại diện (TUT-01 đến TUT-05), tự động xử lý tốt lỗi mất kết nối hoặc thiếu thiết bị thu âm. Đóng góp lớn nhất của AI Tutor là ràng buộc hội thoại theo cấp độ và từ vựng mục tiêu của người học, đồng thời tận dụng Web Speech API để giảm tải cho server và chuyển đổi các lỗi hội thoại thành dữ liệu ôn tập có ích. Hạn chế hiện tại là các chỉ số đánh giá phát âm chi tiết (fluency, accuracy, pronunciation) chưa được triển khai do giới hạn kỹ thuật của Web Speech API thông thường.

5.5 Xây dựng GraphRAG lựa chọn nghĩa cho dịch thuật chuyên ngành

5.5.1 Giới thiệu bài toán

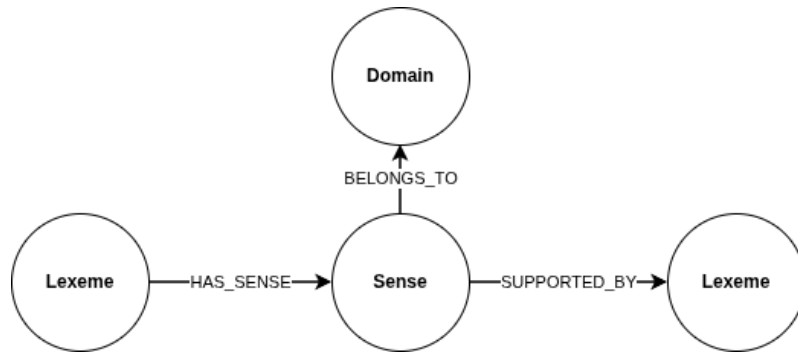
Trong dịch thuật Nhật–Việt chuyên ngành, các mô hình ngôn ngữ lớn (LLM) thường gặp lỗi lựa chọn nghĩa sai đối với các từ đa nghĩa (homonyms/polysemes), gây ảnh hưởng nghiêm trọng đến độ chính xác kỹ thuật mặc dù câu dịch vẫn đảm bảo độ trôi chảy. Khó khăn này đặc biệt rõ ràng trên các mô hình có kích thước nhỏ và chi phí thấp như `llama-3.1-8b-instant`. Để đảm bảo độ chính xác thuật ngữ chuyên môn, các giải pháp thông thường phải sử dụng các mô hình có số lượng tham số rất lớn và đắt tiền như `qwen/qwen3-32b`, điều này làm tăng đáng

kể chi phí vận hành (inference cost) và độ trễ phản hồi. Việc lưu trữ từ điển quan hệ thông thường cũng gặp khó khăn trong việc biểu diễn mối liên kết động giữa từ vựng, miền chuyên ngành và các tín hiệu ngữ cảnh trực quan (cues). Trong khi đó, việc đưa toàn bộ cơ sở dữ liệu từ điển vào prompt của mô hình là không khả thi do giới hạn độ dài ngữ cảnh và chi phí. Do đó án không có tập dữ liệu gán nhãn đủ lớn để huấn luyện một mô hình giải quyết nhập nhằng nghĩa từ (WSD) chuyên dụng, bài toán đặt ra là xây dựng một pipeline GraphRAG (Retrieval-Augmented Generation dựa trên đồ thị) gọn nhẹ để tự động truy xuất và xếp hạng các nghĩa từ phù hợp nhất từ ngữ liệu và từ điển sẵn có trước khi thực hiện dịch. Giải pháp này nhằm nâng cao chất lượng bản dịch của các mô hình kích thước nhỏ có chi phí rẻ như `llama-3.1-8b-instant` lên mức tiệm cận các mô hình lớn và đắt đỏ hơn, tối ưu hóa sự cân bằng giữa chi phí và chất lượng.

5.5.2 Giải pháp

Giải pháp gồm hai phần: xây dựng đồ thị tri thức ngoại tuyến và truy xuất–xếp hạng nghĩa khi dịch.

Xây dựng dữ liệu ngoại tuyến. Trích xuất nguồn JMdict thu được 3.321 từ đa nghĩa có nghĩa chuyên ngành thuộc 6 miền ánh xạ: `technology`, `medicine`, `academic`, `business`, `culture`, `general`. Đồ thị tri thức được mô tả trong Hình 5.2. Các tín hiệu ngữ cảnh (`SUPPORTED_BY`) được xây dựng bằng cách quét 248.678 câu tiếng Nhật từ Tatoeba qua thuật toán Aho–Corasick để đếm tần suất đồng xuất hiện. Đồ thị sau khi nạp vào Neo4j bao gồm 6 miền, 3.321 Lexeme, 9.514 Sense và 24.153 cạnh liên kết ngữ cảnh.



Hình 5.2: Mô hình đồ thị tri thức phục vụ lựa chọn nghĩa

Truy xuất và xếp hạng khi dịch. Văn bản cần dịch được phân tích cú pháp bằng SudachiPy (Mode B). Pipeline thực hiện hai lượt truy vấn Neo4j:

1. **Phát hiện miền:** Các token bỏ phiếu cho các miền chuyên ngành liên kết. Miền chuyên ngành d được giữ lại nếu số phiếu $v_d \geq \max(1; 0,3 \times v_{max})$ (với v_{max} là số phiếu cao nhất).

2. Xếp hạng nghĩa: Tính điểm cho mỗi nghĩa s theo công thức:

$$score(s) = 0,60 \times domainMatch(s) + 0,40 \times (N_m/N_t)$$

Trong đó, $domainMatch(s)$ là 1 nếu nghĩa khớp miền đã phát hiện, và N_m/N_t là tỷ lệ số tín hiệu ngữ cảnh xuất hiện thực tế trên tổng số tín hiệu của nghĩa.

Nghĩa có điểm cao nhất (top_sense) cùng ngữ cảnh bổ trợ được chèn trực tiếp làm bằng chứng đồ thị (graph evidence) vào prompt. Mô hình LLM dịch văn bản dựa trên các bằng chứng này để tạo ra bản dịch chuyên ngành chính xác và có thể kiểm chứng.

5.5.3 Kết quả đạt được

Chức năng dịch thuật chuyên sâu tích hợp GraphRAG đã được triển khai và hoạt động tốt trên môi trường thử nghiệm. Đóng góp nổi bật nhất của giải pháp là đã nâng cao chất lượng dịch thuật chuyên ngành của mô hình ngôn ngữ nhỏ, chi phí vận hành rẻ như llama-3.1-8b-instant để đạt được kết quả chính xác tương đương với các mô hình lớn hơn như qwen/qwen3-32b. Nhờ việc đưa bằng chứng đồ thị (graph evidence) có cấu trúc từ Neo4j vào prompt một cách chọn lọc, mô hình nhỏ có thể lựa chọn chính xác nghĩa chuyên ngành của thuật ngữ kỹ thuật dựa trên điểm số đánh giá trực quan rõ ràng theo miền và ngữ cảnh. Sự kết hợp này mang lại hiệu quả kinh tế lớn cho hệ thống thực tế bằng việc giảm đáng kể tài nguyên tính toán cần thiết khi suy luận mà vẫn duy trì chất lượng dịch chuyên môn ở mức tốt. Hạn chế hiện tại là hệ thống chưa có bộ dữ liệu chuẩn lớn để đánh giá định lượng chính xác tỉ lệ cải thiện chất lượng dịch, và các từ không nằm trong đồ thị vẫn phải dựa vào phán đoán mặc định của mô hình ngôn ngữ.

5.6 Tổng kết

Chương này đã làm rõ năm đóng góp chính của đề án từ khâu phát biểu bài toán, đề xuất giải pháp kỹ thuật cụ thể tới đánh giá kết quả đạt được. Các nội dung được tổng hợp trong Bảng 5.3.

Đóng góp	Vấn đề trung tâm	Giá trị đạt được
Vòng học tập tích hợp	Dữ liệu bị chia cắt giữa tra cứu, ghi nhớ và vận dụng	Tạo luồng dữ liệu hai chiều giữa từ điển, flashcard, Review và Tutor.
SM-2 bốn mức	Khác biệt giữa thang 0–5 và trải nghiệm bốn nút	Lập lịch xác định, có trạng thái trong phiên và kiểm thử được.

Đóng góp	Vấn đề trung tâm	Giá trị đạt được
AI Review Pipeline	LLM sinh nội dung không ổn định về cấu trúc	Sinh Quiz/Story cá nhân hóa và giữ phần hợp lệ khi phản hồi thiếu.
AI Tutor đa phương thức	Chatbot tổng quát không gắn mục tiêu học	Hội thoại theo JLPT/từ mục tiêu, sửa lỗi, hỗ trợ STT/TTS và lưu kết quả.
GraphRAG lựa chọn nghĩa	Thuật ngữ đa nghĩa phụ thuộc miền và ngữ cảnh	Truy xuất, xếp hạng nghĩa giúp nâng cao chất lượng dịch của LLM giá rẻ (như llama-3.1-8b-instant) ngang tầm mô hình lớn, tối ưu chi phí.

Bảng 5.3: Tổng hợp các giải pháp và đóng góp nổi bật

Đóng góp bao trùm của đề án là thiết kế một hệ thống hỗ trợ học tập thông minh, trong đó tính xác định (PostgreSQL, lập lịch SRS, quyền truy cập) do backend quản lý, còn tính linh hoạt ngôn ngữ do AI đảm nhiệm dưới sự kiểm soát chặt chẽ của các pipeline lọc và định dạng. Các kết quả kiểm thử bước đầu chứng minh tính khả thi của mô hình đề xuất và mở ra định hướng nghiên cứu sâu hơn.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương 5 đã trình bày các giải pháp và đóng góp nổi bật của đồ án. Chương này tổng kết lại quá trình thực hiện, so sánh kết quả với các sản phẩm tương tự, tự đánh giá ưu khuyết điểm, đóng góp nổi bật, rút ra bài học kinh nghiệm và đề xuất hướng phát triển tương lai.

6.1 Kết luận

6.1.1 So sánh với các sản phẩm tương tự

Khảo sát hiện trạng cho thấy các nhóm sản phẩm hiện nay (như từ điển Mazi-i/Jisho, flashcard Anki, ứng dụng học Duolingo/Bunpo, dịch thuật DeepL và các chatbot AI) đều có thể mạnh chuyên biệt rất lớn về dữ liệu và hạ tầng. Tuy nhiên, chúng thường hoạt động độc lập khiến dữ liệu học tập bị phân tán.

Sản phẩm nguyên mẫu của đồ án tuy chưa thể cạnh tranh về quy mô dữ liệu và độ hoàn thiện nhưng đã bước đầu giải quyết được hạn chế này thông qua việc xây dựng một vòng học tập tích hợp liên tục. Dữ liệu từ vựng tra cứu được chuyển tiếp làm flashcard, lên lịch ôn tập SRS, làm từ vựng mục tiêu trong Quiz, Story và AI Tutor, đồng thời lỗi phát sinh khi hội thoại lại quay về phục vụ cá nhân hóa phiên học. Ngoài ra, việc bổ sung GraphRAG với cơ sở dữ liệu đồ thị Neo4j giúp nâng cao tính chính xác và khả năng giải thích ngữ cảnh của bản dịch chuyên ngành.

6.1.2 Đánh giá kết quả thực hiện

Trong suốt quá trình thực hiện đồ án, sinh viên đã hoàn thành và nhận diện các nội dung sau:

Những nội dung đã thực hiện được:

- Phân tích và thiết kế hệ thống; xây dựng các chức năng cơ bản (xác thực, tra cứu từ điển, quản lý bộ thẻ, flashcard).
- Tích hợp và điều chỉnh thuật toán lặp lại ngắt quãng SM-2 sang thang đánh giá bốn mức độ ghi nhớ.
- Thiết lập AI Review Pipeline để tự động sinh câu hỏi Quiz, Story và AI Tutor tương tác thoại tiếng Nhật có phản hồi sửa lỗi.
- Xây dựng quy trình GraphRAG kết nối Neo4j nhằm gợi ý nghĩa thuật ngữ chuyên ngành dựa trên miền tri thức ngữ cảnh.
- Thực hiện tích hợp hệ thống trên môi trường cục bộ và đánh giá kịch bản qua kiểm thử.

Những nội dung chưa thực hiện được và hạn chế:

- Nguyên mẫu chỉ chạy thử nghiệm ở localhost, chưa triển khai môi trường cloud để đánh giá độ ổn định tải và hiệu năng thực tế.
- Dữ liệu từ điển và đồ thị tri thức Neo4j còn giới hạn độ phủ, chưa bao quát được nhiều lĩnh vực chuyên sâu.
- Chưa tổ chức thử nghiệm lâm sàng dài hạn với người học và thiếu tập dữ liệu đối chứng định lượng để chứng minh tỷ lệ cải thiện chất lượng dịch của GraphRAG.

6.1.3 Đóng góp nổi bật và bài học kinh nghiệm

Đóng góp nổi bật của đề án:

- Thiết kế và hiện thực thành công vòng học tập tích hợp dữ liệu liên tục giữa các tác vụ tra cứu, ôn tập và luyện giao tiếp.
- Gắn lỗi hội thoại và từ vựng mục tiêu thành dữ liệu đầu vào cho hoạt động ôn tập thông minh của AI.
- Xây dựng pipeline AI có ràng buộc cấu trúc (parser, validation) và GraphRAG phân tích nghĩa chuyên ngành có khả năng giải thích.

Bài học kinh nghiệm rút ra:

- Khi làm việc với LLM, luôn cần thiết kế cơ chế kiểm soát lỗi, parser kết quả và các phương án dự phòng khi dịch vụ AI gặp sự cố.
- Tách biệt rõ ràng vai trò: các logic nghiệp vụ cố định (xác thực, lịch SRS) phải do backend xử lý; AI chỉ đóng vai trò sinh ngữ cảnh.
- Kiểm thử tích hợp cần triển khai sớm để phát hiện bất cập trong cấu hình phân quyền bảo mật của các luồng đa phương thức.

6.2 Hướng phát triển

Định hướng phát triển sản phẩm trong tương lai được phân chia thành hai giai đoạn rõ rệt: hoàn thiện các chức năng hiện tại và nghiên cứu mở rộng các hướng đi mới.

6.2.1 Hoàn thiện các chức năng hiện tại

- **Khắc phục bảo mật và kiểm thử:** Thống nhất quyền truy cập tệp âm thanh TTS với Spring Security và tăng độ bao phủ của unit/integration test.
- **Tối ưu hóa pipeline AI:** Thêm giới hạn thời gian phản hồi (timeout), cơ chế tự động thử lại, và tinh chỉnh chất lượng Quiz/Story thông qua tập đánh giá của chuyên gia.
- **Nâng cấp dữ liệu và triển khai:** Làm sạch cơ sở dữ liệu từ điển, mở rộng các nút quan hệ đồ thị Neo4j, đồng thời đóng gói Docker và triển khai ứng dụng lên môi trường production (Cloud).

6.2.2 Nâng cấp và mở rộng hướng nghiên cứu mới

- **Cá nhân hóa thích ứng:** Xây dựng mô hình năng lực người học dựa trên tần suất tương tác, thời gian phản hồi để gợi ý bài học phù hợp.
- **Nâng cấp AI Tutor thoại:** Tích hợp cơ chế đánh giá phát âm trực tiếp từ âm thanh người học (thay vì qua văn bản transcript) và đa dạng hóa kịch bản giao tiếp công sở.
- **Nghiên cứu GraphRAG chuyên sâu:** Tổ chức kiểm thử đối chứng định lượng chất lượng dịch thuật và mở rộng đồ thị tri thức sang các miền kỹ thuật khác như y tế, kinh tế, cơ khí.

TÀI LIỆU THAM KHẢO

- [1] Vue.js Core Team. “Vue.js documentation.” (), [Online]. Available: <https://vuejs.org/guide/introduction.html> (visited on 06/07/2026).
- [2] Vite Team. “Vite guide.” (), [Online]. Available: <https://vite.dev/guide/> (visited on 06/07/2026).
- [3] Pinia Team. “Pinia documentation.” (), [Online]. Available: <https://pinia.vuejs.org/> (visited on 06/07/2026).
- [4] Axios Project. “Axios documentation.” (), [Online]. Available: <https://axios-http.com/docs/intro> (visited on 06/07/2026).
- [5] Tailwind Labs. “Tailwind css documentation.” (), [Online]. Available: <https://tailwindcss.com/docs> (visited on 06/07/2026).
- [6] Spring Team. “Spring boot reference documentation.” (), [Online]. Available: <https://docs.spring.io/spring-boot/reference/index.html> (visited on 06/07/2026).
- [7] Spring Team. “Spring security reference.” (), [Online]. Available: <https://docs.spring.io/spring-security/reference/> (visited on 06/07/2026).
- [8] M. Jones, J. Bradley, and N. Sakimura, “Json web token (JWT),” Internet Engineering Task Force, RFC 7519, 2015.
- [9] PostgreSQL Global Development Group. “Postgresql 16 documentation.” (), [Online]. Available: <https://www.postgresql.org/docs/16/> (visited on 06/07/2026).
- [10] S. Ramirez. “Fastapi documentation.” (), [Online]. Available: <https://fastapi.tiangolo.com/> (visited on 06/07/2026).
- [11] Google LLC. “Gemini api documentation.” (), [Online]. Available: <https://ai.google.dev/gemini-api/docs> (visited on 06/07/2026).
- [12] A. Yang, A. Li, B. Yang, *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [13] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems 33*, Curran Associates, Inc., 2020, pp. 9459–9474.
- [14] Works Applications Co., Ltd. “Sudachipy documentation.” (), [Online]. Available: <https://worksapplications.github.io/sudachi.rs/python/> (visited on 06/07/2026).
- [15] Neo4j, Inc. “Cypher manual, neo4j 5.” (), [Online]. Available: <https://neo4j.com/docs/cypher-manual/5/> (visited on 06/07/2026).

- [16] P. A. Wozniak, “Optimization of learning,” M.S. thesis, Poznan University of Technology, Poznan, Poland, 1990.
- [17] E. Liu. “Web speech api,” World Wide Web Consortium. (May 21, 2026), [Online]. Available: <https://webaudio.github.io/web-speech-api/>.
- [18] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, Honolulu, Hawaii, USA, 2023, pp. 28 492–28 518.